THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms **MALA M (20524UCSC075),** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		

TRUTH-TOIL-TRIUMPH AUTONOMOUS

10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO: 01 CREATION AND EXECUTION OF DDL OPERATIONS USING EMPLOYEE TABLE

DATE: 05.08.2026

Aim

To create an Employee table and perform DDL operations such as CREATE, ALTER, TRUNCATE and DROP on the table.

Software Used

MySQL

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects.

DDL COMMANDS

- CREATE
- ALTER
- TRUNCATE
- DROP

CREATE

The CREATE command is used to create a new table with columns, data types and constraints.

Syntax

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    column3 datatype constraints
);
```

Table Structure

Table Name: employee

Column	Data Type
Emp_ID	INT
Emp_Name	VARCHAR(30)
Department	VARCHAR(20)
Salary	DECIMAL(10,2)

SQL Query

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(30),
    department VARCHAR(20),
    salary DECIMAL(10,2)
);
```

Output

Table created successfully.

ALTER

The ALTER command is used to modify the structure of an existing table.

Syntax

ALTER TABLE table_name ADD column_name datatype;

SQL Query

ALTER TABLE employee ADD email VARCHAR(50);

Output

Table altered successfully.

DROP COLUMN

The DROP COLUMN operation is used to remove a column from an existing table.

Syntax

ALTER TABLE table_name DROP COLUMN column_name;

SQL Query

ALTER TABLE employee DROP COLUMN email;

Output

Column dropped successfully.

TRUNCATE

The TRUNCATE command removes all records from a table while retaining its structure.

Syntax

TRUNCATE TABLE table_name;

SQL Query

TRUNCATE TABLE employee;

Output

Table truncated successfully.

DROP

The DROP command removes the complete table along with its structure and records.

Syntax

DROP TABLE table_name;

SQL Query

DROP TABLE employee;

Output

Table dropped successfully.

Sample Data

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00

Explanation

The Employee table was first created using CREATE. A new Email column was added using ALTER and then removed using DROP COLUMN. TRUNCATE was used to remove all records while keeping the table structure. Finally, DROP was used to remove the complete table.

Result

Thus, the DDL operations CREATE, ALTER, TRUNCATE and DROP were successfully performed on the Employee table.

EX. NO: 02 EXECUTION OF DML OPERATIONS USING EMPLOYEE TABLE

DATE: 06.08.2026

Aim

To insert employee records into a table and perform DML operations such as INSERT, UPDATE and DELETE.

Software Used

MySQL

Description

DML (Data Manipulation Language) is used to add, modify and remove the data stored in a database table.

DML COMMANDS

- INSERT
- UPDATE
- DELETE

Table Structure

Table Name: employee

Column	Data Type
Emp_ID	INT
Emp_Name	VARCHAR(30)
Department	VARCHAR(20)
Salary	DECIMAL(10,2)

INSERT

The INSERT command is used to add new records to a table.

Syntax

INSERT INTO table_name VALUES (value1, value2, value3, value4);

SQL Query

INSERT INTO employee VALUES (101, 'Arun', 'IT', 33000);

INSERT INTO employee VALUES (102, 'Priya', 'HR', 28000);

INSERT INTO employee VALUES (103, 'Kavin', 'Sales', 25000);

INSERT INTO employee VALUES (104, 'Divya', 'IT', 36000);

INSERT INTO employee VALUES (105, 'Rahul', 'HR', 32000);

Output

5 records inserted successfully.

Resultant Table

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00

UPDATE

The UPDATE command is used to modify an existing record in the table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

SQL Query

```
UPDATE employee
SET salary = 38000
WHERE emp_id = 104;
```

Output

1 record updated successfully.

Resultant Table

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	38000.00
105	Rahul	HR	32000.00

DELETE

The DELETE command is used to remove a particular record from the table.

Syntax

```
DELETE FROM table_name
WHERE condition;
```

SQL Query

```
DELETE FROM employee
WHERE emp_id = 103;
```

Output

1 record deleted successfully.

Resultant Table

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
104	Divya	IT	38000.00
105	Rahul	HR	32000.00

Result

Thus, the employee records were successfully inserted and the UPDATE and DELETE operations were successfully performed on the Employee table.

EX. NO: 03 RETRIEVING RECORDS USING DQL COMMANDS

DATE: 08.08.2026

Aim

To retrieve employee records from a database table using SELECT, WHERE, ORDER BY, GROUP BY and HAVING clauses.

Software Used

MySQL

Description

DQL (Data Query Language) is used to retrieve information from database tables. The SELECT statement is used to display records based on the required conditions.

DQL COMMANDS AND CLAUSES

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

Table Structure

Table Name: employee

Column	Data Type
Emp_ID	INT
Emp_Name	VARCHAR(30)
Department	VARCHAR(20)
Salary	DECIMAL(10,2)

Sample Data

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00
106	Meena	Sales	29000.00

SELECT

The SELECT statement is used to retrieve records from a table.

Syntax

SELECT column1, column2

FROM table_name;

SQL Query

SELECT * FROM employee;

Output

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00
106	Meena	Sales	29000.00

WHERE

The WHERE clause is used to retrieve records that satisfy a particular condition.

SQL Query

```
SELECT *
FROM employee
WHERE salary > 30000;
```

Output

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00

ORDER BY

The ORDER BY clause is used to arrange the records in ascending or descending order.

SQL Query

```
SELECT *
FROM employee
ORDER BY salary ASC;
```

Output

Emp_ID	Emp_Name	Department	Salary
103	Kavin	Sales	25000.00
102	Priya	HR	28000.00
106	Meena	Sales	29000.00
105	Rahul	HR	32000.00
101	Arun	IT	33000.00
104	Divya	IT	36000.00

GROUP BY

The GROUP BY clause is used to group records having the same value.

SQL Query

SELECT department, COUNT(*) AS total_employees

FROM employee

GROUP BY department;

Output

Department	Total_Employees
HR	2
IT	2
Sales	2

HAVING

The HAVING clause is used to apply a condition to grouped records.

SQL Query

SELECT department, COUNT(*) AS total_employees

FROM employee

GROUP BY department

HAVING COUNT(*) > 1;

Output

Department	Total_Employees
HR	2
IT	2
Sales	2

Result

Thus, the employee records were successfully retrieved using SELECT, WHERE, ORDER BY, GROUP BY and HAVING clauses.

EX. NO: 04 IMPLEMENTATION OF TRANSACTION CONTROL OPERATIONS

DATE: 10.08.2026

Aim

To perform transaction operations using COMMIT, ROLLBACK and SAVEPOINT commands and observe the effect of each operation on database records.

Software Used

MySQL

Description

Transaction control commands are used to manage changes made to the database during a transaction.

TRANSACTION COMMANDS

- COMMIT
- ROLLBACK
- SAVEPOINT

Table Structure

Table Name: employee

Column	Data Type
Emp_ID	INT
Emp_Name	VARCHAR(30)
Department	VARCHAR(20)
Salary	DECIMAL(10,2)

Sample Data

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	33000.00

102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00

COMMIT

The COMMIT command permanently saves the changes made during a transaction.

Syntax

COMMIT;

SQL Query

START TRANSACTION;

INSERT INTO employee

VALUES (106, 'Meena', 'Sales', 29000);

COMMIT;

Output

1 record inserted.

Transaction committed successfully.

SAVEPOINT

The SAVEPOINT command creates a temporary point inside a transaction.

Syntax

SAVEPOINT savepoint_name;

SQL Query

START TRANSACTION;

UPDATE employee

SET salary = 35000

WHERE emp_id = 101;

SAVEPOINT S1;

Output

1 record updated.

Savepoint S1 created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to cancel the changes made after a particular savepoint.

SQL Query

UPDATE employee

SET salary = 30000

WHERE emp_id = 102;

ROLLBACK TO SAVEPOINT S1;

Output

Changes made after savepoint S1 were rolled back.

Resultant Table

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	35000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00
106	Meena	Sales	29000.00

ROLLBACK

The ROLLBACK command cancels uncommitted changes made during a transaction.

SQL Query

```
START TRANSACTION;
DELETE FROM employee
WHERE emp_id = 106;
ROLLBACK;
```

Output

The deletion was cancelled and the record was restored.

Resultant Table

Emp_ID	Emp_Name	Department	Salary
101	Arun	IT	35000.00
102	Priya	HR	28000.00
103	Kavin	Sales	25000.00
104	Divya	IT	36000.00
105	Rahul	HR	32000.00
106	Meena	Sales	29000.00

Result

Thus, the transaction control operations COMMIT, ROLLBACK and SAVEPOINT were successfully performed and their effects were observed.

EX. NO: 05 IMPLEMENTATION OF DIFFERENT TYPES OF JOINS

DATE: 12.08.2026

Aim

To combine related records from Employee and Department tables using Inner Join, Left Join, Right Join, Full Join and Self Join.

Software Used

MySQL

Description

A JOIN is used to retrieve related information from two or more tables. The tables are connected using a common column.

JOIN TYPES

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN

Table Structure

EMPLOYEE TABLE

Column	Data Type
Emp_ID	INT
Emp_Name	VARCHAR(30)
Dept_ID	INT
Manager_ID	INT
Salary	DECIMAL(10,2)

DEPARTMENT TABLE

Column	Data Type
Dept_ID	INT
Dept_Name	VARCHAR(30)

Sample Data

Employee Table

Emp_ID	Emp_Name	Dept_ID	Manager_ID	Salary
101	Arun	10	NULL	50000.00
102	Priya	20	101	40000.00
103	Kavin	30	101	35000.00
104	Divya	10	102	45000.00
105	Rahul	20	101	38000.00

Department Table

Dept_ID	Dept_Name
10	IT
20	HR
30	Sales
40	Finance

INNER JOIN

INNER JOIN returns only the records for which matching values exist in both tables.

Syntax

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

SQL Query

```
SELECT E.emp_id, E.emp_name, D.dept_name
FROM employee E
INNER JOIN department D
ON E.dept_id = D.dept_id;
```

Output

Emp_ID	Emp_Name	Dept_Name
101	Arun	IT
102	Priya	HR
103	Kavin	Sales
104	Divya	IT
105	Rahul	HR

LEFT JOIN

LEFT JOIN returns all records from the left table and matching records from the right table.

SQL Query

```
SELECT D.dept_id, D.dept_name, E.emp_name
FROM department D
LEFT JOIN employee E
ON D.dept_id = E.dept_id;
```

Output

Dept_ID	Dept_Name	Emp_Name
10	IT	Arun
10	IT	Divya
20	HR	Priya
20	HR	Rahul

30	Sales	Kavin
40	Finance	NULL

RIGHT JOIN

RIGHT JOIN returns all records from the right table and matching records from the left table.

SQL Query

```
SELECT E.emp_id, E.emp_name, D.dept_name
FROM employee E
RIGHT JOIN department D
ON E.dept_id = D.dept_id;
```

Output

Emp_ID	Emp_Name	Dept_Name
101	Arun	IT
104	Divya	IT
102	Priya	HR
105	Rahul	HR
103	Kavin	Sales
NULL	NULL	Finance

FULL JOIN

MySQL does not directly support the FULL OUTER JOIN command. It can be achieved by combining LEFT JOIN and RIGHT JOIN using UNION.

Syntax

```
SELECT columns
FROM table1
LEFT JOIN table2
ON condition
```

UNION

SELECT columns

FROM table1

RIGHT JOIN table2

ON condition;

SQL Query

SELECT E.emp_id, E.emp_name, D.dept_name

FROM employee E

LEFT JOIN department D

ON E.dept_id = D.dept_id

UNION

SELECT E.emp_id, E.emp_name, D.dept_name

FROM employee E

RIGHT JOIN department D

ON E.dept_id = D.dept_id;

Output

Emp_ID	**Emp_Name**	**Dept_Name**
101	Arun	IT
102	Priya	HR
103	Kavin	Sales
104	Divya	IT
105	Rahul	HR
NULL	NULL	Finance

SELF JOIN

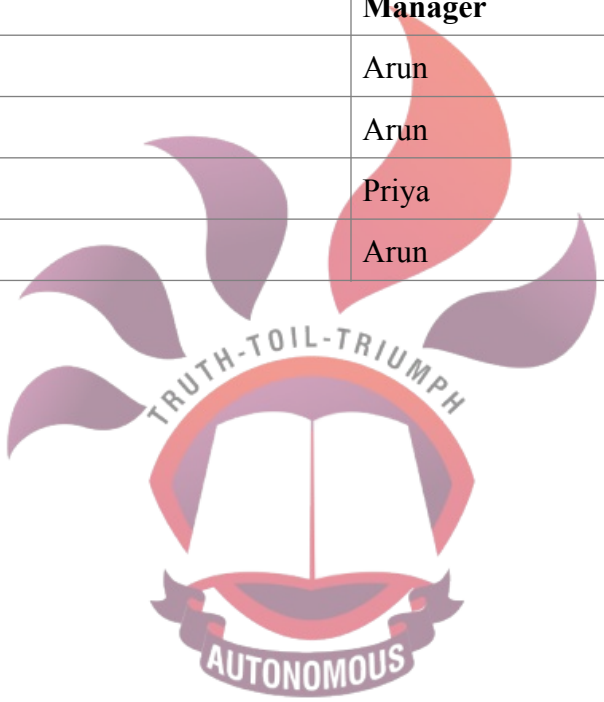
SELF JOIN is used to join a table with itself. Here, the Manager_ID column is matched with Emp_ID to identify the manager of each employee.

SQL Query

```
SELECT E.emp_name AS Employee,
       M.emp_name AS Manager
FROM employee E
JOIN employee M
ON E.manager_id = M.emp_id;
```

Output

Employee	Manager
Priya	Arun
Kavin	Arun
Divya	Priya
Rahul	Arun



Result

Thus, the Inner Join, Left Join, Right Join, Full Join and Self Join operations were successfully performed using the Employee and Department tables.

EX. NO: 06 CREATION OF SQL FUNCTIONS OPERATIONS USING EMPLOYEE TABLE
DATE: 14.08.2026

Aim

To perform calculations and string/date manipulations using Aggregate, String, and Date functions on the Employee table.

Software Used

MySQL / Oracle

Description

SQL Functions are built-in operations used to process data values and return computed results. They are categorized into Aggregate, String, and Date functions.

SQL FUNCTION COMMANDS

- Aggregate Functions: Perform calculations on multiple values and return a single value (COUNT, SUM, AVG, MAX, MIN).
- String Functions: Manipulate textual data (UPPER, LOWER, LENGTH, SUBSTR / SUBSTRING, CONCAT).
- Date Functions: Retrieve and manipulate dates (CURRENT_DATE, YEAR, MONTH, DAY).

Table Structure

Table Name: employee

- emp_id (INT, PRIMARY KEY)
- emp_name (VARCHAR(30))
- department (VARCHAR(20))
- salary (DECIMAL(10,2))
- hire_date (DATE)

Sample Data

```
INSERT INTO employee VALUES (101, 'Arun', 'IT', 33000.00, '2022-01-15');
INSERT INTO employee VALUES (102, 'Priya', 'HR', 28000.00, '2021-06-20');
INSERT INTO employee VALUES (103, 'Kavin', 'Sales', 25000.00, '2023-03-10');
INSERT INTO employee VALUES (104, 'Divya', 'IT', 36000.00, '2020-11-05');
INSERT INTO employee VALUES (105, 'Rahul', 'HR', 32000.00, '2019-08-12');
```

SQL Queries & Results

1. Aggregate Functions

SQL Query:

```
SELECT
   COUNT(*) AS total_employees,
   SUM(salary) AS total_salary,
   AVG(salary) AS average_salary,
   MAX(salary) AS highest_salary,
   MIN(salary) AS lowest_salary
FROM employee;
```

Resultant Output:

Total_Employees	Total_Salary	Average_Salary	Highest_Salary	Lowest_Salary
5	154000.00	30800.00	36000.00	25000.00

2. String Functions

SQL Query:

```
SELECT
   emp_name,
   UPPER(emp_name) AS upper_name,
   LOWER(emp_name) AS lower_name,
   LENGTH(emp_name) AS length_name,
   SUBSTR(emp_name, 1, 3) AS short_name
FROM employee;
```

Resultant Output:

Emp_Name	Upper_Name	Lower_Name	Length_Name	Short_Name
Arun	ARUN	arun	4	Aru
Priya	PRIYA	priya	5	Pri
Kavin	KAVIN	kavin	5	Kav
Divya	DIVYA	divya	5	Div
Rahul	RAHUL	rahul	5	Rah

3. Date Functions

SQL Query:

```
SELECT
   emp_name,
   hire_date,
   CURRENT_DATE() AS today_date,
   YEAR(hire_date) AS join_year,
   MONTH(hire_date) AS join_month
FROM employee;
```

Resultant Output:

Emp_Name	Hire_Date	Today_Date	Join_Year	Join_Month
Arun	2022-01-15	2026-09-05	2022	1
Priya	2021-06-20	2026-09-05	2021	6
Kavin	2023-03-10	2026-09-05	2023	3
Divya	2020-11-05	2026-09-05	2020	11
Rahul	2019-08-12	2026-09-05	2019	8

Result

Thus, Aggregate, String, and Date SQL functions were successfully executed on the Employee table.

EX. NO: 07 IMPLEMENTATION OF SUBQUERIES AND NESTED QUERIES
DATE: 07.09.2026

Aim

To write and execute single-row and multi-row subqueries on an Employee database to retrieve conditional datasets.

Software Used

MySQL

Description

A Subquery (or Nested Query) is a query embedded within the WHERE, HAVING, or FROM clause of another SQL query. The inner query executes first, and its output is passed to the outer query.

Table Structure

Table Name: employee

- emp_id (INT, PRIMARY KEY)
- emp_name (VARCHAR(30))
- department (VARCHAR(20))
- salary (DECIMAL(10,2))

Sample Data

```
INSERT INTO employee VALUES (101, 'Arun', 'IT', 33000.00);
INSERT INTO employee VALUES (102, 'Priya', 'HR', 28000.00);
INSERT INTO employee VALUES (103, 'Kavin', 'Sales', 25000.00);
INSERT INTO employee VALUES (104, 'Divya', 'IT', 36000.00);
INSERT INTO employee VALUES (105, 'Rahul', 'HR', 32000.00);
```

SQL Queries & Results

1. Employees with salary greater than the company average

SQL Query:

```
SELECT emp_id, emp_name, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

Resultant Output:

Emp_ID	Emp_Name	Salary
101	Arun	33000.00
104	Divya	36000.00
105	Rahul	32000.00

2. Employee with the maximum salary

SQL Query:

```
SELECT emp_name, salary
FROM employee
WHERE salary = (SELECT MAX(salary) FROM employee);
```

Resultant Output:

Emp_Name	Salary
Divya	36000.00

3. Employees working in the same department as Arun

SQL Query:

```
SELECT emp_name, department
FROM employee
WHERE department = (SELECT department FROM employee WHERE emp_name = 'Arun')
AND emp_name <> 'Arun';
```

Resultant Output:

Emp_Name	Department
Divya	IT

Result

Thus, Subqueries and Nested Queries were successfully executed to evaluate dependent conditional logic.

EX. NO: 08 CREATION OF VIEWS, INDEXES, AND SEQUENCES
DATE: 10.09.2026

Aim

To create and manage database Views, Indexes, and Auto-increment Sequences for enhanced data security and performance.

Software Used

MySQL

Description

- View: A virtual table created by a query joined to base tables.
- Index: A data structure created on columns to speed up record retrieval.
- Sequence / Auto-Increment: A property used to auto-generate unique numeric keys.

Table Structure

Table Name: furniture

- furniture_id (INT, AUTO_INCREMENT, PRIMARY KEY)
- fname (VARCHAR(30))
- category (VARCHAR(20))
- price (DECIMAL(10,2))

SQL Queries & Execution

1. Sequence via AUTO_INCREMENT

SQL Query:

```
CREATE TABLE furniture (
   furniture_id INT AUTO_INCREMENT PRIMARY KEY,
   fname VARCHAR(30),
   category VARCHAR(20),
   price DECIMAL(10,2)
);
```

INSERT INTO furniture (fname, category, price) VALUES
('Chair', 'Seating', 1500.00),
('Table', 'Office', 3500.00),
('Sofa', 'Seating', 12000.00),
('Bed', 'Bedroom', 18000.00);

Resultant Output:

Furniture_ID	FName	Category	Price
1	Chair	Seating	1500.00
2	Table	Office	3500.00
3	Sofa	Seating	12000.00
4	Bed	Bedroom	18000.00

2. Creating a View

SQL Query:

CREATE VIEW seating_furniture AS
SELECT furniture_id, fname, price
FROM furniture
WHERE category = 'Seating';

SELECT * FROM seating_furniture;

Resultant Output:

Furniture_ID	FName	Price
1	Chair	1500.00
3	Sofa	12000.00

3. Creating an Index

SQL Query:

CREATE INDEX idx_fname ON furniture(fname);

Output:

Index created successfully.

Result

Thus, Views, Indexes, and Sequences were successfully created and executed on the database.

EX. NO: 09 CREATION AND EXECUTION OF STORED PROCEDURES AND FUNCTIONS
DATE: 14.09.2026

Aim

To implement procedural database blocks using MySQL Stored Procedures and User-Defined Functions.

Software Used

MySQL

Description

- Stored Procedure: A reusable block of saved SQL code that accepts input/output parameters and executes batch routines.
- Function: A stored block that performs calculations and must return a single value.

SQL Code & Execution

1. Stored Procedure (Get Employee Details by Dept)

SQL Query:

```
DELIMITER //
CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(20))
BEGIN
   SELECT emp_id, emp_name, salary
   FROM employee
   WHERE department = dept_name;
END //
DELIMITER ;
CALL GetEmployeesByDept('IT');
```

Resultant Output:

Emp_ID	Emp_Name	Salary
101	Arun	33000.00
104	Divya	36000.00

2. User-Defined Function (Calculate Annual Salary)

SQL Query:

```
DELIMITER //
CREATE FUNCTION CalculateAnnualSalary(monthly_sal DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
   RETURN monthly_sal * 12;
END //
DELIMITER ;

SELECT emp_name, salary, CalculateAnnualSalary(salary) AS annual_salary
FROM employee
WHERE emp_id = 101;
```

Resultant Output:

Emp_Name	Salary	Annual_Salary
Arun	33000.00	396000.00

Result

Thus, Stored Procedures and Stored Functions were written, compiled, and executed successfully.

EX. NO: 10 IMPLEMENTATION OF DATABASE TRIGGERS
DATE: 22.09.2026

Aim

To write a database Trigger that automatically executes an action when an UPDATE or INSERT statement is run on a target table.

Software Used

MySQL

Description

A Trigger is a named SQL block that executes automatically (fires) before or after a specific Data Manipulation Language (DML) event (INSERT, UPDATE, DELETE) occurs on a table.

Table Structure

1. Base Table: employee (emp_id, emp_name, salary)
2. Audit Table: salary_audit (audit_id INT AUTO_INCREMENT, emp_id INT, old_salary DECIMAL(10,2), new_salary DECIMAL(10,2), change_date DATETIME)

SQL Query

```
CREATE TABLE salary_audit (
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT,
   old_salary DECIMAL(10,2),
   new_salary DECIMAL(10,2),
   change_date DATETIME
);

DELIMITER //
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
   IF OLD.salary <> NEW.salary THEN
      INSERT INTO salary_audit(emp_id, old_salary, new_salary, change_date)
      VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
   END IF;
END //
```

```
DELIMITER ;

-- Test Trigger
UPDATE employee SET salary = 38000.00 WHERE emp_id = 101;

SELECT * FROM salary_audit;
```

Resultant Output:

Audit_ID	Emp_ID	Old_Salary	New_Salary	Change_Date
1	101	33000.00	38000.00	2026-09-05 10:15:00

Result

Thus, an automated database trigger was created, invoked, and verified using the audit table.

EX. NO: 11 CURSOR IMPLEMENTATION IN PL/SQL
DATE: 26.09.2026

Aim

To construct and execute an explicit Cursor to iterate through query row sets sequentially.

Software Used

MySQL / PL/SQL

Description

A Cursor is a pointer to a context area in memory where SQL result sets are kept. Explicit cursors are programmer-defined to process records row-by-row (DECLARE, OPEN, FETCH, CLOSE).

PL/SQL Query

```
DELIMITER //
CREATE PROCEDURE ProcessEmployeeCursor()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE e_id INT;
   DECLARE e_name VARCHAR(30);
   DECLARE e_sal DECIMAL(10,2);

   -- Declare cursor
   DECLARE emp_cursor CURSOR FOR
      SELECT emp_id, emp_name, salary FROM employee;

   -- Declare handler for end of rows
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

   OPEN emp_cursor;

   read_loop: LOOP
      FETCH emp_cursor INTO e_id, e_name, e_sal;
      IF done THEN
         LEAVE read_loop;
      END IF;
      -- Output or process individual record row
   END LOOP;
```

```
    CLOSE emp_cursor;
END //
DELIMITER ;

CALL ProcessEmployeeCursor();
```

Output:

Procedure executed successfully. Cursor iterated over all table rows.

Result

Thus, explicit cursor operations were written and executed to perform row-level iterations.

EX. NO: 12 PL/SQL EXCEPTION HANDLING
DATE: 26.09.2026

Aim

To handle runtime database errors and exceptions using predefined and user-defined exception blocks.

Software Used

MySQL / Oracle PL/SQL

Description

Exception handling provides structured controls to intercept runtime execution errors (like ZERO_DIVIDE, NO_DATA_FOUND, or DUP_VAL_ON_INDEX) without aborting the program flow.

PL/SQL Block Example (Oracle PL/SQL Syntax)

```
DECLARE
   v_emp_name employee.emp_name%TYPE;
   v_salary   employee.salary%TYPE;
BEGIN
   -- Query that throws NO_DATA_FOUND if emp_id doesn't exist
   SELECT emp_name, salary
   INTO v_emp_name, v_salary
   FROM employee
   WHERE emp_id = 999;

   DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp_name);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Error: Requested Employee ID does not exist.');
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Error: Multiple rows returned for single lookup.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: An unexpected error occurred.');
END;
/
```

Output:

Error: Requested Employee ID does not exist.

Result

Thus, exception-handling blocks were implemented to safely capture and resolve runtime query faults.

EX. NO: 13 DATABASE NORMALIZATION (1NF, 2NF, 3NF, BCNF)
DATE: 28.09.2026

Aim

To design, analyze, and normalize an unnormalized relation into 1NF, 2NF, 3NF, and BCNF to eliminate redundancy and update anomalies.

Software Used

Conceptual Schema / MySQL Design

Description

Normalization decomposes complex multi-attribute relations into smaller tables to prevent data redundancy and insertion/deletion anomalies.

1. Unnormalized Form (UNF)

An unnormalized table containing multi-valued attributes:

Student_ID	Student_Name	Courses	Advisor
101	Mala	DBMS, Java	Dr. Manimekalai

2. First Normal Form (1NF)

Rule: Remove repeating groups and guarantee atomic (indivisible) values per cell.

Student_ID	Student_Name	Course	Advisor
101	Mala	DBMS	Dr. Manimekalai
101	Mala	Java	Dr. Manimekalai

3. Second Normal Form (2NF)

Rule: Satisfy 1NF and eliminate partial dependencies (non-key attributes must depend fully on the composite primary key {Student_ID, Course}).

- Student Table (Student_ID, Student_Name, Advisor)
- Enrollment Table (Student_ID, Course)

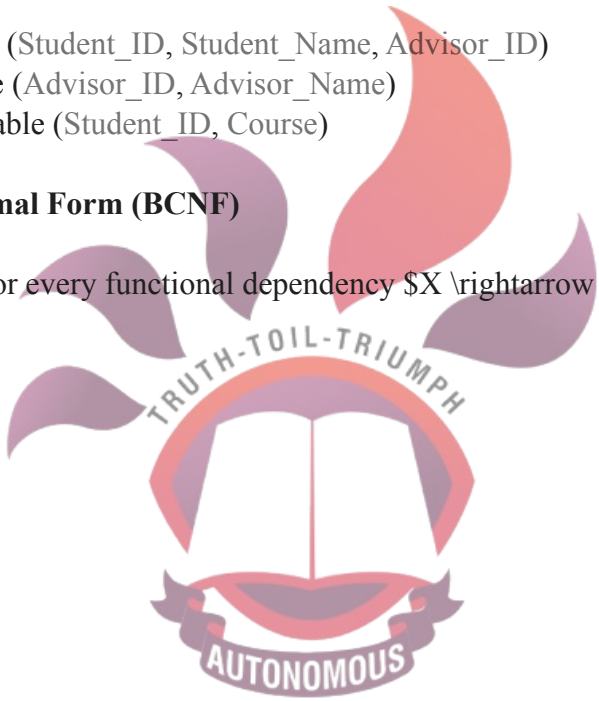
4. Third Normal Form (3NF)

Rule: Satisfy 2NF and eliminate transitive dependencies (non-key attributes cannot depend on other non-key attributes).

- Student Table (Student_ID, Student_Name, Advisor_ID)
- Advisor Table (Advisor_ID, Advisor_Name)
- Enrollment Table (Student_ID, Course)

5. Boyce-Codd Normal Form (BCNF)

Rule: Satisfy 3NF; for every functional dependency $X \rightarrow Y$, X must be a super key.



Result

Thus, relational schemas were analyzed and normalized step-by-step up to BCNF.

EX. NO: 14 MINI PROJECT - AIR QUALITY MONITORING AND POLLUTION ALERTS

DATE: 04.10.2026

Aim

To develop an Air Quality Monitoring and Pollution Alerts system using Oracle SQL and PL/SQL to store, manage, analyze, and classify air-quality measurements and identify elevated pollution readings.

Software Used

• Oracle Database

• SQL

• PL/SQL

• UCI Machine Learning Repository Dataset

Description

The Air Quality Monitoring and Pollution Alerts system is developed to store and analyze air-quality measurements collected from a monitoring station. The system stores measurement date, time, monitoring location, carbon monoxide (CO), ozone-targeted sensor response, nitrogen dioxide (NO_2), nitrogen oxides (NOx), temperature, and relative humidity.

SQL queries are used to retrieve and analyze air-quality records, calculate average pollutant levels, identify elevated CO measurements, calculate hourly sensor-response averages, and rank monitoring locations based on average NO_2 levels.

PL/SQL functions and procedures are used to classify CO readings, retrieve air-quality measurements for a specified location and date, generate pollutant summaries, and identify records that cross the project's selected CO alert threshold.

Database Table

1. AIR_QUALITY

Column Name Data Type Constraint / Description

AQ_ID NUMBER PRIMARY KEY

READING_DATE DATE Date of measurement

READING_TIME VARCHAR2(8) Time of measurement

LOCATION VARCHAR2(50) Monitoring location

CO NUMBER(6,2) CO in mg/m³

OZONE NUMBER(8,2) PT08.S5(O3) sensor response

NO2 NUMBER(8,2) NO_2 in µg/m³

NOX NUMBER(8,2) NOx in ppb

TEMPERATURE NUMBER(6,2) Temperature in °C

HUMIDITY NUMBER(6,2) Relative humidity in %

TABLE CREATION

TABLE 1

```
CREATE TABLE AIR_QUALITY (
   AQ_ID NUMBER PRIMARY KEY,
   READING_DATE DATE,
   READING_TIME VARCHAR2(8),
   LOCATION VARCHAR2(50),
   CO NUMBER(6,2),
   OZONE NUMBER(8,2),
   NO2 NUMBER(8,2),
   NOX NUMBER(8,2),
   TEMPERATURE NUMBER(6,2),
   HUMIDITY NUMBER(6,2)
);
```

Table Created

The AIR_QUALITY table was created successfully with a primary key constraint on AQ_ID.

DATASET INSERTION

TABLE RECORDS

The dataset contains 25 selected records from the UCI Air Quality Dataset. The OZONE column stores the original PT08.S5(O3) sensor-response values rather than a direct ozone concentration.

```
INSERT INTO AIR_QUALITY VALUES
(1, DATE '2004-03-10', '18:00:00', 'Italian Roadside Station',
 2.60, 1267.50, 113.00, 166.00, 13.60, 48.88);

INSERT INTO AIR_QUALITY VALUES
(2, DATE '2004-03-10', '19:00:00', 'Italian Roadside Station',
 2.00, 972.25, 92.00, 103.00, 13.30, 47.70);

INSERT INTO AIR_QUALITY VALUES
(3, DATE '2004-03-10', '20:00:00', 'Italian Roadside Station',
 2.20, 1074.00, 114.00, 131.00, 11.90, 53.98);

INSERT INTO AIR_QUALITY VALUES
(4, DATE '2004-03-10', '21:00:00', 'Italian Roadside Station',
 2.20, 1203.25, 122.00, 172.00, 11.00, 60.00);

INSERT INTO AIR_QUALITY VALUES
(5, DATE '2004-03-10', '22:00:00', 'Italian Roadside Station',
 1.60, 1110.00, 116.00, 131.00, 11.15, 59.58);

INSERT INTO AIR_QUALITY VALUES
(6, DATE '2004-03-10', '23:00:00', 'Italian Roadside Station',
 1.20, 949.25, 96.00, 89.00, 11.18, 59.18);

INSERT INTO AIR_QUALITY VALUES
(7, DATE '2004-03-11', '00:00:00', 'Italian Roadside Station',
 1.20, 732.50, 77.00, 62.00, 11.33, 56.78);

INSERT INTO AIR_QUALITY VALUES
(8, DATE '2004-03-11', '01:00:00', 'Italian Roadside Station',
 1.00, 729.50, 76.00, 62.00, 10.68, 60.00);

INSERT INTO AIR_QUALITY VALUES
(9, DATE '2004-03-11', '02:00:00', 'Italian Roadside Station',
 0.90, 619.50, 60.00, 45.00, 10.65, 59.67);
```

```
INSERT INTO AIR_QUALITY VALUES
(10, DATE '2004-03-11', '05:00:00', 'Italian Roadside Station',
 0.70, 421.75, 28.00, 16.00, 11.00, 56.18);

INSERT INTO AIR_QUALITY VALUES
(11, DATE '2004-03-11', '06:00:00', 'Italian Roadside Station',
 0.70, 471.50, 48.00, 34.00, 10.45, 58.13);

INSERT INTO AIR_QUALITY VALUES
(12, DATE '2004-03-11', '07:00:00', 'Italian Roadside Station',
 1.10, 729.75, 82.00, 98.00, 10.20, 59.60);

INSERT INTO AIR_QUALITY VALUES
(13, DATE '2004-03-11', '08:00:00', 'Italian Roadside Station',
 2.00, 1101.50, 112.00, 174.00, 10.75, 57.43);

INSERT INTO AIR_QUALITY VALUES
(14, DATE '2004-03-11', '09:00:00', 'Italian Roadside Station',
 2.20, 1027.75, 101.00, 129.00, 10.50, 60.60);

INSERT INTO AIR_QUALITY VALUES
(15, DATE '2004-03-11', '10:00:00', 'Italian Roadside Station',
 1.70, 859.75, 98.00, 112.00, 10.80, 58.35);

INSERT INTO AIR_QUALITY VALUES
(16, DATE '2004-03-11', '11:00:00', 'Italian Roadside Station',
 1.50, 670.50, 92.00, 95.00, 10.50, 57.93);

INSERT INTO AIR_QUALITY VALUES
(17, DATE '2004-03-11', '12:00:00', 'Italian Roadside Station',
 1.60, 664.00, 95.00, 104.00, 9.53, 66.77);

INSERT INTO AIR_QUALITY VALUES
(18, DATE '2004-03-11', '13:00:00', 'Italian Roadside Station',
 1.90, 799.00, 112.00, 146.00, 8.30, 76.43);

INSERT INTO AIR_QUALITY VALUES
(19, DATE '2004-03-11', '14:00:00', 'Italian Roadside Station',
 2.90, 1036.50, 128.00, 207.00, 8.00, 81.15);

INSERT INTO AIR_QUALITY VALUES
(20, DATE '2004-03-11', '15:00:00', 'Italian Roadside Station',
 2.20, 946.25, 126.00, 184.00, 8.33, 79.80);
```

```
INSERT INTO AIR_QUALITY VALUES
(21, DATE '2004-03-11', '16:00:00', 'Italian Roadside Station',
 2.20, 956.75, 131.00, 193.00, 9.70, 71.15);

INSERT INTO AIR_QUALITY VALUES
(22, DATE '2004-03-11', '17:00:00', 'Italian Roadside Station',
 2.90, 1104.00, 135.00, 243.00, 9.78, 67.62);

INSERT INTO AIR_QUALITY VALUES
(23, DATE '2004-03-11', '18:00:00', 'Italian Roadside Station',
 4.80, 1408.50, 151.00, 281.00, 10.35, 64.17);

INSERT INTO AIR_QUALITY VALUES
(24, DATE '2004-03-11', '19:00:00', 'Italian Roadside Station',
 6.90, 1704.00, 172.00, 383.00, 9.65, 69.30);

INSERT INTO AIR_QUALITY VALUES
(25, DATE '2004-03-11', '20:00:00', 'Italian Roadside Station',
 6.10, 1653.75, 165.00, 351.00, 9.65, 67.75);

COMMIT;
```

Records Inserted

A total of 25 air-quality measurement records were inserted into the AIR_QUALITY table.

SQL QUERY 1: Display Air-Quality Records

```
SELECT
   AQ_ID,
   READING_DATE,
   READING_TIME,
   LOCATION,
   CO,
   OZONE,
   NO2,
   NOX,
   TEMPERATURE,
   HUMIDITY
FROM AIR_QUALITY
ORDER BY READING_DATE, READING_TIME;
```

Output

The query displays all 25 air-quality records with their date, time, monitoring location, pollutant readings, temperature, and humidity.

SQL QUERY 2: Average Pollutant Levels

```
SELECT
   LOCATION,
   ROUND(AVG(CO), 2) AS AVG_CO,
   ROUND(AVG(OZONE), 2) AS AVG_OZONE_SENSOR,
   ROUND(AVG(NO2), 2) AS AVG_NO2,
   ROUND(AVG(NOX), 2) AS AVG_NOX
FROM AIR_QUALITY
GROUP BY LOCATION;
```

Output

LOCATION: Italian Roadside Station

AVG_CO: 2.25

AVG_OZONE_SENSOR: 968.52

AVG_NO2: 105.68

AVG_NOX: 148.44

The query calculates the average CO, ozone-targeted sensor response, NO_2, and NOx values for the monitoring location.

SQL QUERY 3: Identify Elevated CO Measurements

For this project, CO greater than 4 mg/m³ is used as the selected educational alert threshold. This threshold is used only for project classification and should not be interpreted as a direct hourly regulatory standard.

```
SELECT
   AQ_ID,
   READING_DATE,
   READING_TIME,
   LOCATION,
   CO
FROM AIR_QUALITY
```

```
WHERE CO > 4
ORDER BY CO DESC;
```

Output

AQ_ID DATE TIME CO

24 11-03-2004 19:00:00 6.90

25 11-03-2004 20:00:00 6.10

23 11-03-2004 18:00:00 4.80

A total of three records have CO readings greater than 4 mg/m³.

SQL QUERY 4: Calculate Hourly Average Ozone Sensor Response

The OZONE field represents the original PT08.S5(O3) sensor-response variable from the UCI dataset. Therefore, this query calculates the average ozone-targeted sensor response and does not represent direct ozone concentration.

```
SELECT
   SUBSTR(READING_TIME, 1, 2) AS HOUR,
   ROUND(AVG(OZONE), 2) AS AVG_OZONE_SENSOR
FROM AIR_QUALITY
GROUP BY SUBSTR(READING_TIME, 1, 2)
ORDER BY HOUR;
```

Output

The query groups the 25 records according to the hour component of READING_TIME and calculates the average ozone-targeted sensor response for each available hour.

SQL QUERY 5: Rank Monitoring Locations by Average NO_2

```
SELECT
   LOCATION,
   ROUND(AVG(NO2), 2) AS AVG_NO2,
   RANK() OVER (ORDER BY AVG(NO2) DESC) AS LOCATION_RANK
FROM AIR_QUALITY
GROUP BY LOCATION
ORDER BY LOCATION_RANK;
```

Output

LOCATION: Italian Roadside Station

AVG_NO2: 105.68

LOCATION_RANK: 1

Note: The selected 25 records represent one monitoring location. Therefore, the ranking contains only one location and does not provide a meaningful comparison between different monitoring stations.

PL/SQL FUNCTION 1: Classify CO Reading

Create a function to classify a CO reading using the project's selected reference thresholds.

CO Classification:

≤ 4 mg/m³ → NORMAL

4 and ≤ 7 mg/m³ → ELEVATED

7 mg/m³ → HIGH

These categories are used as a project-level classification reference.

```
CREATE OR REPLACE FUNCTION CLASSIFY_CO
(
   P_CO NUMBER
)
RETURN VARCHAR2
IS
BEGIN

   IF P_CO IS NULL THEN
      RETURN 'NO DATA';

   ELSIF P_CO <= 4 THEN
      RETURN 'NORMAL';

   ELSIF P_CO <= 7 THEN
      RETURN 'ELEVATED';

   ELSE
      RETURN 'HIGH';
```

```
    END IF;

END;
/
```

Test the Function

```
SELECT
    AQ_ID,
    CO,
    CLASSIFY_CO(CO) AS CO_STATUS
FROM AIR_QUALITY
ORDER BY AQ_ID;
```

Example Output

AQ_ID CO CO_STATUS

1 2.60 NORMAL

2 2.00 NORMAL

22 2.90 NORMAL

23 4.80 ELEVATED

24 6.90 ELEVATED

25 6.10 ELEVATED

The function successfully classifies each CO measurement according to the defined project thresholds.

PL/SQL PROCEDURE 2: Retrieve Air-Quality Data

Create a procedure to retrieve air-quality measurements for a specified monitoring location and date.

```
CREATE OR REPLACE PROCEDURE GET_AIR_DATA
(
    P_LOCATION VARCHAR2,
    P_DATE DATE
)
```

```
IS
BEGIN

   FOR R IN
   (
      SELECT
         AQ_ID,
         READING_TIME,
         CO,
         OZONE,
         NO2,
         NOX,
         TEMPERATURE,
         HUMIDITY
      FROM AIR_QUALITY
      WHERE UPPER(LOCATION) = UPPER(P_LOCATION)
        AND TRUNC(READING_DATE) = TRUNC(P_DATE)
      ORDER BY READING_TIME
   )
   LOOP

      DBMS_OUTPUT.PUT_LINE(
         'ID: ' || R.AQ_ID ||
         ' | Time: ' || R.READING_TIME ||
         ' | CO: ' || R.CO ||
         ' | Ozone Sensor: ' || R.OZONE ||
         ' | NO2: ' || R.NO2 ||
         ' | NOX: ' || R.NOX ||
         ' | Temp: ' || R.TEMPERATURE ||
         ' | Humidity: ' || R.HUMIDITY
      );

   END LOOP;

END;
/
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Execution

```
SET SERVEROUTPUT ON;

BEGIN

   GET_AIR_DATA(
      'Italian Roadside Station',
      DATE '2004-03-11'
   );

END;
/
```

Output

The procedure retrieves the air-quality measurements recorded at the specified monitoring location on 11-03-2004 and displays the pollutant readings, temperature, and humidity for each available measurement time.

PL/SQL PROCEDURE 3: Generate Pollution Alert Summary

Create a procedure to generate a pollutant summary and identify records crossing the project's CO alert threshold.

Project Alert Threshold:

CO > 4 mg/m³

```
CREATE OR REPLACE PROCEDURE POLLUTION_ALERT
IS
   V_AVG_CO NUMBER;
   V_AVG_NO2 NUMBER;
   V_AVG_NOX NUMBER;
   V_ALERT_COUNT NUMBER;

BEGIN

   SELECT
      AVG(CO),
      AVG(NO2),
      AVG(NOX)
   INTO
```

```
    V_AVG_CO,
    V_AVG_NO2,
    V_AVG_NOX
FROM AIR_QUALITY;

SELECT COUNT(*)
INTO V_ALERT_COUNT
FROM AIR_QUALITY
WHERE CO > 4;

DBMS_OUTPUT.PUT_LINE('----- POLLUTANT SUMMARY -----');

DBMS_OUTPUT.PUT_LINE(
    'Average CO  : ' || ROUND(V_AVG_CO, 2)
);

DBMS_OUTPUT.PUT_LINE(
    'Average NO2 : ' || ROUND(V_AVG_NO2, 2)
);

DBMS_OUTPUT.PUT_LINE(
    'Average NOX : ' || ROUND(V_AVG_NOX, 2)
);

DBMS_OUTPUT.PUT_LINE(
    'CO Alert Count: ' || V_ALERT_COUNT
);

DBMS_OUTPUT.PUT_LINE('----- ALERT RECORDS -----');

FOR R IN
(
    SELECT
        AQ_ID,
        READING_DATE,
        READING_TIME,
        CO
    FROM AIR_QUALITY
    WHERE CO > 4
    ORDER BY CO DESC
)
LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
            'ALERT -> ID: ' || R.AQ_ID ||
            ' | Date: ' ||
            TO_CHAR(R.READING_DATE,'DD-MM-YYYY') ||
            ' | Time: ' || R.READING_TIME ||
            ' | CO: ' || R.CO || ' mg/m3'
        );

    END LOOP;

END;
/
```

Execution

```
SET SERVEROUTPUT ON;

BEGIN

    POLLUTION_ALERT;

END;
/
```

Expected Summary

```
----- POLLUTANT SUMMARY -----
Average CO  : 2.25
Average NO2 : 105.68
Average NOX : 148.44
CO Alert Count: 3

----- ALERT RECORDS -----
ALERT -> ID: 24 | Date: 11-03-2004 | Time: 19:00:00 | CO: 6.9 mg/m3
ALERT -> ID: 25 | Date: 11-03-2004 | Time: 20:00:00 | CO: 6.1 mg/m3
ALERT -> ID: 23 | Date: 11-03-2004 | Time: 18:00:00 | CO: 4.8 mg/m3
```

Dataset Insights

• The dataset contains 25 selected air-quality measurement records.

• The records represent measurements from the Italian Roadside Station.

• The dataset contains CO, ozone-targeted sensor response, NO_2, NOx, temperature, and humidity measurements.

• The average CO value across the 25 records is approximately 2.25 mg/m³.

• The average NO_2 value is approximately 105.68 µg/m³.

• The average NOx value is approximately 148.44 ppb.

• Three records have CO readings greater than the project's selected threshold of 4 mg/m³.

• The highest CO reading in the selected dataset is 6.90 mg/m³ at 19:00 on 11-03-2004.

• SQL queries can be used to identify elevated pollution measurements and calculate pollutant averages.

• PL/SQL functions can classify individual CO readings into NORMAL, ELEVATED, and HIGH categories.

• PL/SQL procedures can retrieve measurements and generate automated pollution alert summaries.

• The selected records contain only one monitoring location, so location ranking is demonstrated technically but does not provide a comparison between multiple locations.

Result

Thus, the Air Quality Monitoring and Pollution Alerts system was successfully implemented using Oracle SQL and PL/SQL.

The AIR_QUALITY table was created and populated with 25 records from the selected UCI Air Quality Dataset. SQL queries were executed successfully to display air-quality records, calculate average pollutant levels, identify elevated CO measurements, calculate hourly ozone-targeted sensor-response averages, and rank the available monitoring location.

A PL/SQL function was implemented to classify CO readings, and PL/SQL procedures were created to retrieve air-quality measurements and generate pollution alert summaries.

The system successfully demonstrates the use of database tables, SQL queries, aggregate functions, analytical functions, PL/SQL functions, procedures, and automated alert processing for an air-quality monitoring application.

PRACTICE-BASED LEARNING DECLARATION

I hereby declare that this SQL Practical Observation Record is my original work. The SQL queries, observations, and outputs included in this record were prepared as part of the Practice-Based Learning activities.

Student Signature: _____
Faculty Signature: _____

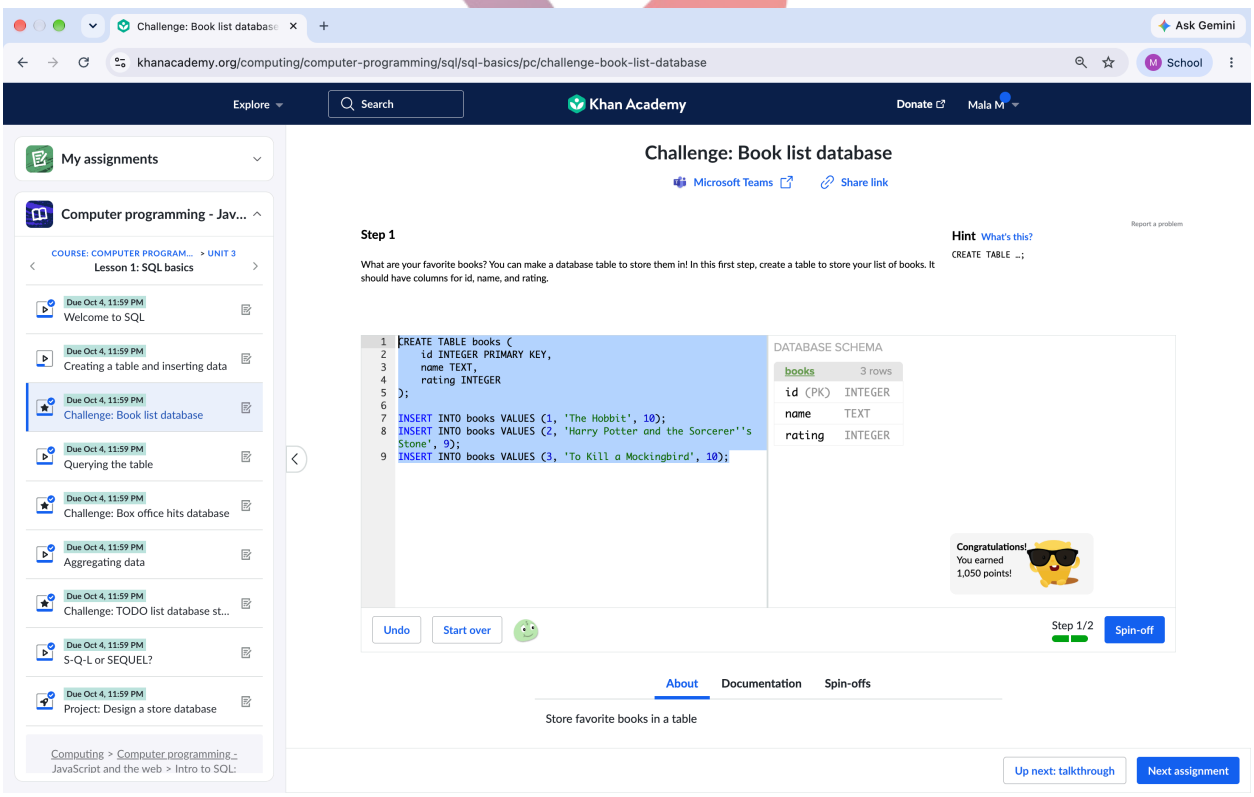
Date: _____

Khan Academy

Book list database

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.

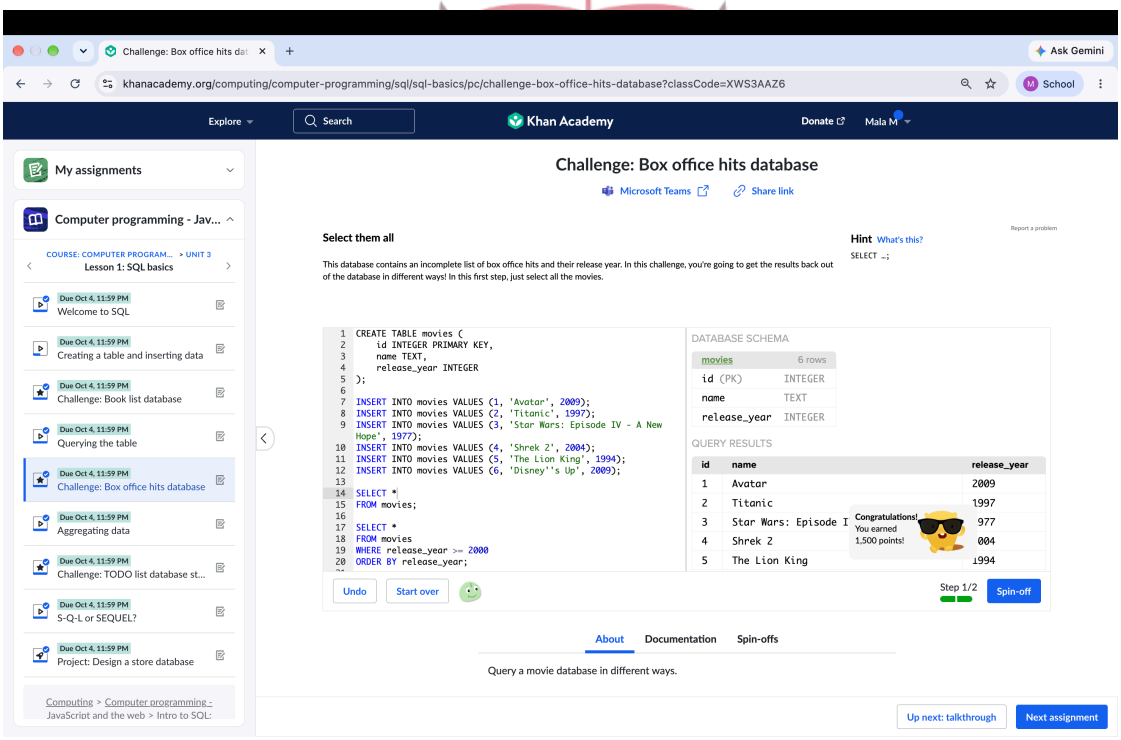
```
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    name TEXT,
    rating INTEGER
);
INSERT INTO books VALUES (1, 'The Hobbit', 10);
INSERT INTO books VALUES (2, 'Harry Potter and the Sorcerer''s Stone', 9);
INSERT INTO books VALUES (3, 'To Kill a Mockingbird', 10);
```



Box office hits database

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

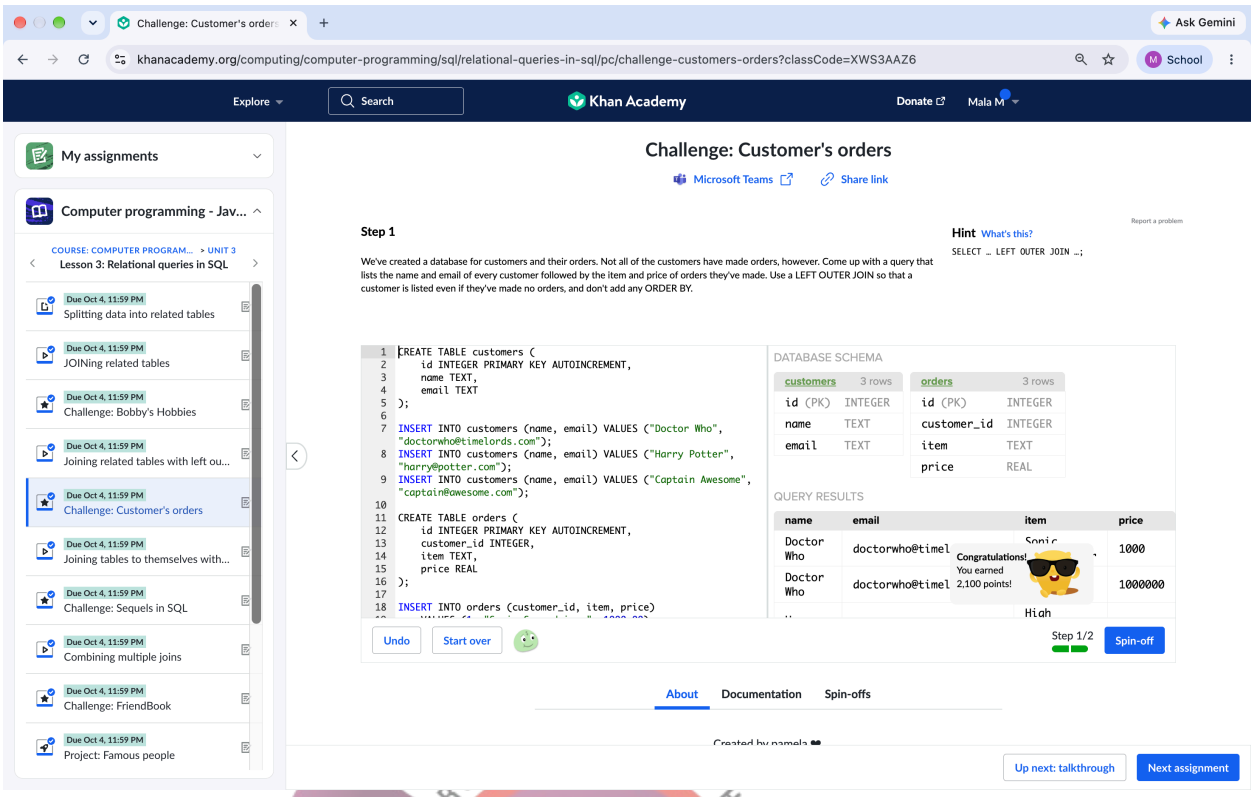
```
CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    name TEXT,
    release_year INTEGER
);

INSERT INTO movies VALUES (1, 'Avatar', 2009);
INSERT INTO movies VALUES (2, 'Titanic', 1997);
INSERT INTO movies VALUES (3, 'Star Wars: Episode IV - A New Hope', 1977);
INSERT INTO movies VALUES (4, 'Shrek 2', 2004);
INSERT INTO movies VALUES (5, 'The Lion King', 1994);
INSERT INTO movies VALUES (6, 'Disney''s Up', 2009);

SELECT *
FROM movies;

SELECT *
FROM movies
WHERE release_year >= 2000
ORDER BY release_year;
```



Design a store database

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

```
CREATE TABLE bike_store (
    id INTEGER PRIMARY KEY,
    bike_name TEXT,
    brand TEXT,
    price INTEGER,
    stock INTEGER
);

INSERT INTO bike_store VALUES (1, 'Mountain Pro', 'Trek', 800, 10);
INSERT INTO bike_store VALUES (2, 'Road Racer', 'Giant', 1200, 5);
INSERT INTO bike_store VALUES (3, 'City Cruiser', 'Schwinn', 500, 15);
INSERT INTO bike_store VALUES (4, 'Trail Blazer', 'Trek', 950, 8);
INSERT INTO bike_store VALUES (5, 'Speedster', 'Specialized', 1500, 4);
INSERT INTO bike_store VALUES (6, 'Eco Ride', 'Hero', 300, 20);
INSERT INTO bike_store VALUES (7, 'Urban Rider', 'Atlas', 450, 12);
INSERT INTO bike_store VALUES (8, 'Mountain King', 'Cannondale', 1100, 6);
INSERT INTO bike_store VALUES (9, 'Road Master', 'Giant', 1300, 7);
INSERT INTO bike_store VALUES (10, 'Adventure X', 'Trek', 1400, 3);
INSERT INTO bike_store VALUES (11, 'Kids Fun Bike', 'Hero', 200, 25);
INSERT INTO bike_store VALUES (12, 'Hybrid Plus', 'Specialized', 900, 9);
INSERT INTO bike_store VALUES (13, 'Commuter Bike', 'Atlas', 600, 14);
INSERT INTO bike_store VALUES (14, 'Speed X', 'Cannondale', 1700, 2);
INSERT INTO bike_store VALUES (15, 'Trail Rider', 'Schwinn', 750, 11);

SELECT *
FROM bike_store
ORDER BY price ASC;

SELECT AVG(price) AS average_price
FROM bike_store;
```


Challenge: Customer's orders
Ask Gemini
khanacademy.org/computing/computer-programming/sql/relational-queries-in-sql/pc/challenge-customers-orders?classCode=XWS3AAZ6
School
Explore
Search
Khan Academy
Donate
Mala M
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 3: Relational queries in SQL
Due Oct 4, 11:59 PM
Splitting data into related tables
Due Oct 4, 11:59 PM
JOINing related tables
Due Oct 4, 11:59 PM
Challenge: Bobby's Hobbies
Due Oct 4, 11:59 PM
Joining related tables with left ou...
Due Oct 4, 11:59 PM
Challenge: Customer's orders
Due Oct 4, 11:59 PM
Joining tables to themselves with...
Due Oct 4, 11:59 PM
Challenge: Sequels in SQL
Due Oct 4, 11:59 PM
Combining multiple joins
Due Oct 4, 11:59 PM
Challenge: FriendBook
Due Oct 4, 11:59 PM
Project: Famous people
Challenge: Customer's orders
Microsoft Teams
Share link
Step 1
We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.
Hint What's this?
SELECT … LEFT OUTER JOIN …;
Report a problem
CREATE TABLE customers (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
email TEXT
);
INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");
CREATE TABLE orders (
id INTEGER PRIMARY KEY AUTOINCREMENT,
customer_id INTEGER,
item TEXT,
price REAL
);
INSERT INTO orders (customer_id, item, price)
DATABASE SCHEMA
customers 3 rows
id (PK) INTEGER
name TEXT
email TEXT
orders 3 rows
id (PK) INTEGER
customer_id INTEGER
item TEXT
price REAL
QUERY RESULTS
name
email
item
price
Doctor Who
doctorwho@timel
1000
Doctor Who
doctorwho@timel
1000000
Congratulations!
You earned 2,100 points!
Undo
Start over
Step 1/2
Spin-off
About
Documentation
Spin-offs
Up next: talkthrough
Next assignment

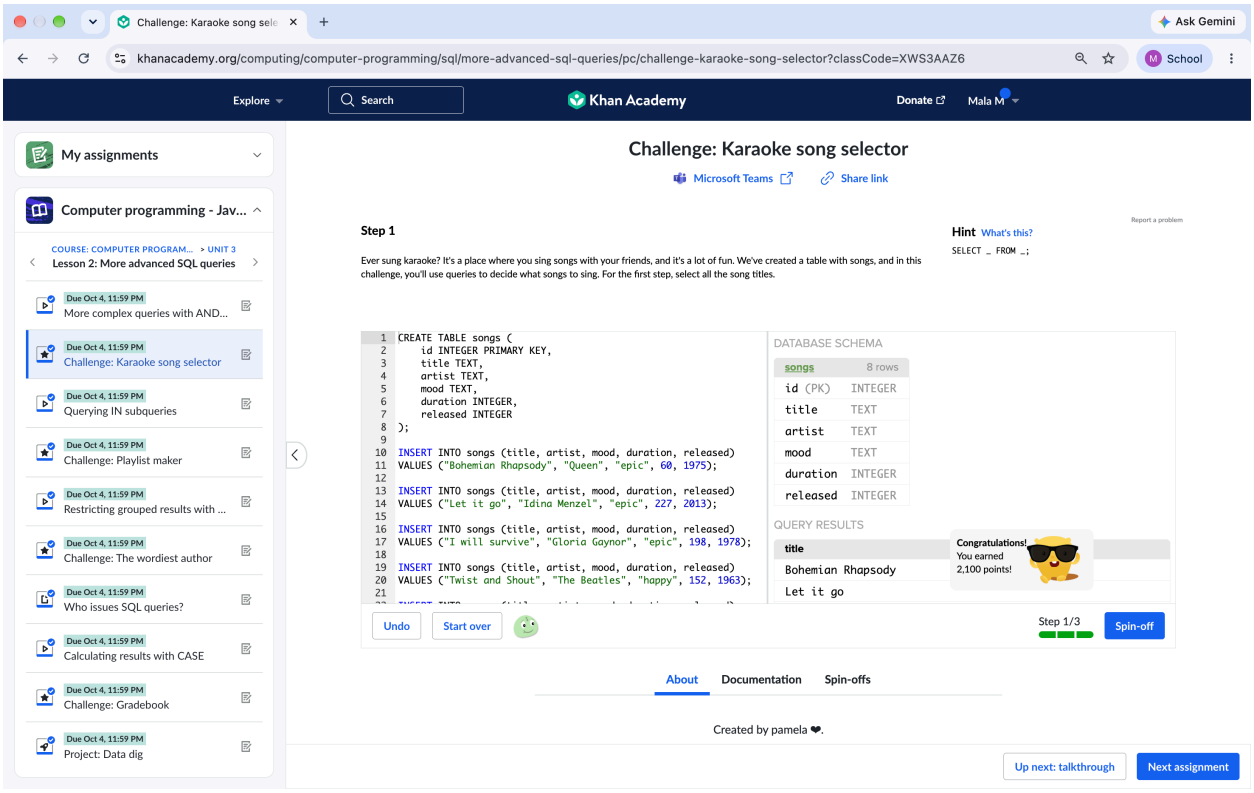
AUTONOMOUS

Karaoke song selector

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER
);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);

INSERT INTO songs (title, artist, mood, duration, released)
VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

-- Select all song titles
SELECT title
FROM songs;
```

```
-- Select titles of songs that are epic OR released after 1990
SELECT title
FROM songs
WHERE mood = "epic" OR released > 1990;

-- Select titles of songs that are epic, released after 1990, and less than 4 minutes long
SELECT title
FROM songs
WHERE mood = "epic"
  AND released > 1990
  AND duration < 240;
```



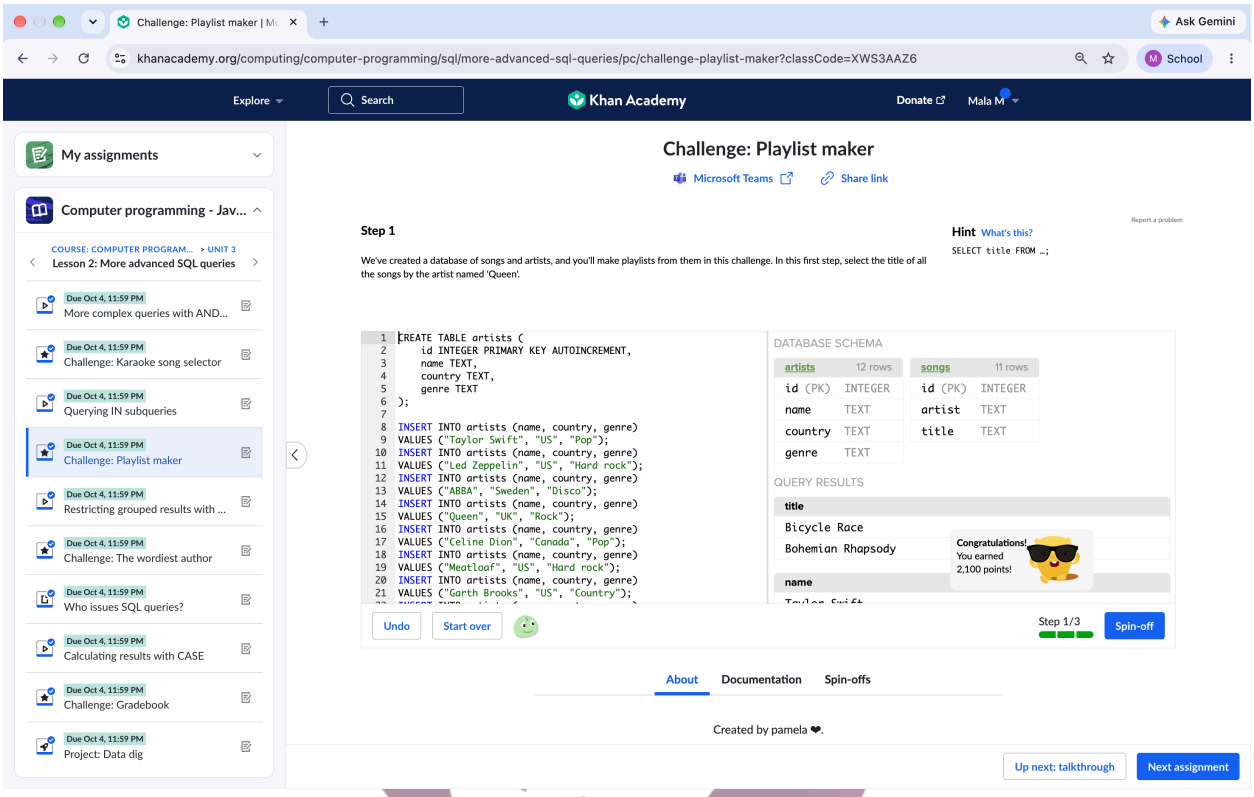
Playlist maker

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named ‘Queen'.

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT
);

INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Bob Marley", "Jamaica", "Reggae");

CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT
);
```

```
INSERT INTO songs (artist, title)
VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
VALUES ("Guns N' Roses", "Don't cry");

-- Select all songs by Queen
SELECT title
FROM songs
WHERE artist = "Queen";

-- Select all Pop artists
SELECT name
FROM artists
WHERE genre = "Pop";

-- Select all songs by Pop artists
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = "Pop"
);
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS


Challenge: Playlist maker | M
Ask Gemini
khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pc/challenge-playlist-maker?classCode=XWS3AAZ6
School
Explore
Search
Khan Academy
Donate
Mala M
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Due Oct 4, 11:59 PM
More complex queries with AND...
Due Oct 4, 11:59 PM
Challenge: Karaoke song selector
Due Oct 4, 11:59 PM
Querying IN subqueries
Due Oct 4, 11:59 PM
Challenge: Playlist maker
Due Oct 4, 11:59 PM
Restricting grouped results with ...
Due Oct 4, 11:59 PM
Challenge: The wordiest author
Due Oct 4, 11:59 PM
Who issues SQL queries?
Due Oct 4, 11:59 PM
Calculating results with CASE
Due Oct 4, 11:59 PM
Challenge: Gradebook
Due Oct 4, 11:59 PM
Project: Data dig
Challenge: Playlist maker
Microsoft Teams
Share link
Report a problem
Step 1
We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.
Hint What's this?
SELECT title FROM ...;
CREATE TABLE artists (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
country TEXT,
genre TEXT
);
INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");
DATABASE SCHEMA
artists 12 rows
id (PK) INTEGER
name TEXT
country TEXT
genre TEXT
songs 11 rows
id (PK) INTEGER
artist TEXT
title TEXT
QUERY RESULTS
title
Bicycle Race
Bohemian Rhapsody
Congratulations!
You earned
2,100 points!
name
Undo
Start over
Step 1/3
Spin-off
About
Documentation
Spin-offs
Created by pamela ❤.
Up next: talkthrough
Next assignment

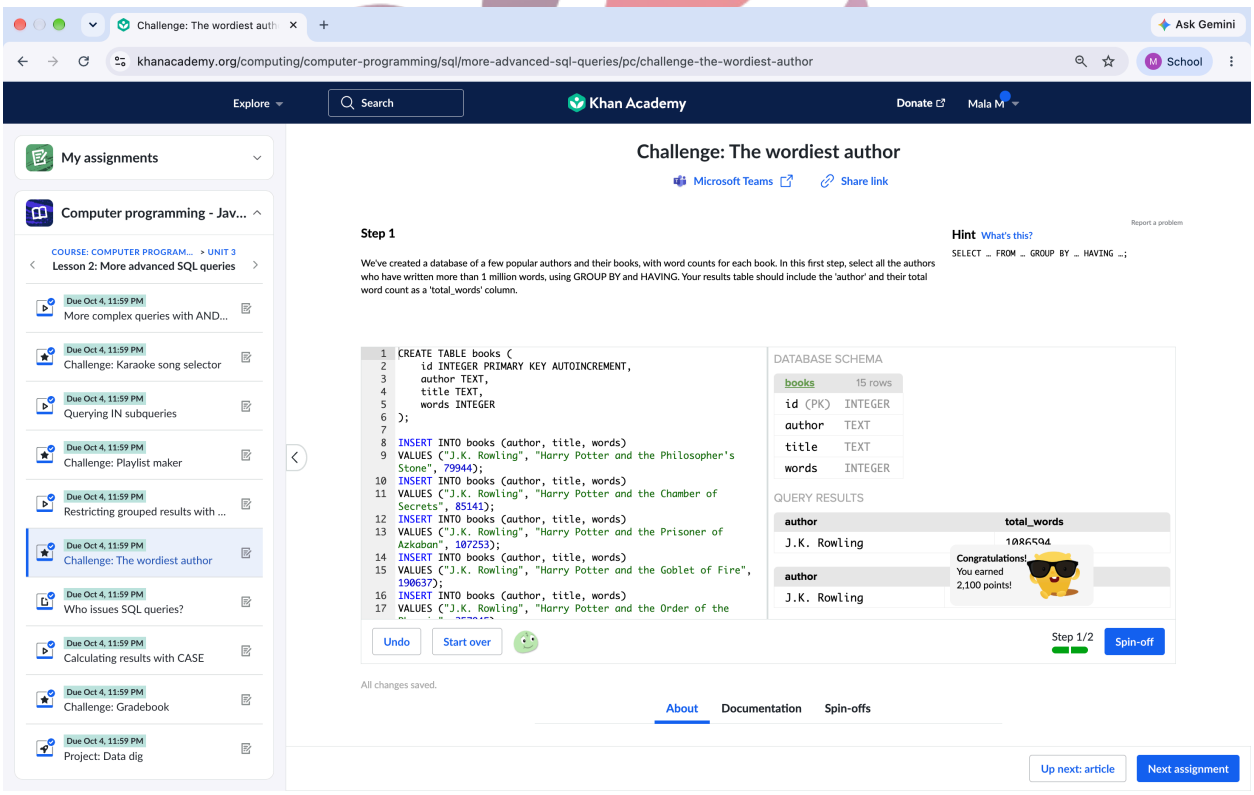
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

The wordiest author

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER
);

INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)
VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
```

```
VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

-- Step 1: Select authors who have written more than 1 million total words
SELECT author,
    SUM(words) AS total_words
FROM books
GROUP BY author
HAVING total_words > 1000000;

-- Step 2: Select authors with an average of more than 150,000 words per book
SELECT author,
    AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING avg_words > 150000;
```

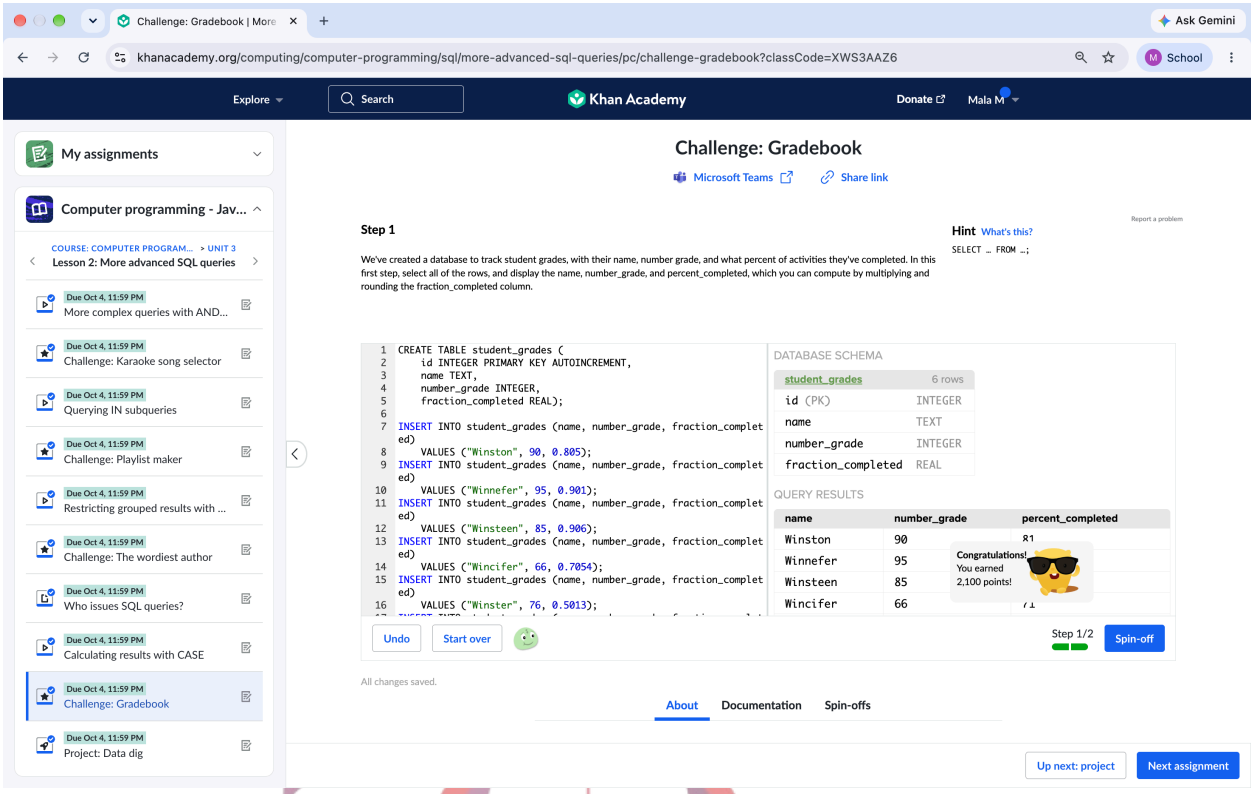


Gradebook

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

```
CREATE TABLE student_grades (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   number_grade INTEGER,
   fraction_completed REAL);

INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);

SELECT name,
      number_grade,
      ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;

SELECT COUNT(*),
   CASE
      WHEN number_grade > 90 THEN 'A'
      WHEN number_grade > 80 THEN 'B'
      WHEN number_grade > 70 THEN 'C'
      ELSE 'F'
   END AS "letter_grade"
FROM student_grades
GROUP BY
   CASE
      WHEN number_grade > 90 THEN 'A'
      WHEN number_grade > 80 THEN 'B'
```

```
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
END;
```

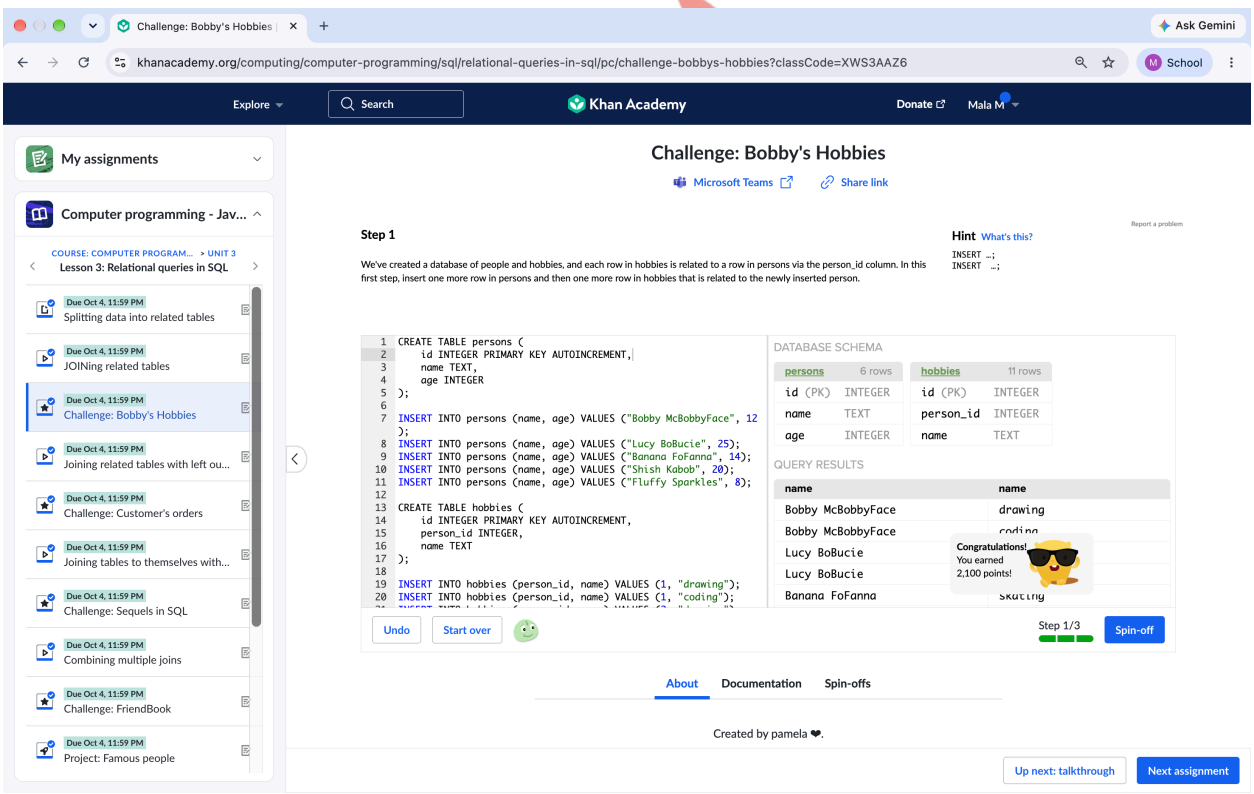


Bobby's Hobbies

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step,
insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER
);

INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);

CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

-- NEW PERSON
INSERT INTO persons (name, age) VALUES ("Starry Nova", 30);

-- NEW HOBBY linked to the new person (id = 6)
INSERT INTO hobbies (person_id, name) VALUES (6, "painting");

-- 1. SELECT the 2 tables with a join to see each person's name next to their hobby
```

```
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
  ON persons.id = hobbies.person_id;

-- 2. ADDITIONAL QUERY: Show only the name and hobbies of 'Bobby McBobbyFace'
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
  ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace”;
```

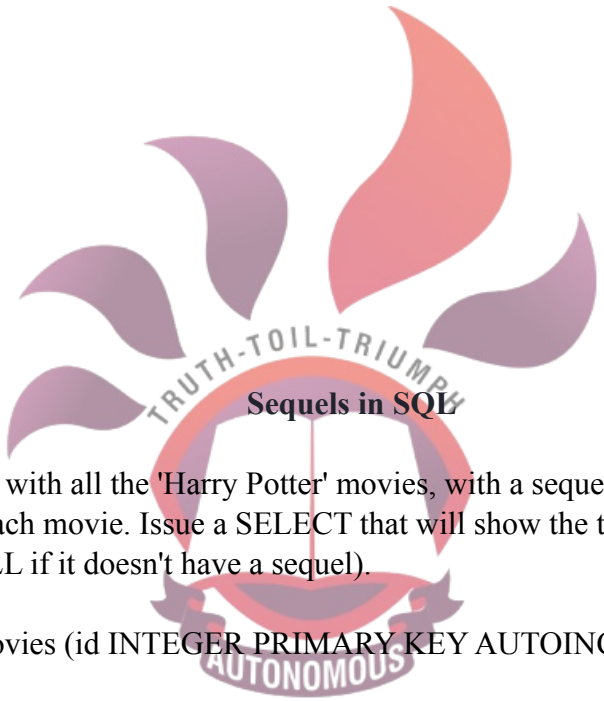


Customer's orders

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT
);

INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");

CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL
);

INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);

-- 1. Query to show all customers and their order items/prices, keeping customers with no orders
SELECT customers.name, customers.email, orders.item, orders.price
FROM customers
LEFT OUTER JOIN orders
  ON customers.id = orders.customer_id;

-- 2. NEW QUERY: Show total money spent per customer, sorted from most to least spent
SELECT customers.name, customers.email, SUM(orders.price) AS total_spent
FROM customers
```

```
LEFT OUTER JOIN orders
  ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```

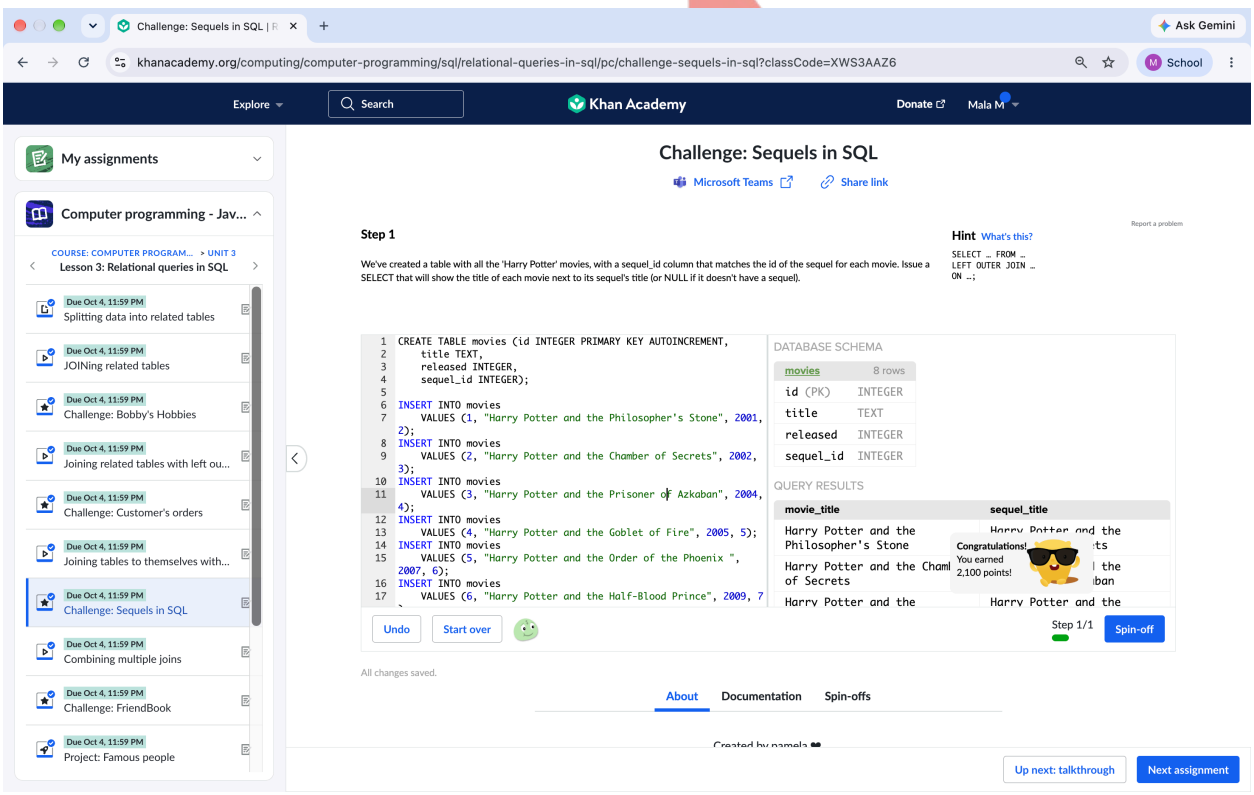


Sequels in SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER);

INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
```

```
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);

-- Self-join query to show each movie title next to its sequel title
SELECT movies.title AS movie_title, sequels.title AS sequel_title
FROM movies
LEFT OUTER JOIN movies sequels
  ON movies.sequel_id = sequels.id;
```

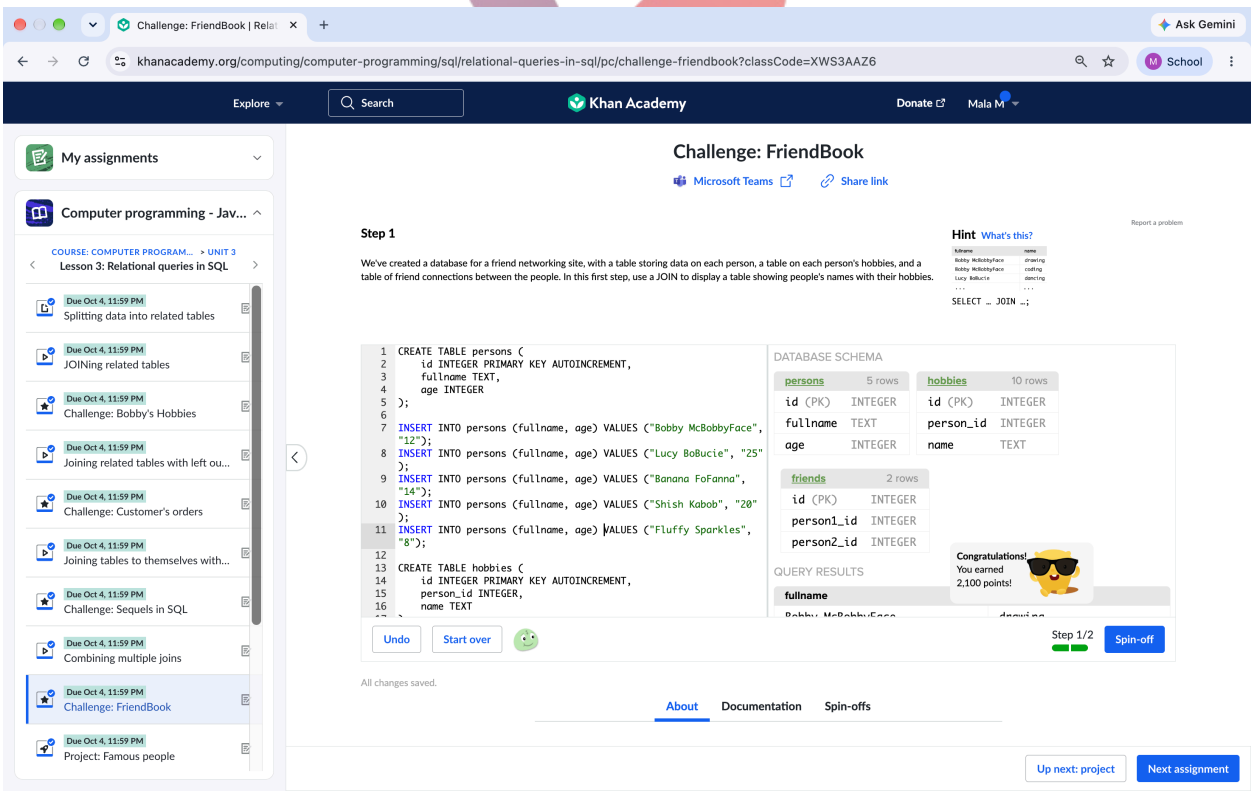


FriendBook

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER
);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

CREATE TABLE friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER
);

INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
```

```
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);

-- Step 1: Use a JOIN to display a table showing people's names with their hobbies
SELECT persons.fullname, hobbies.name
FROM persons
JOIN hobbies
  ON persons.id = hobbies.person_id;

-- Step 2: Use JOINs to show the names of each pair of friends
SELECT p1.fullname AS friend1, p2.fullname AS friend2
FROM friends
JOIN persons p1
  ON friends.person1_id = p1.id
JOIN persons p2
  ON friends.person2_id = p2.id;
```



Hacker Rank

1. Revising the Select Query I

Query the total population of all cities in **CITY** where *District* is **California**.

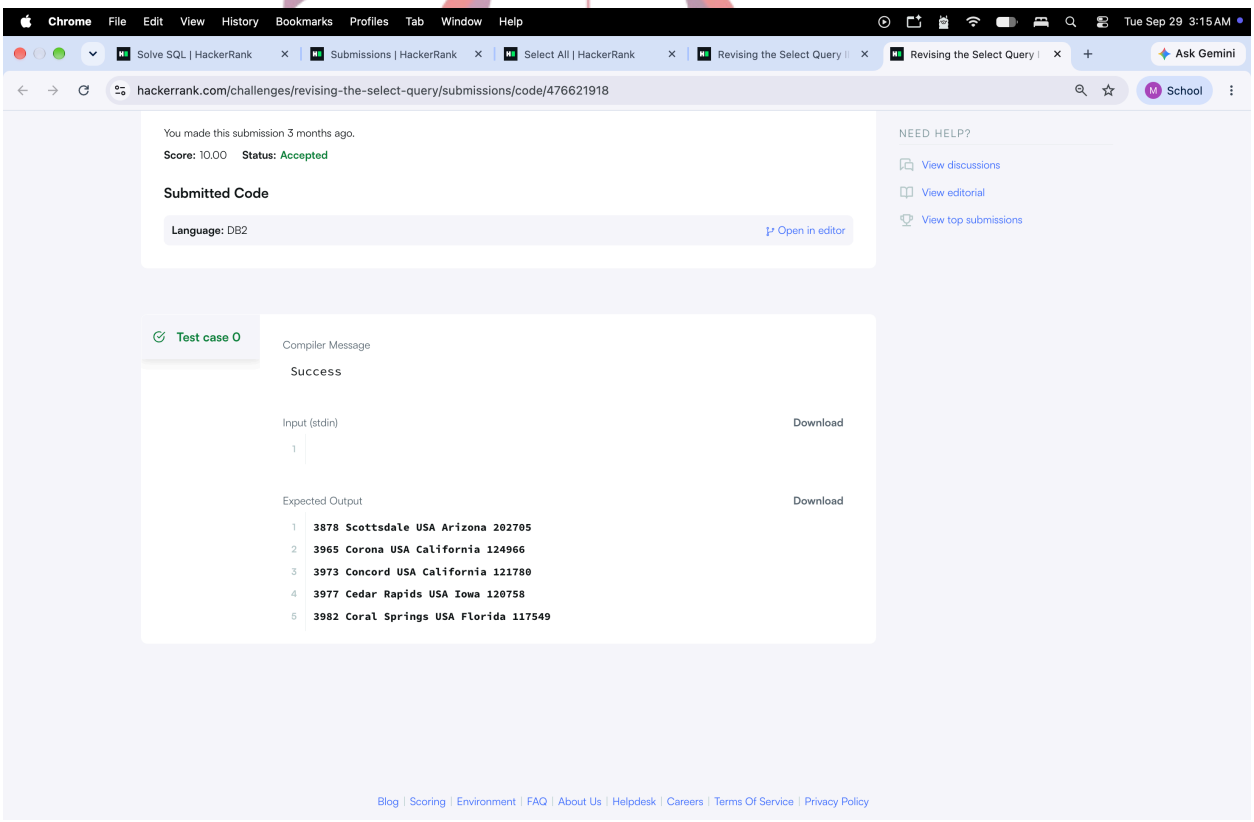
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

OUTPUT:

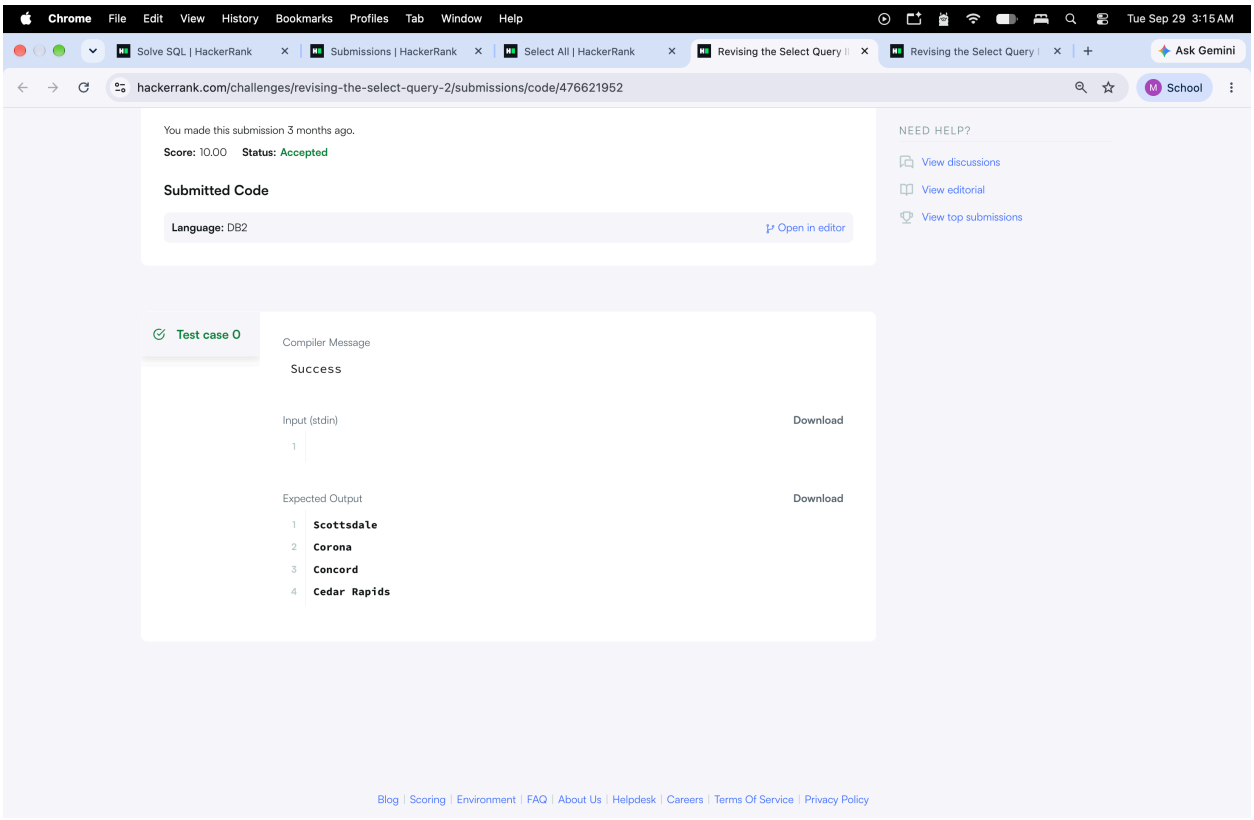


2. Revising the Select Query II

INPUT:

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'USA'
AND POPULATION > 120000;
```

OUTPUT:

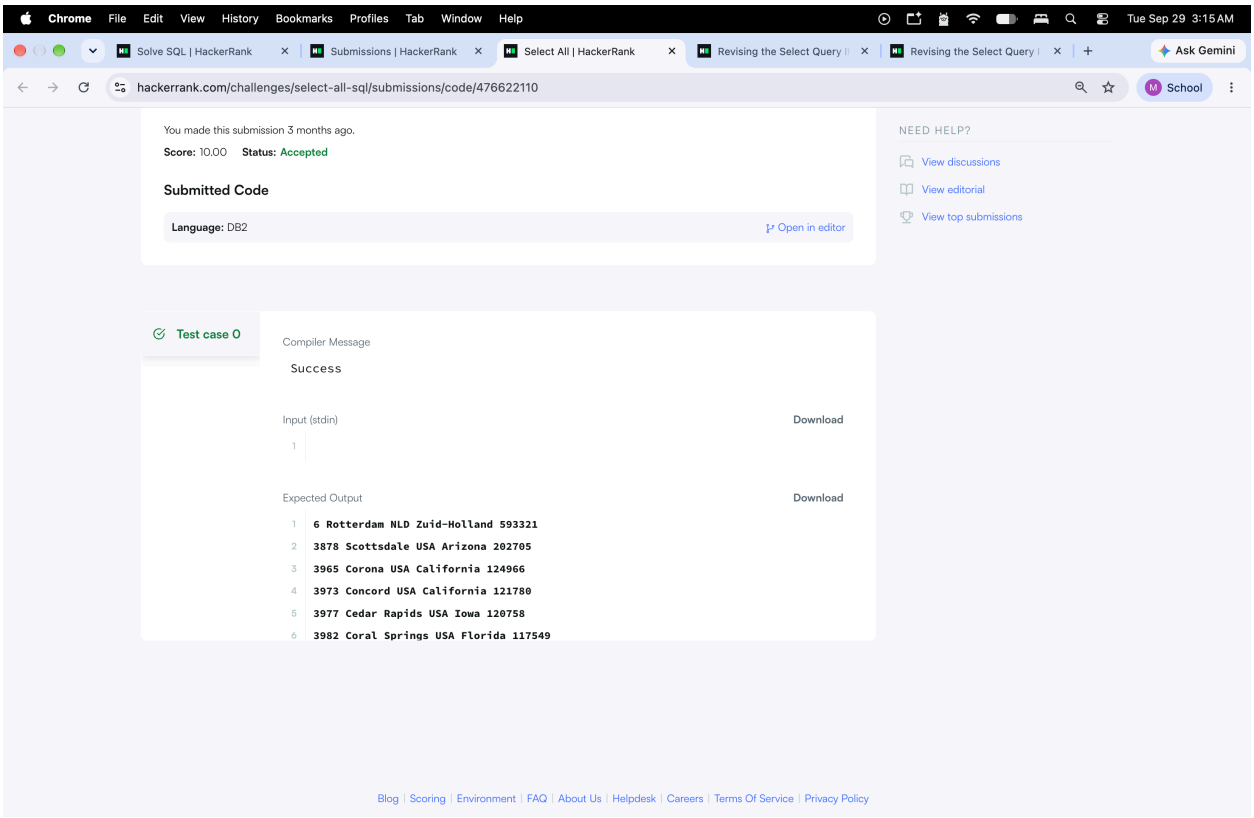


3. Select All

INPUT:

```
SELECT *
FROM CITY;
```

OUTPUT:

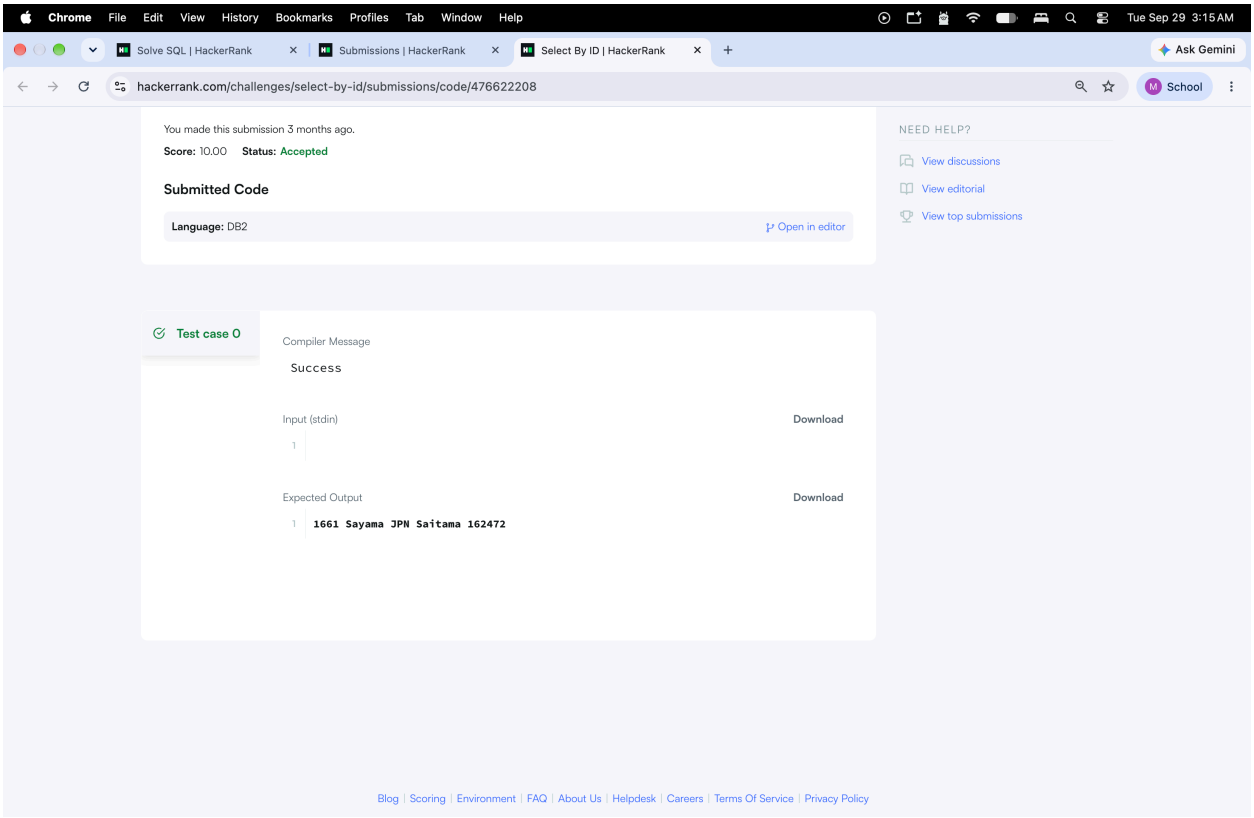


4. Select By ID

INPUT:

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT:

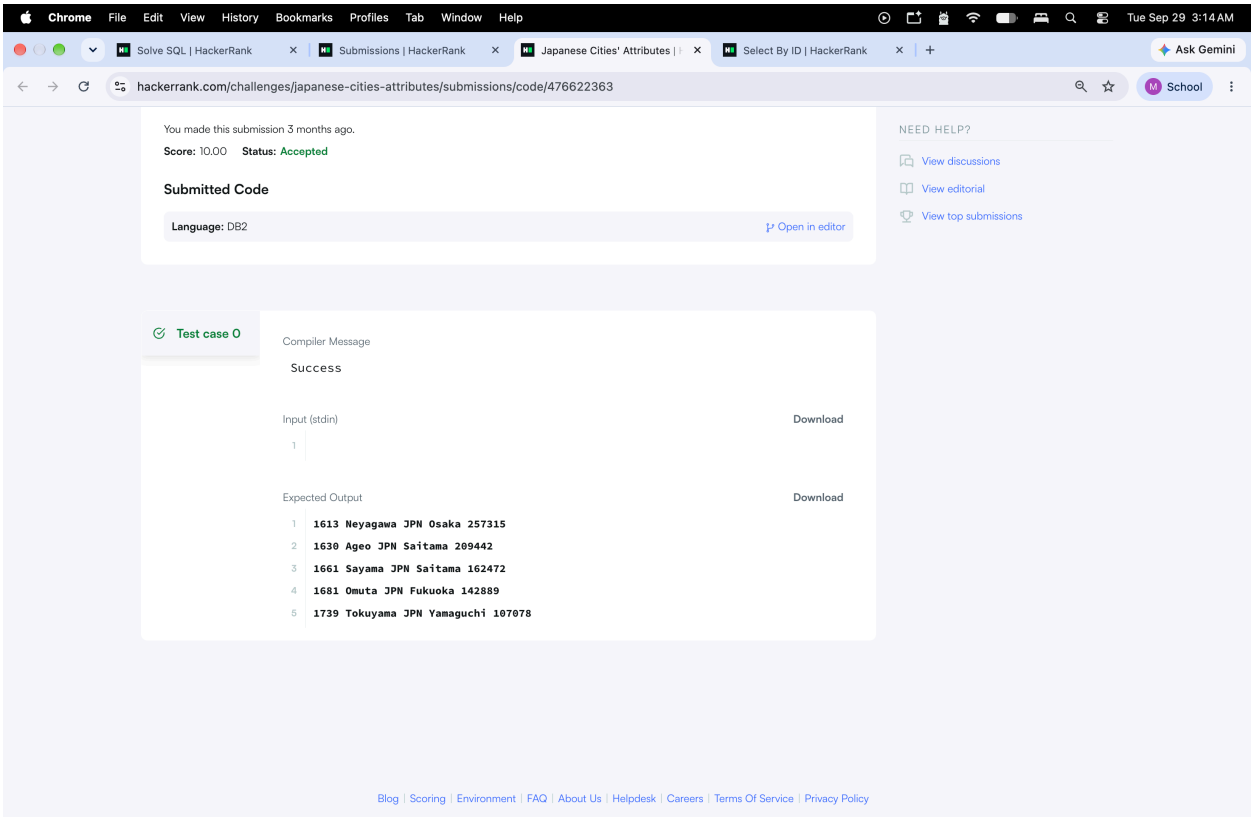


5. Japanese Cities' Attributes

INPUT:

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT:

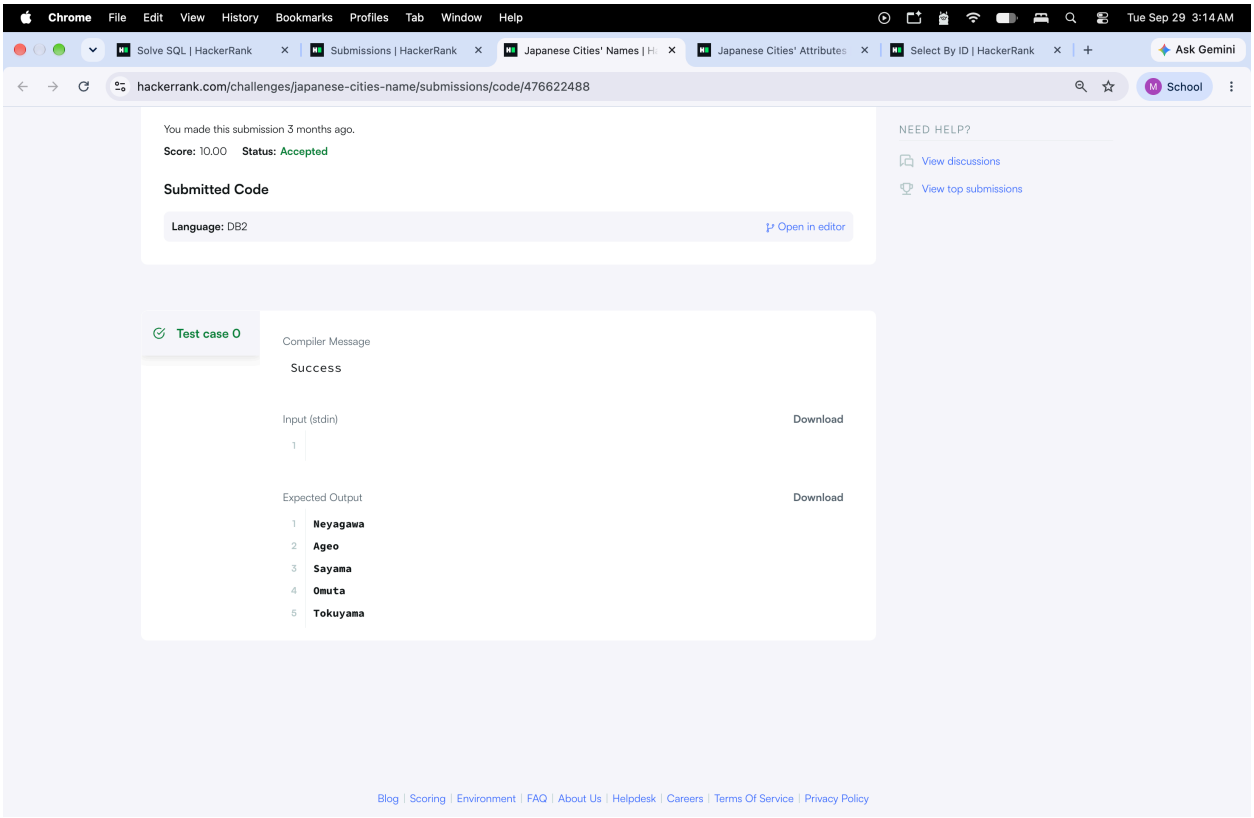


6. Japanese Cities' Names

INPUT:

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT:

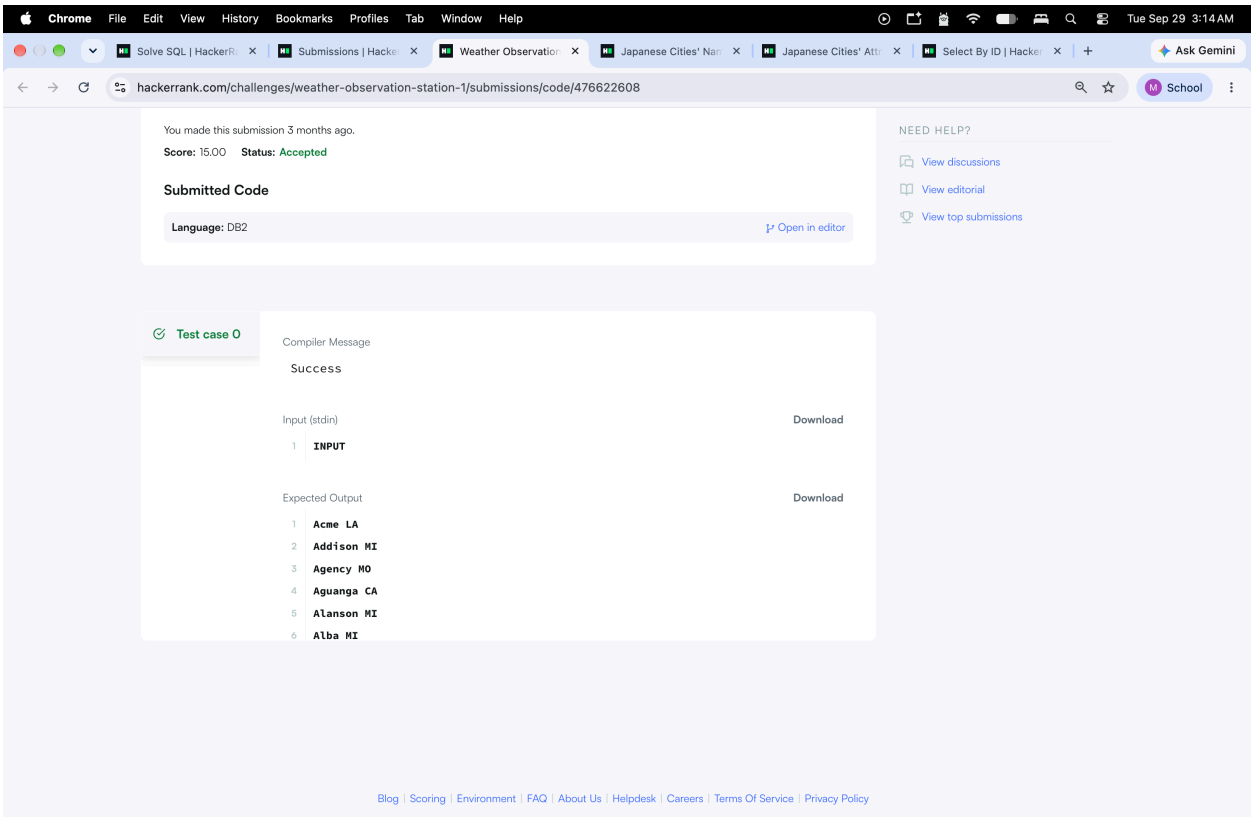


7. Weather Observation Station 1

INPUT:

```
SELECT CITY, STATE
FROM STATION;
```

OUTPUT:

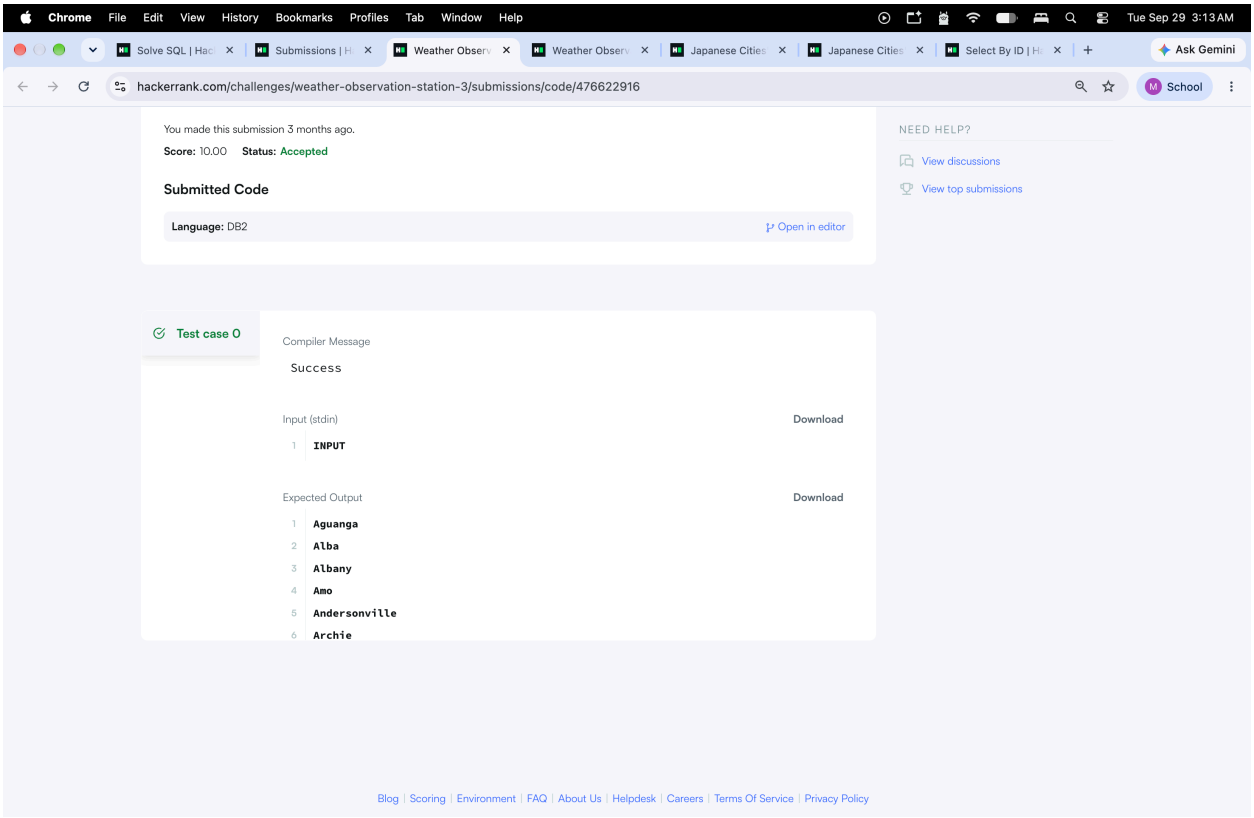


8. Weather Observation Station 3

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE MOD(ID, 2) = 0;
```

OUTPUT:

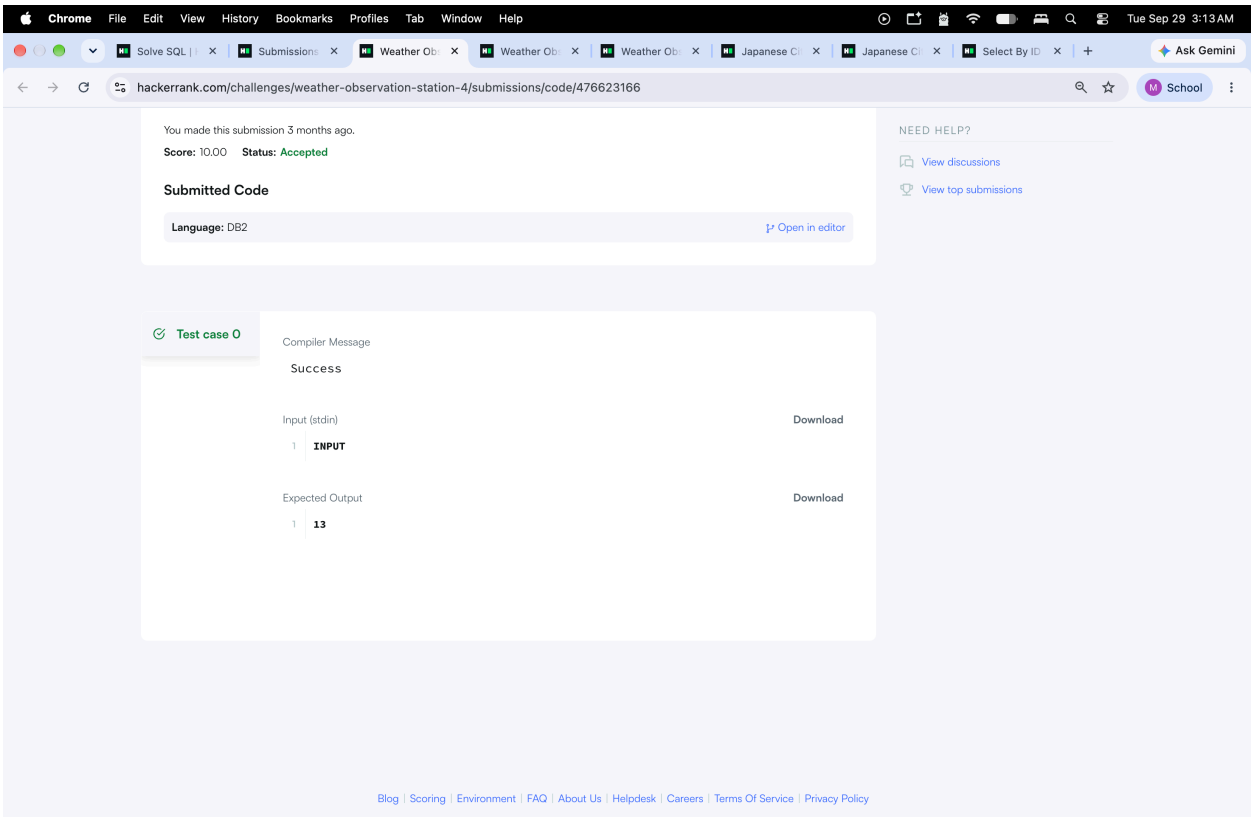


9. Weather Observation Station 4

INPUT:

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY)
FROM STATION;
```

OUTPUT:

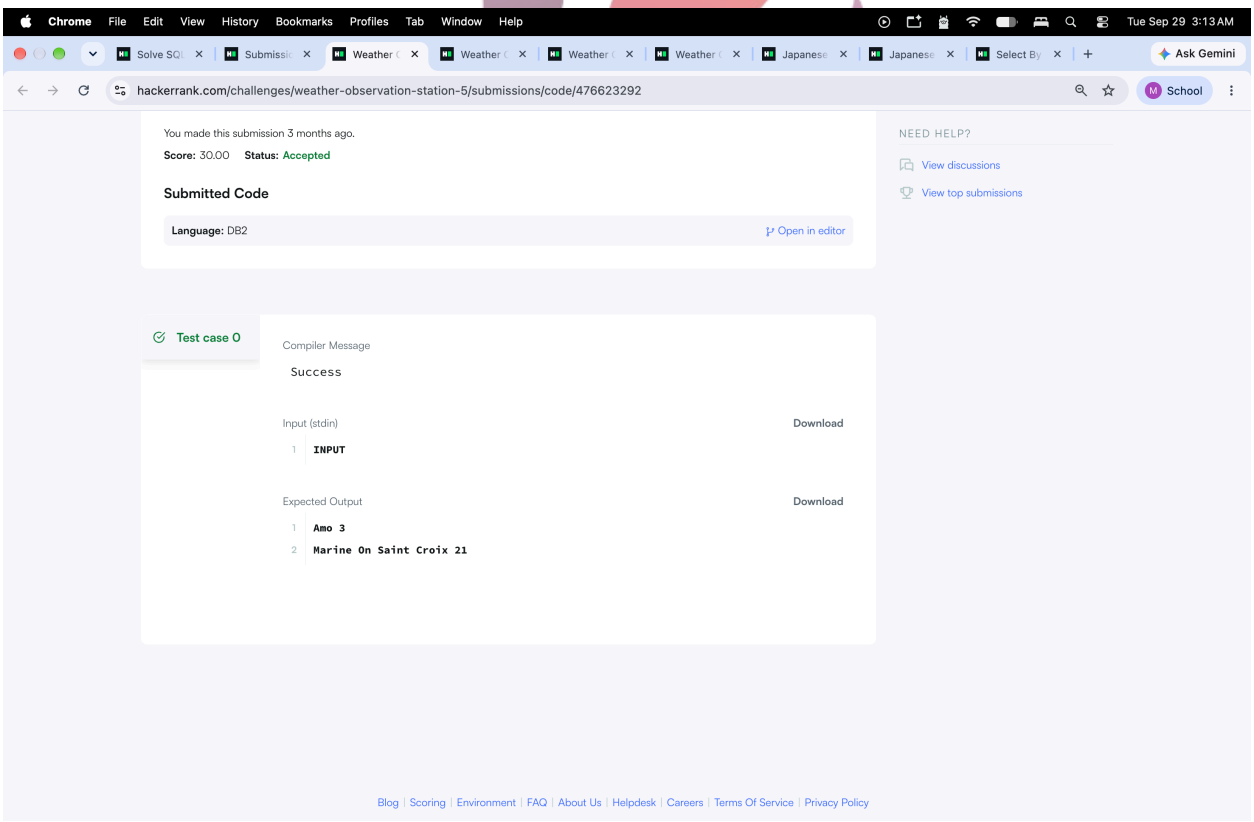


10. Weather Observation Station 5

INPUT:

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) ASC, CITY ASC
FETCH FIRST 1 ROW ONLY;

SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY ASC
FETCH FIRST 1 ROW ONLY;
```

OUTPUT:

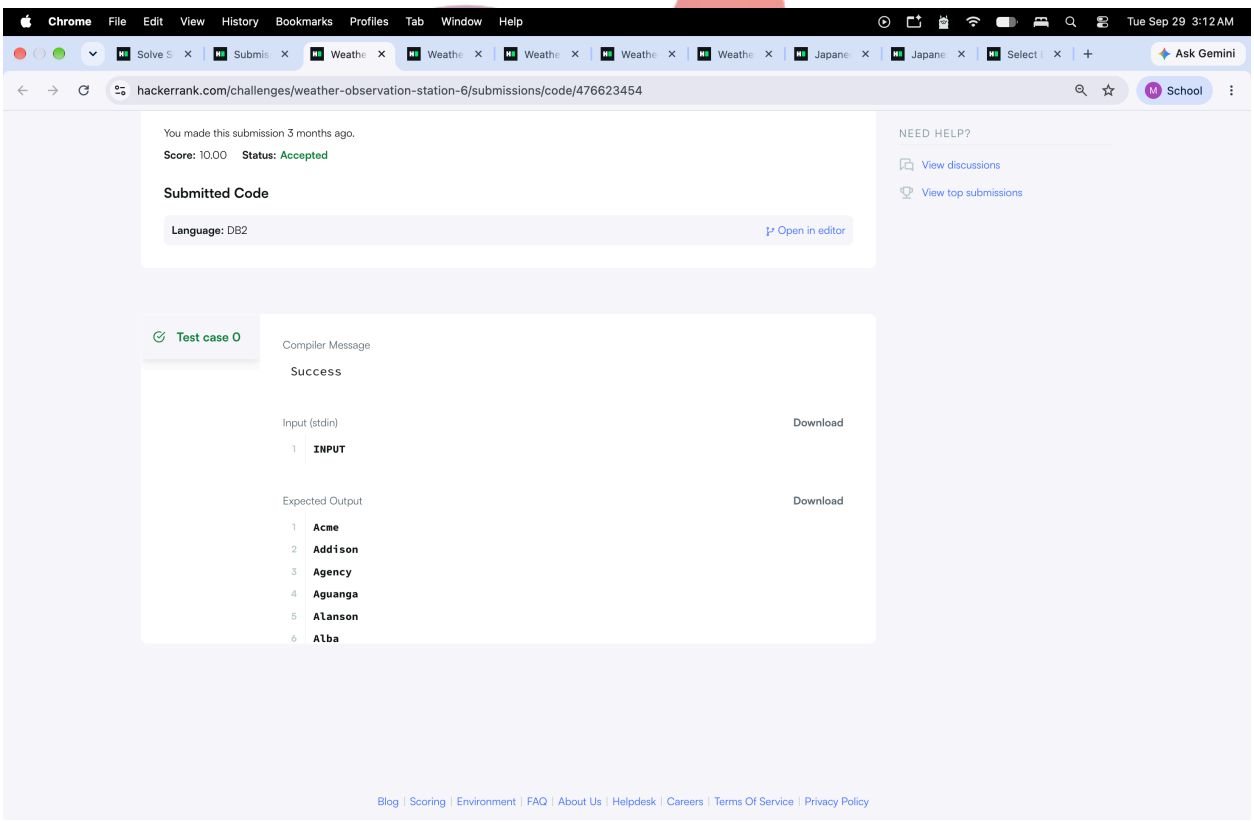


11. Weather Observation Station 6

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE 'A%'
  OR CITY LIKE 'E%'
  OR CITY LIKE 'I%'
  OR CITY LIKE 'O%'
  OR CITY LIKE 'U%';
```

OUTPUT:

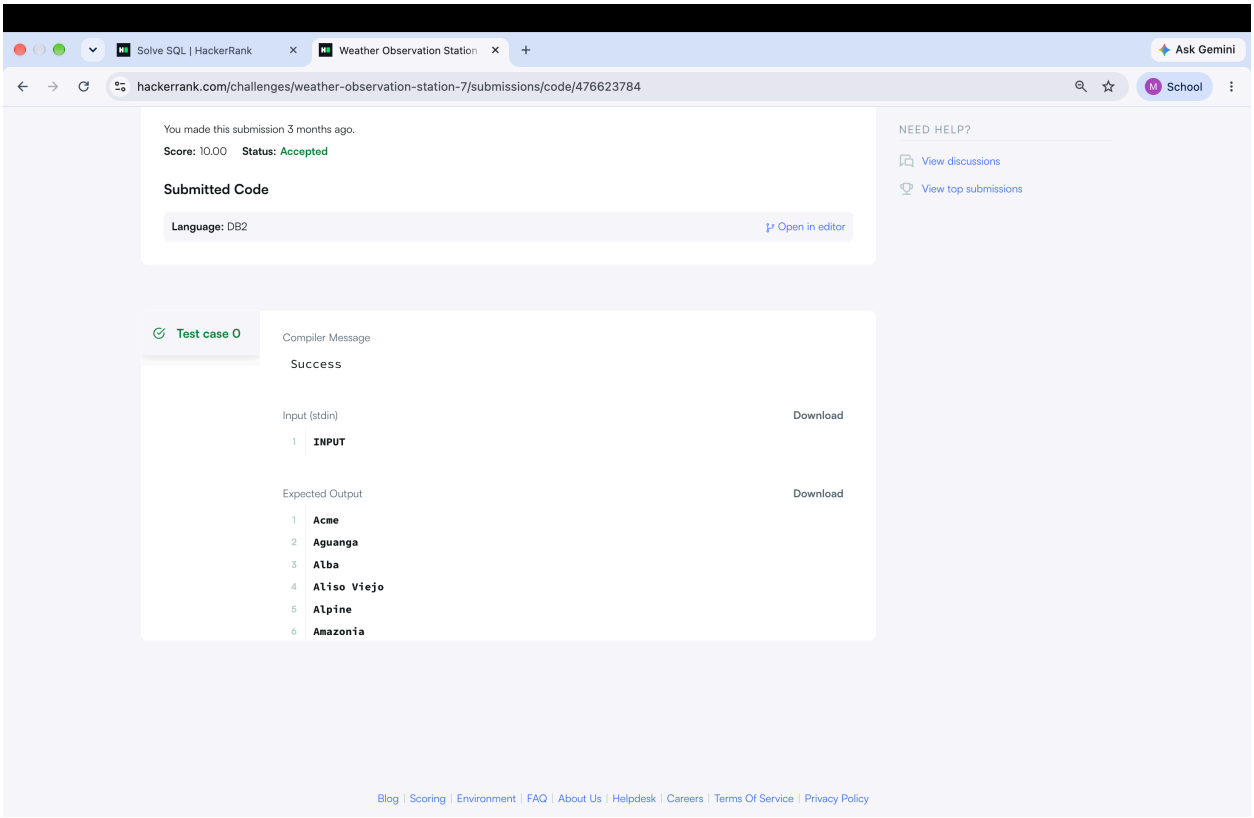


12. Weather Observation Station 7

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

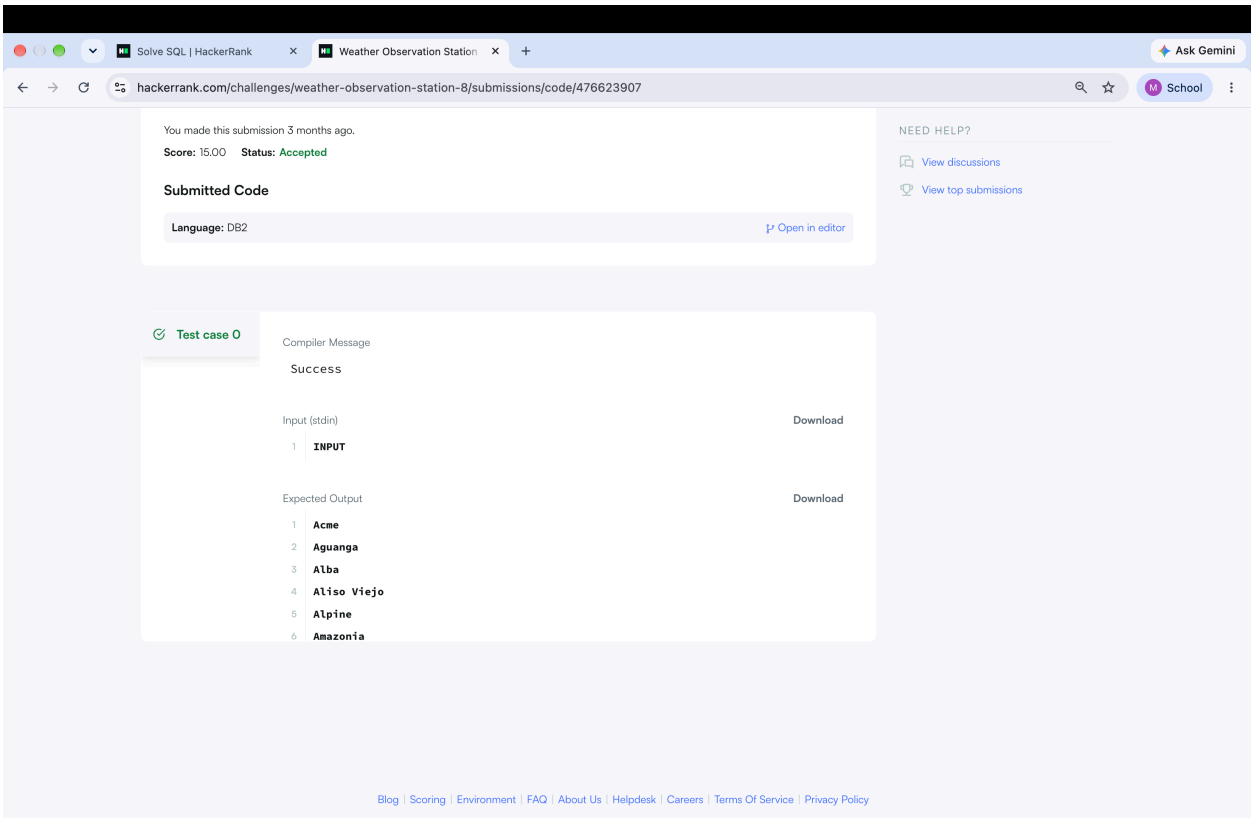


13. Weather Observation Station 8

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u')
 AND LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

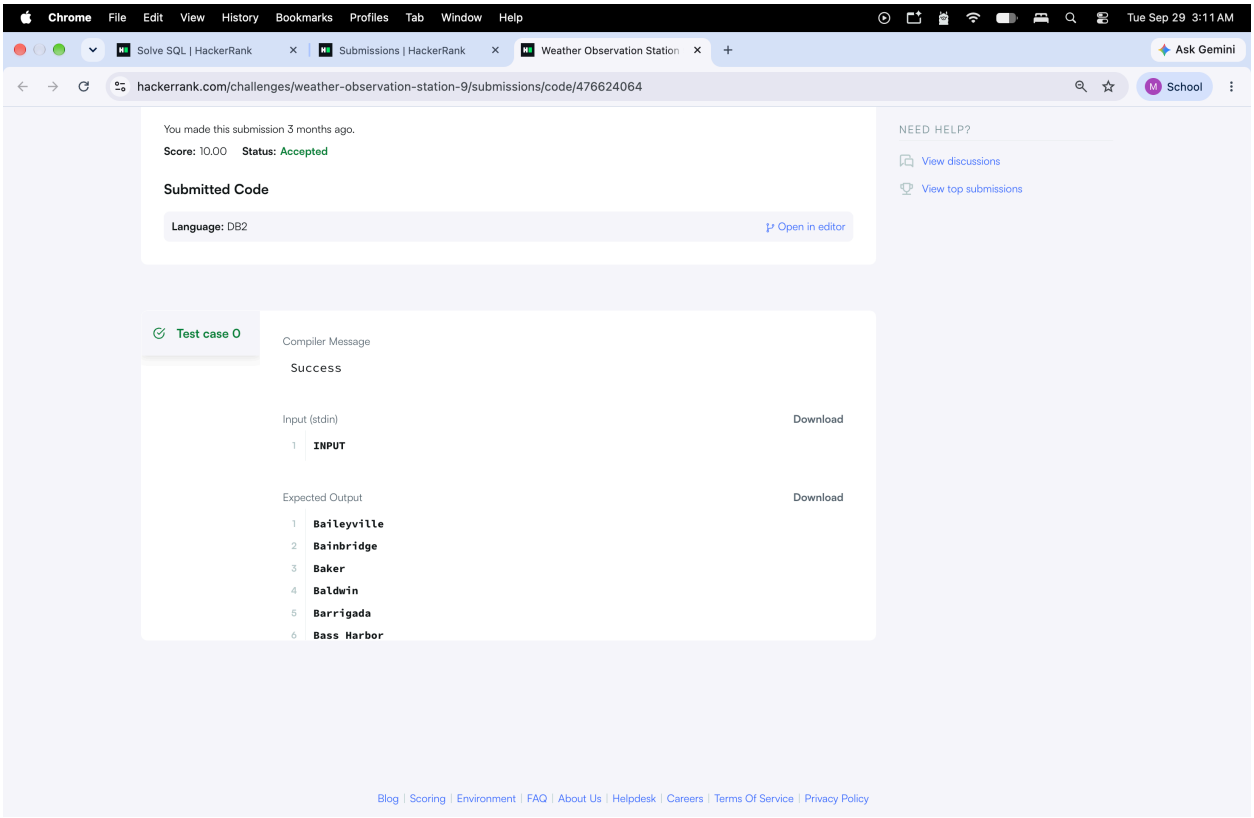


14. Weather Observation Station 9

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

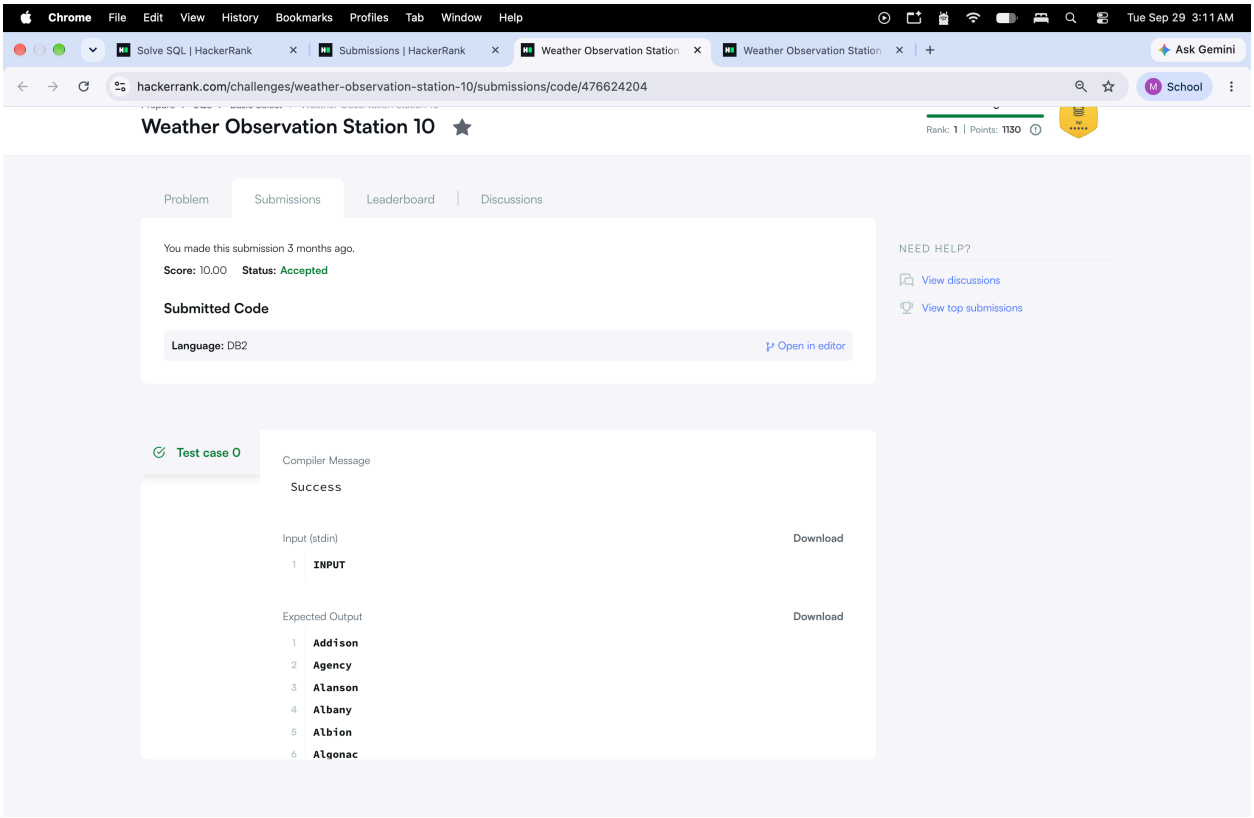


15. Weather Observation Station 10

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

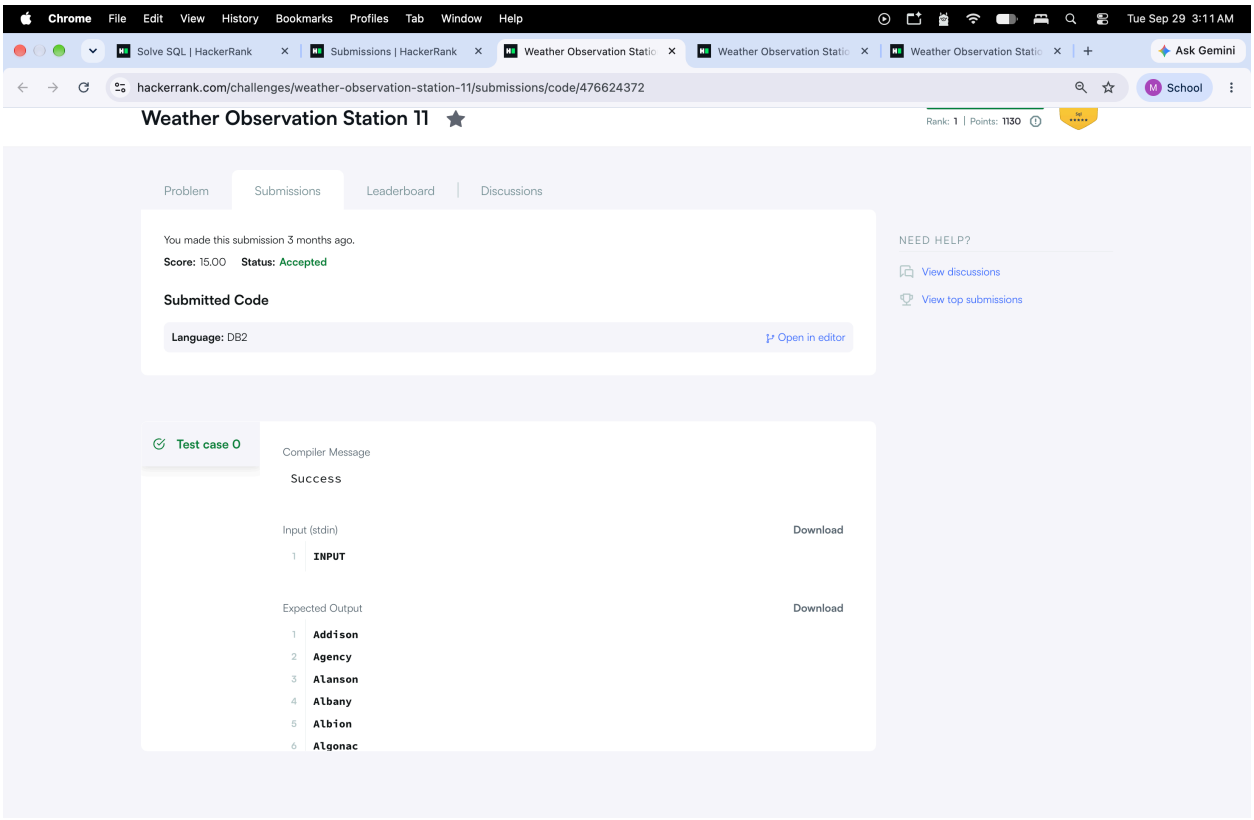


16. Weather Observation Station 11

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
 OR LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

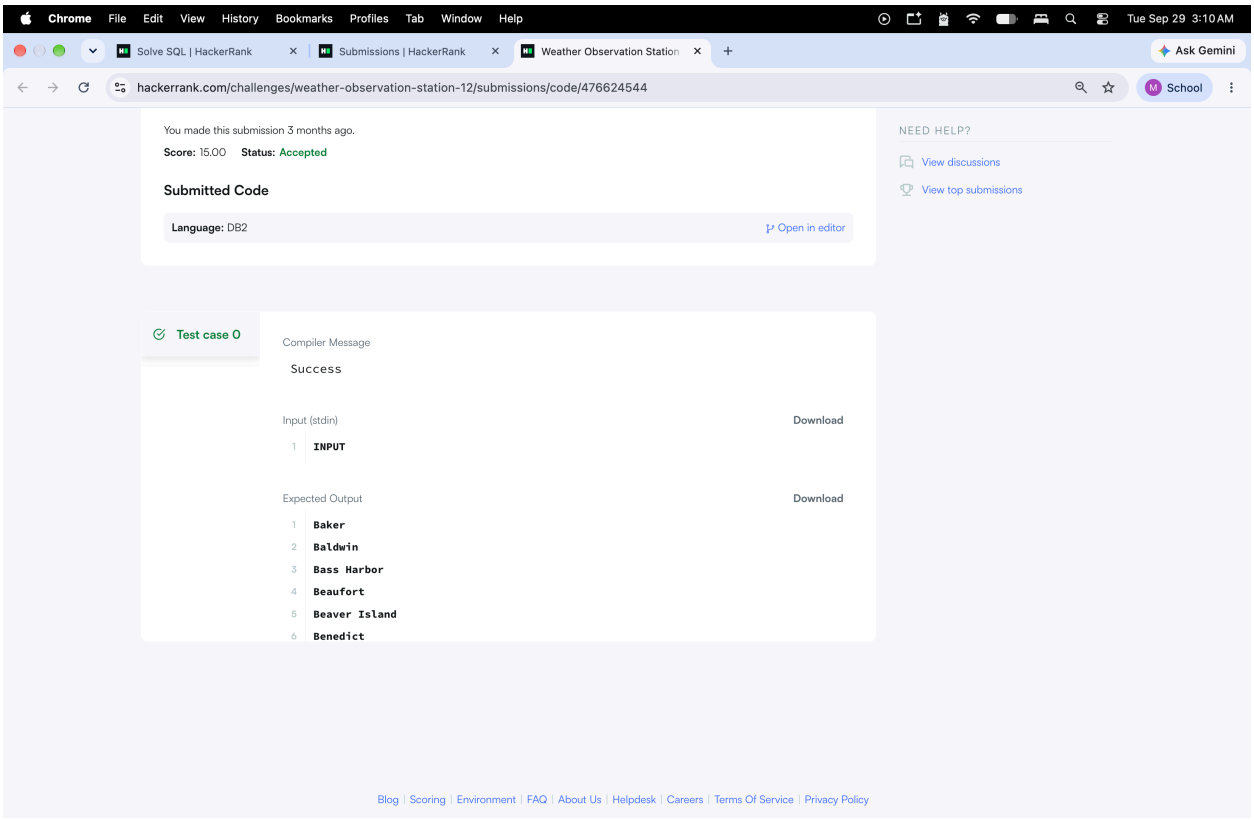


17. Weather Observation Station 12

INPUT:

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
 AND LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT:

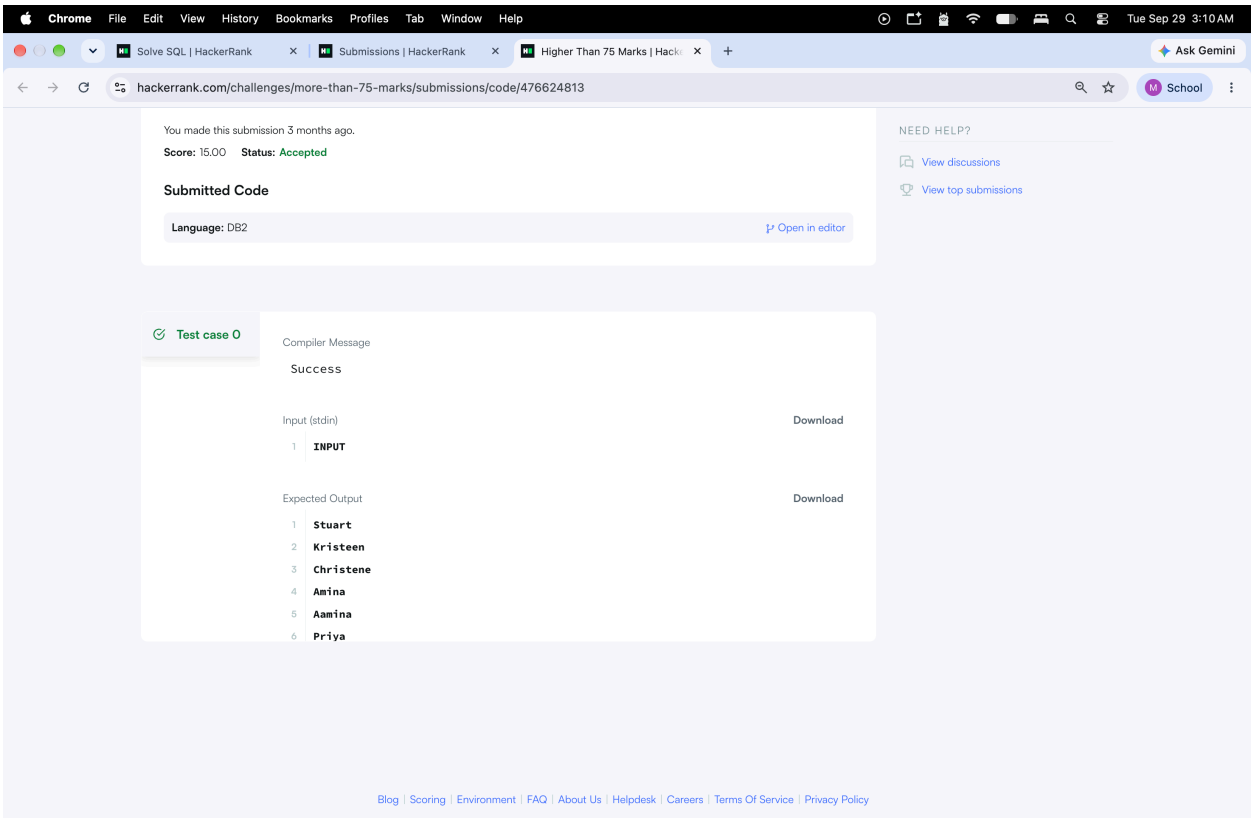


18. Higher Than 75 Marks

INPUT:

```
SELECT NAME
FROM STUDENTS
WHERE MARKS > 75
ORDER BY SUBSTR(NAME, LENGTH(NAME) - 2, 3), ID ASC;
```

OUTPUT:

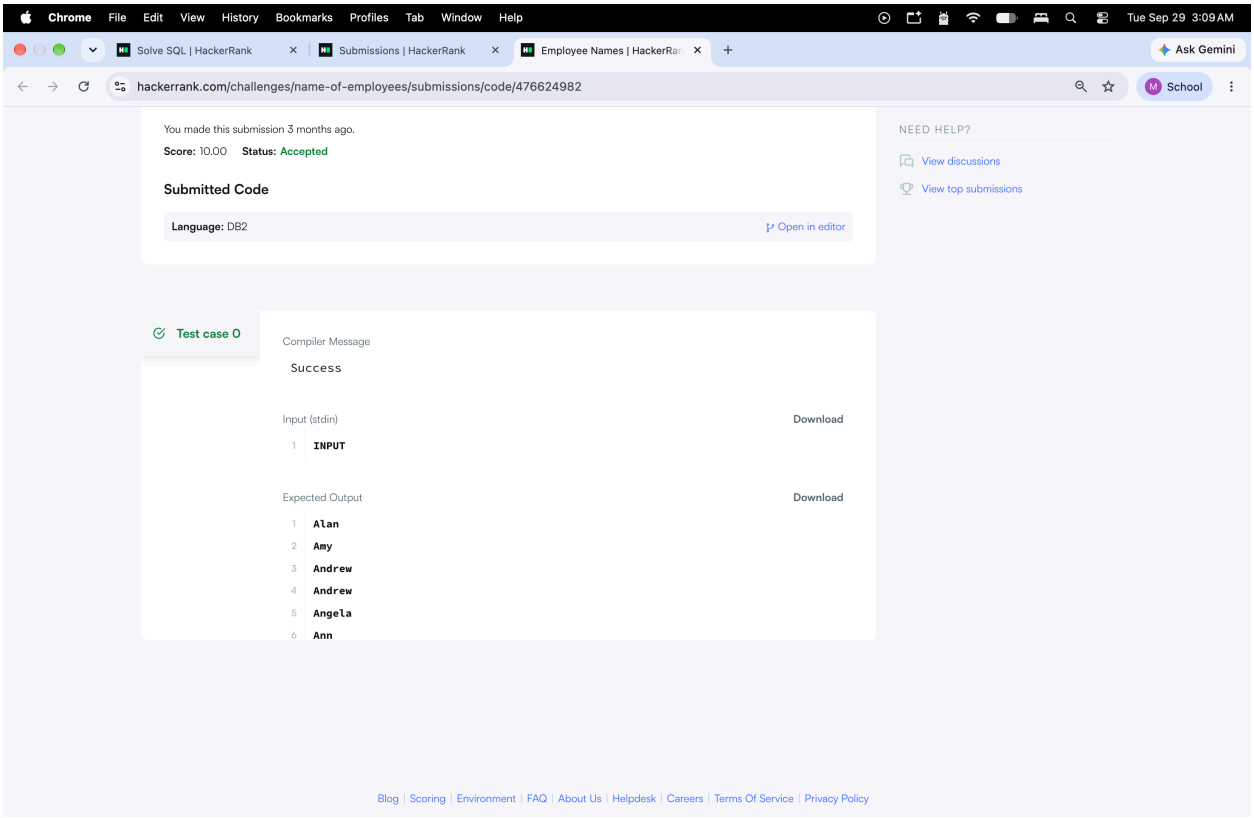


19. Employee Names

INPUT:

```
SELECT name
FROM Employee
ORDER BY name ASC;
```

OUTPUT:

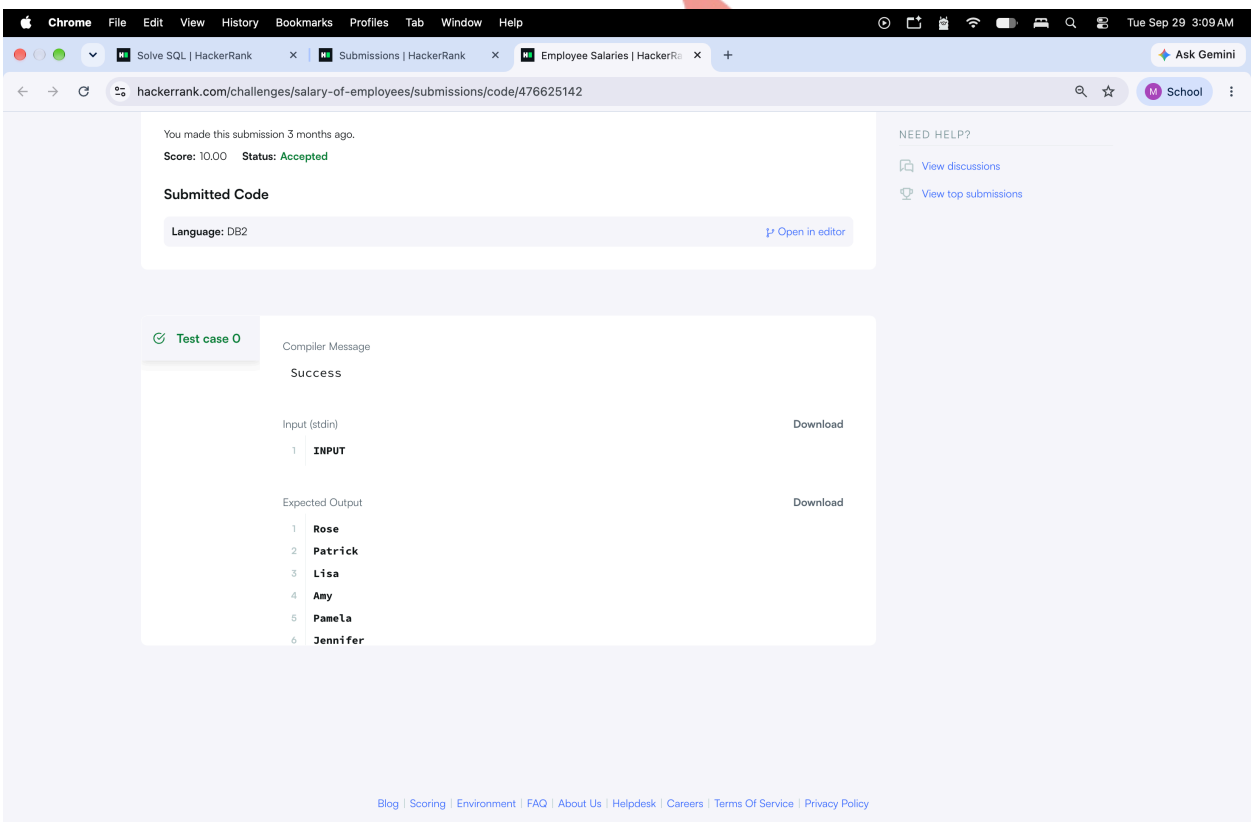


20. Employee Salaries

INPUT:

```
SELECT name
FROM Employee
WHERE salary > 2000
  AND months < 10
ORDER BY employee_id ASC;
```

OUTPUT:



21. Draw The Triangle 2

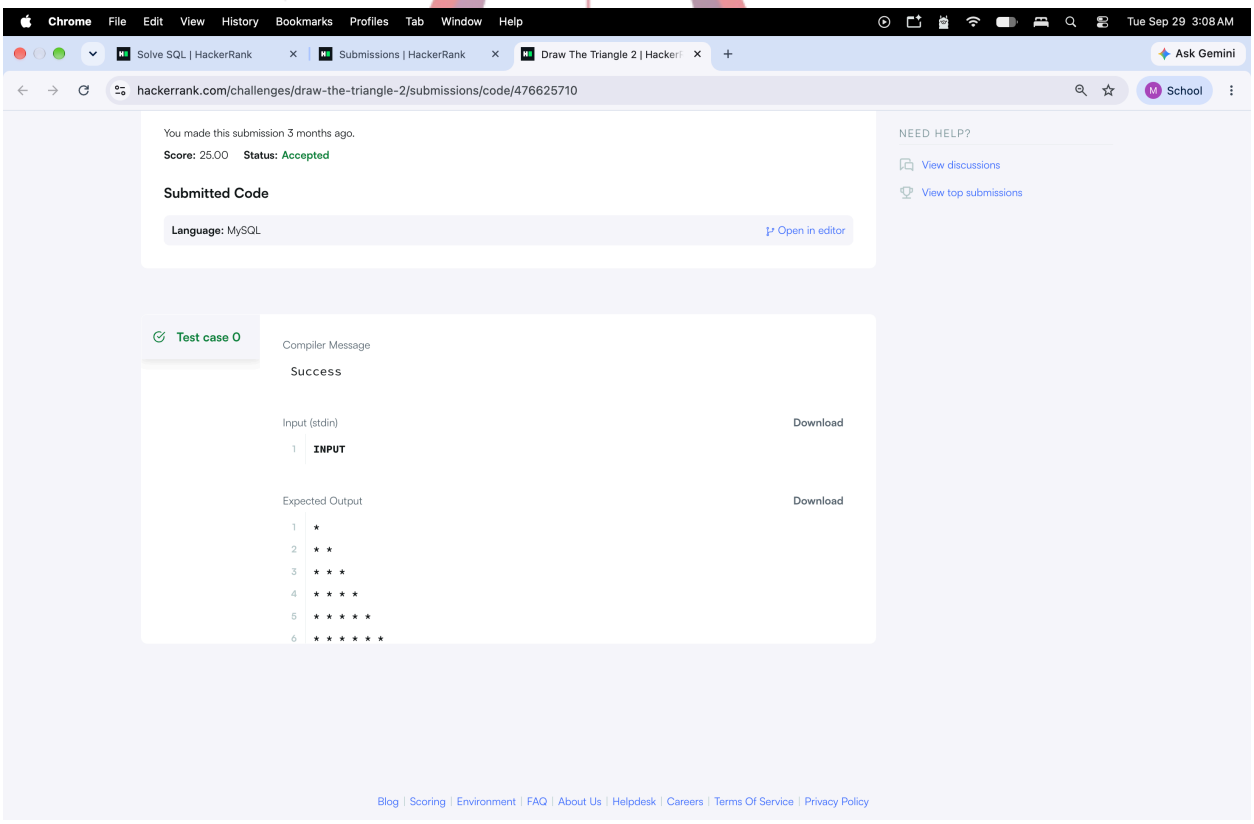
Challenge: Draw The Triangle 2

Language: MYSQL **Score:** 1.0

INPUT:

```
WITH RECURSIVE PATTERN AS (
  SELECT 1 AS N, CAST('*' AS CHAR(50)) AS STR

  UNION ALL

  SELECT N + 1, CONCAT(STR, ' *')
  FROM PATTERN
  WHERE N < 20
)
SELECT STR
FROM PATTERN;
```

OUTPUT:



22. Print Prime Numbers

Challenge: Print Prime Numbers

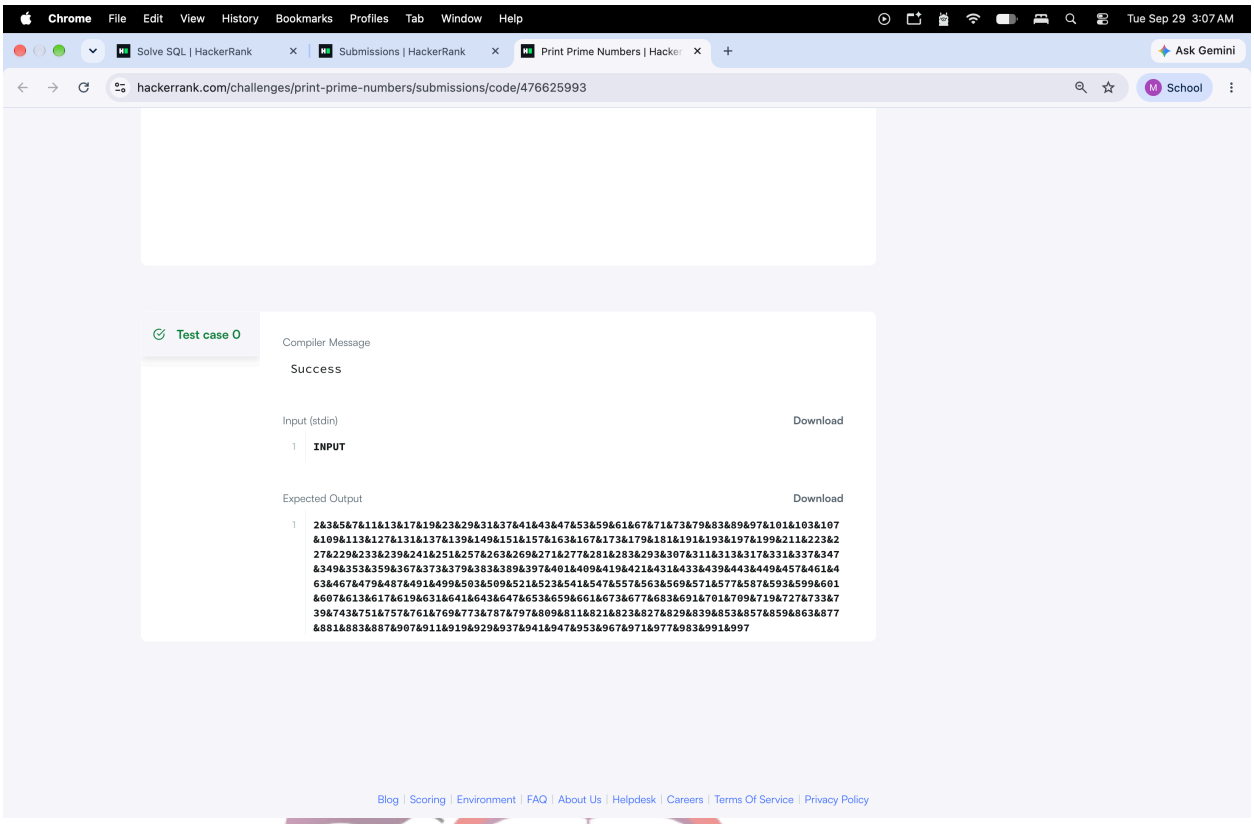
Language: MYSQL **Score:** 1.0

INPUT:

```
WITH RECURSIVE NUMS AS (
  SELECT 2 AS N
  UNION ALL
  SELECT N + 1
  FROM NUMS
  WHERE N < 1000
),
PRIMES AS (
  SELECT N
  FROM NUMS A
  WHERE NOT EXISTS (
    SELECT 1
    FROM NUMS B
    WHERE B.N < A.N
     AND B.N * B.N <= A.N
     AND MOD(A.N, B.N) = 0
  )
)
SELECT GROUP_CONCAT(N SEPARATOR '&')
FROM PRIMES;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

OUTPUT:


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Tue Sep 29 3:07 AM
Solve SQL | HackerRank
Submissions | HackerRank
Print Prime Numbers | Hacker
Ask Gemini
hackerrank.com/challenges/print-prime-numbers/submissions/code/476625993
School
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2&3&5&7&11&13&17&19&23&29&31&37&41&43&47&53&59&61&67&71&73&79&83&89&97&101&103&107
&109&113&127&131&137&139&149&151&157&163&167&173&179&181&191&193&197&199&211&223&2
27&229&233&239&241&251&257&263&269&271&277&281&283&293&307&311&313&317&331&337&347
&349&353&359&367&373&379&383&389&397&401&409&419&421&431&433&439&443&449&457&461&4
63&467&479&487&491&499&503&509&521&523&541&547&557&563&569&571&577&587&593&599&601
&607&613&617&619&631&641&643&647&653&659&661&673&677&683&691&701&709&719&727&733&7
39&743&751&757&761&769&773&787&797&809&811&821&823&827&829&839&853&857&859&863&877
&881&883&887&907&911&919&929&937&941&947&953&967&971&977&983&991&997
Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

AUTONOMOUS

23. Occupations

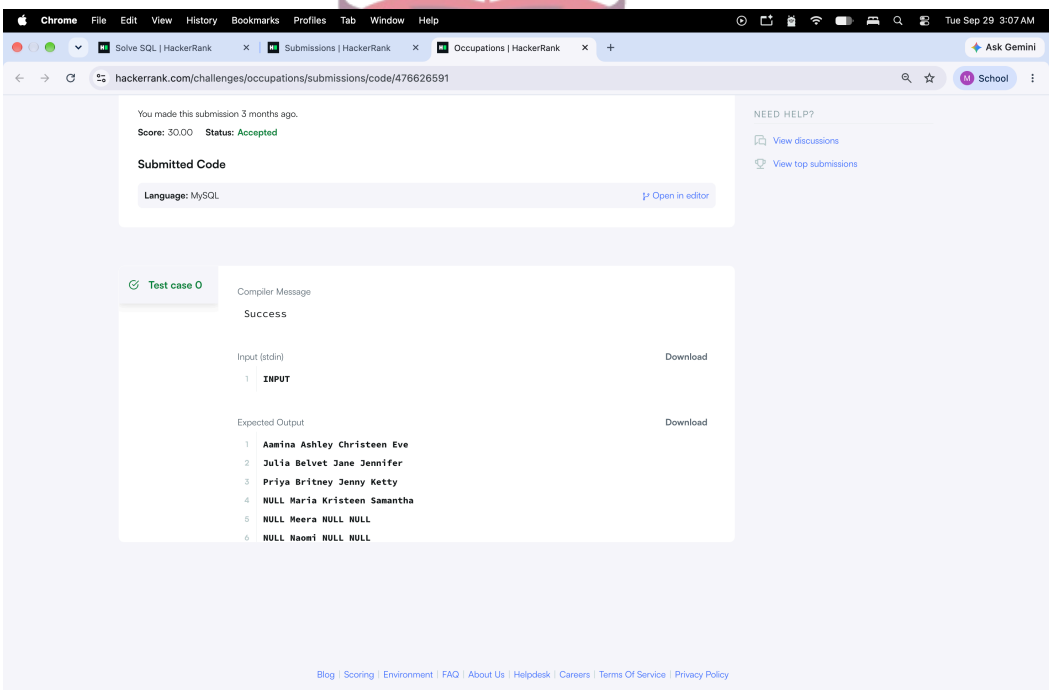
Challenge: Occupations

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS temp
GROUP BY rn
ORDER BY rn;
```

OUTPUT:



24. Binary Tree Nodes

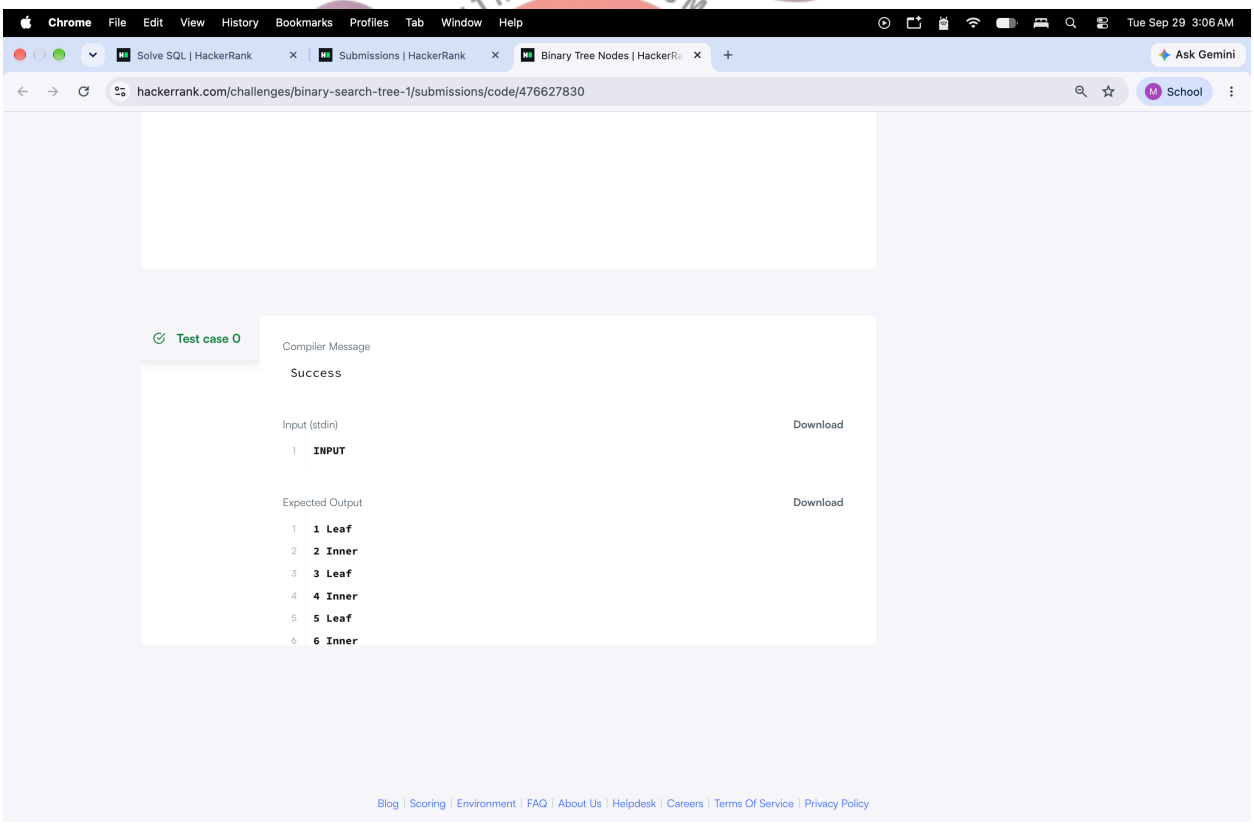
Challenge: Binary Tree Nodes

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT
    N,
    CASE
        WHEN P IS NULL THEN 'Root'
        WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
        ELSE 'Inner'
    END AS NodeType
FROM BST
ORDER BY N;
```

OUTPUT:



25. New Companies

Challenge: New Companies

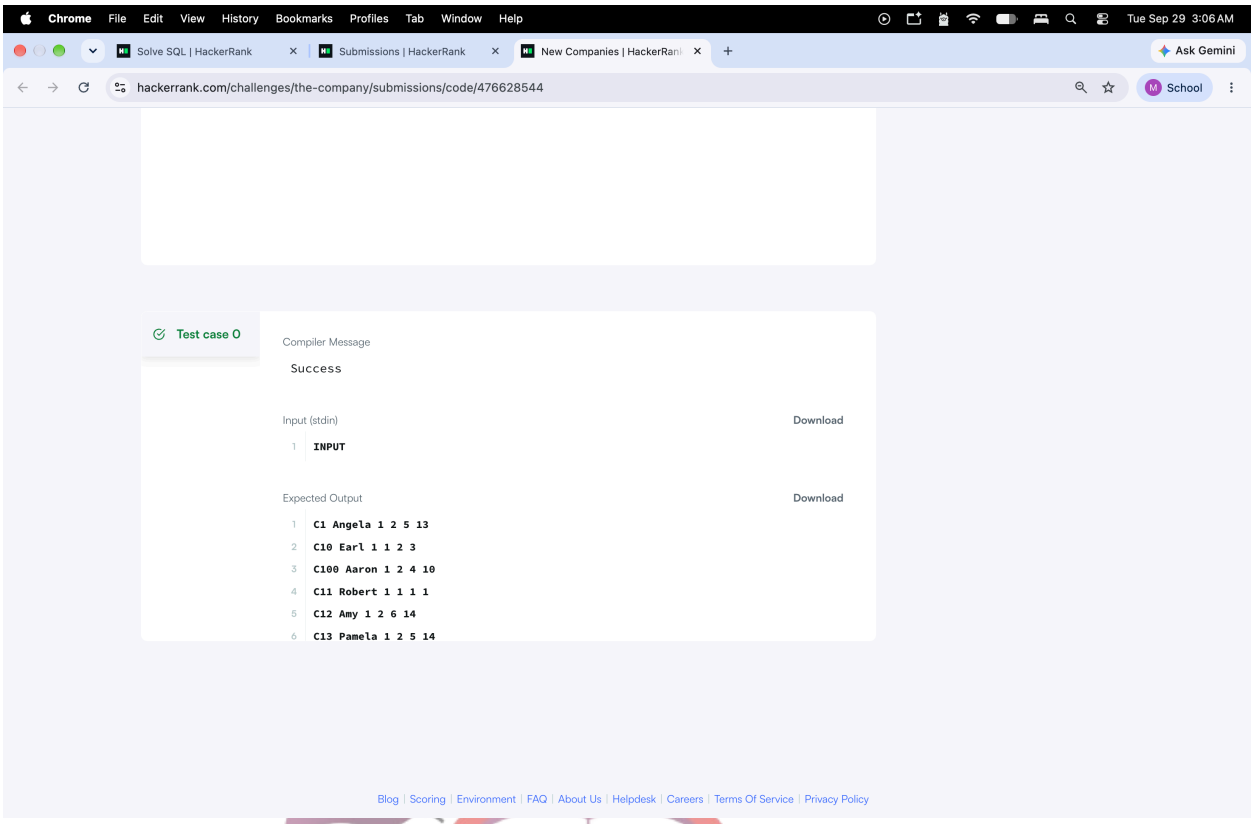
Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT
    C.company_code,
    C.founder,
    COUNT(DISTINCT LM.lead_manager_code),
    COUNT(DISTINCT SM.senior_manager_code),
    COUNT(DISTINCT M.manager_code),
    COUNT(DISTINCT E.employee_code)
FROM Company C
LEFT JOIN Lead_Manager LM
    ON C.company_code = LM.company_code
LEFT JOIN Senior_Manager SM
    ON C.company_code = SM.company_code
LEFT JOIN Manager M
    ON C.company_code = M.company_code
LEFT JOIN Employee E
    ON C.company_code = E.company_code
GROUP BY C.company_code, C.founder
ORDER BY C.company_code;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT:



AUTONOMOUS

26. Type of Triangle

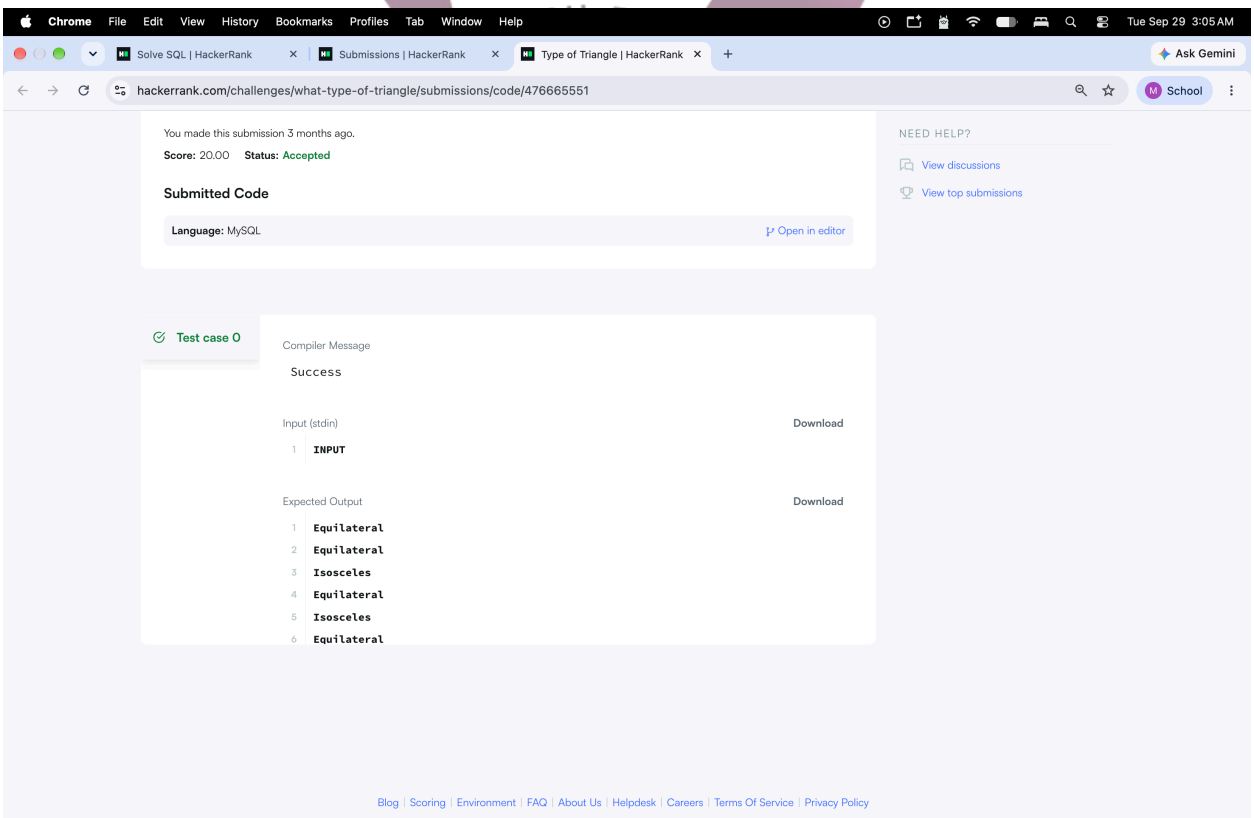
Challenge: Type of Triangle

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT
  CASE
    WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
    WHEN A = B AND B = C THEN 'Equilateral'
    WHEN A = B OR B = C OR A = C THEN 'Isosceles'
    ELSE 'Scalene'
  END
FROM TRIANGLES;
```

OUTPUT:



27. The PADS

Challenge: The PADS

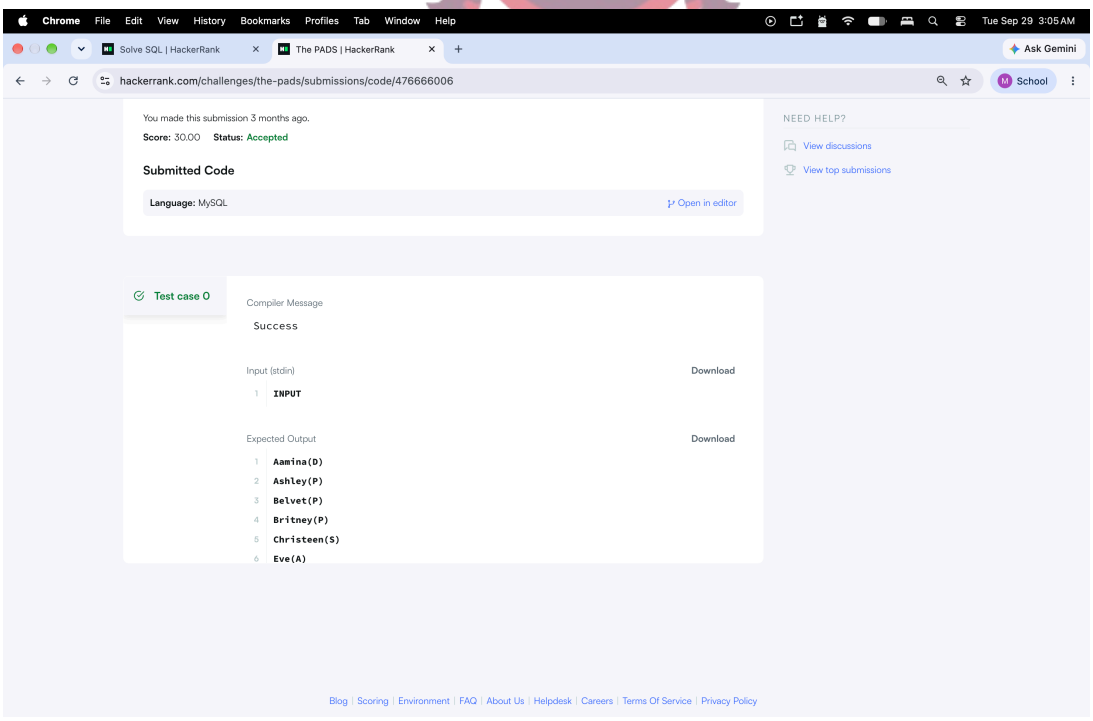
Language: MYSQL **Score:** 1.0

INPUT:

```
-- Query 1
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;

-- Query 2
SELECT CONCAT(
   'There are a total of ',
   COUNT(*),
   ' ',
   LOWER(Occupation),
   's.'
)
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(*), Occupation;
```

OUTPUT:

:



28. SQL Project Planning

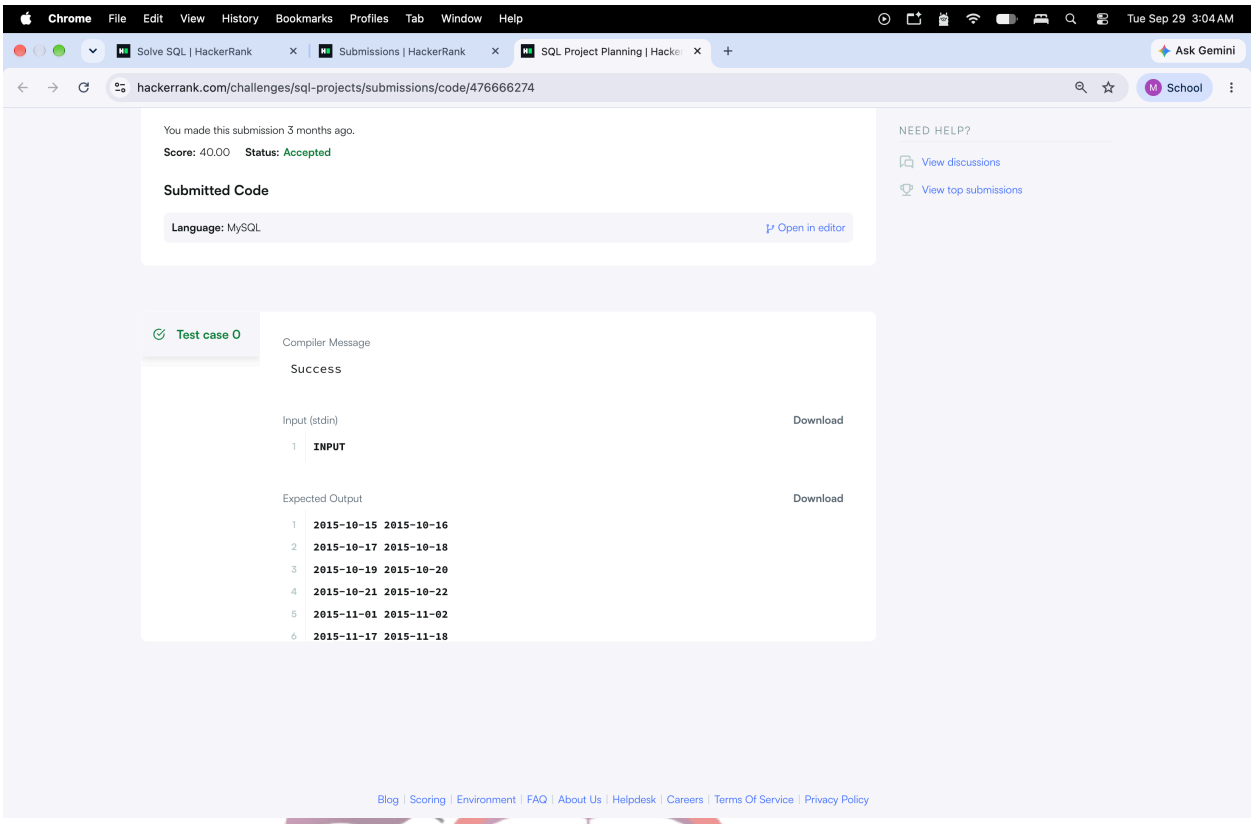
Challenge: SQL Project Planning

Language: MYSQL **Score:** 1.0

INPUT:

```
WITH StartProjects AS (
    SELECT Start_Date,
           ROW_NUMBER() OVER (ORDER BY Start_Date) AS rn
    FROM Projects
    WHERE Start_Date NOT IN (
        SELECT End_Date
        FROM Projects
    )
),
EndProjects AS (
    SELECT End_Date,
           ROW_NUMBER() OVER (ORDER BY End_Date) AS rn
    FROM Projects
    WHERE End_Date NOT IN (
        SELECT Start_Date
        FROM Projects
    )
)
SELECT
    s.Start_Date,
    e.End_Date
FROM StartProjects s
JOIN EndProjects e
ON s.rn = e.rn
ORDER BY DATEDIFF(e.End_Date, s.Start_Date), s.Start_Date;
```

OUTPUT:


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Tue Sep 29 3:04 AM
Solve SQL | HackerRank
Submissions | HackerRank
SQL Project Planning | Hacke
Ask Gemini
hackerrank.com/challenges/sql-projects/submissions/code/476666274
School
You made this submission 3 months ago.
Score: 40.00 Status: Accepted
Submitted Code
Language: MySQL
Open in editor
NEED HELP?
View discussions
View top submissions
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2015-10-15 2015-10-16
2 2015-10-17 2015-10-18
3 2015-10-19 2015-10-20
4 2015-10-21 2015-10-22
5 2015-11-01 2015-11-02
6 2015-11-17 2015-11-18
Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

AUTONOMOUS

29. Placements

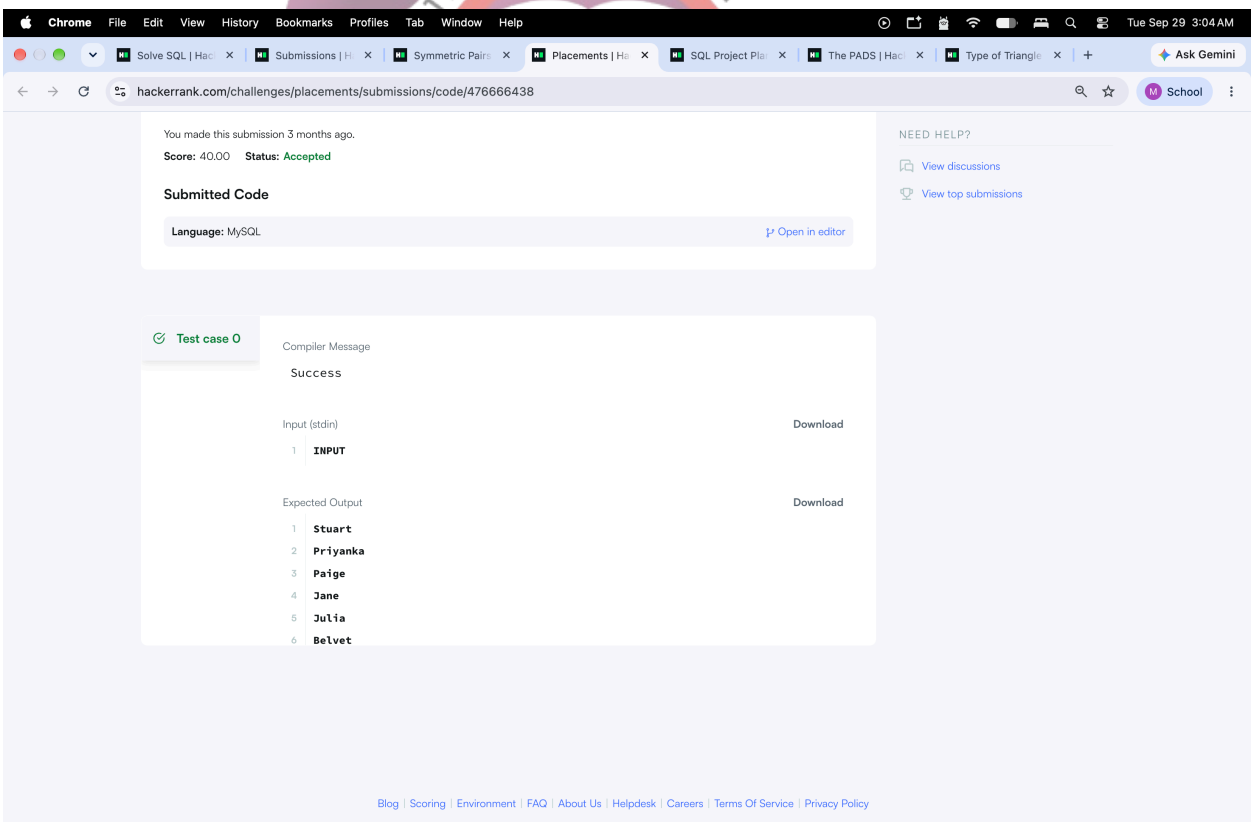
Challenge: Placements

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT s.Name
FROM Students s
JOIN Friends f
   ON s.ID = f.ID
JOIN Packages p1
   ON s.ID = p1.ID
JOIN Packages p2
   ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```

OUTPUT:



30. Symmetric Pairs

Challenge: Symmetric Pairs

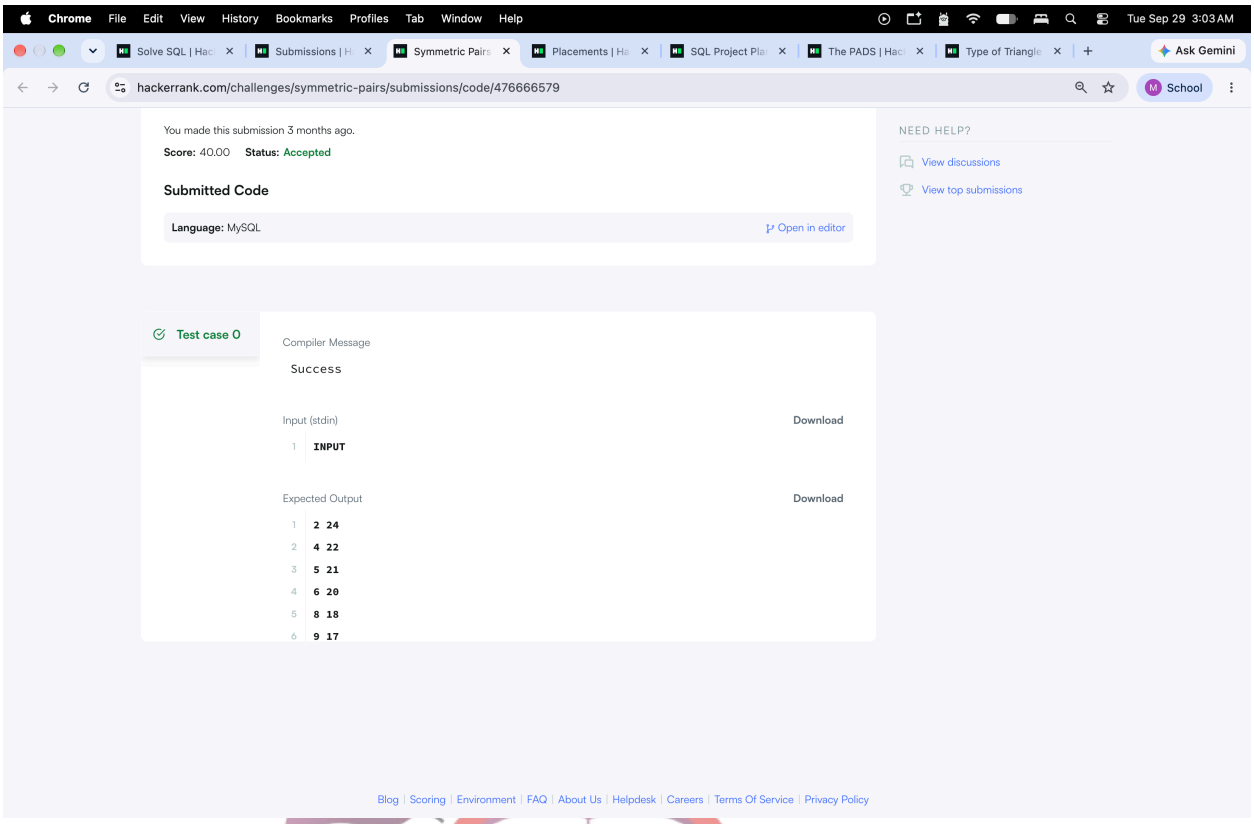
Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT DISTINCT
    f1.X,
    f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y
   AND f1.Y = f2.X
WHERE
    f1.X < f1.Y
    OR (
        f1.X = f1.Y
        AND f1.X IN (
            SELECT X
            FROM Functions
            WHERE X = Y
            GROUP BY X
            HAVING COUNT(*) > 1
        )
    )
ORDER BY f1.X;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

OUTPUT:



AUTONOMOUS

31. Interviews

Challenge: Interviews

Language: MYSQL **Score:** 1.0

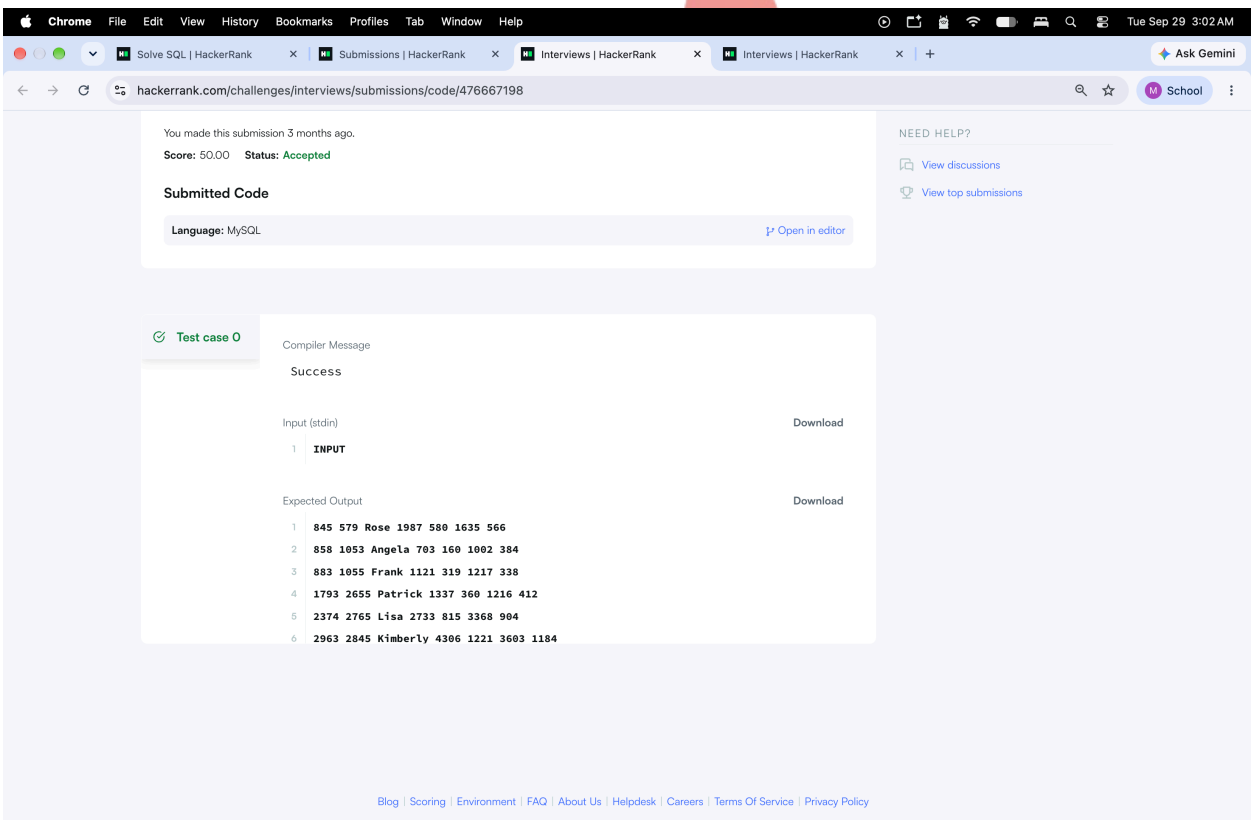
INPUT:

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(IFNULL(s.total_submissions,0)),
    SUM(IFNULL(s.total_accepted_submissions,0)),
    SUM(IFNULL(v.total_views,0)),
    SUM(IFNULL(v.total_unique_views,0))
FROM Contests c
JOIN Colleges co
    ON c.contest_id = co.contest_id
JOIN Challenges ch
    ON co.college_id = ch.college_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s
ON ch.challenge_id = s.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

```
GROUP BY
    c.contest_id,
    c.hacker_id,
    c.name
HAVING
    SUM(IFNULL(s.total_submissions,0)) +
    SUM(IFNULL(s.total_accepted_submissions,0)) +
    SUM(IFNULL(v.total_views,0)) +
    SUM(IFNULL(v.total_unique_views,0)) > 0
ORDER BY c.contest_id;
```

OUTPUT:



32. 15 Days of Learning SQL

Challenge: 15 Days of Learning SQL

Language: MYSQL **Score:** 1.0

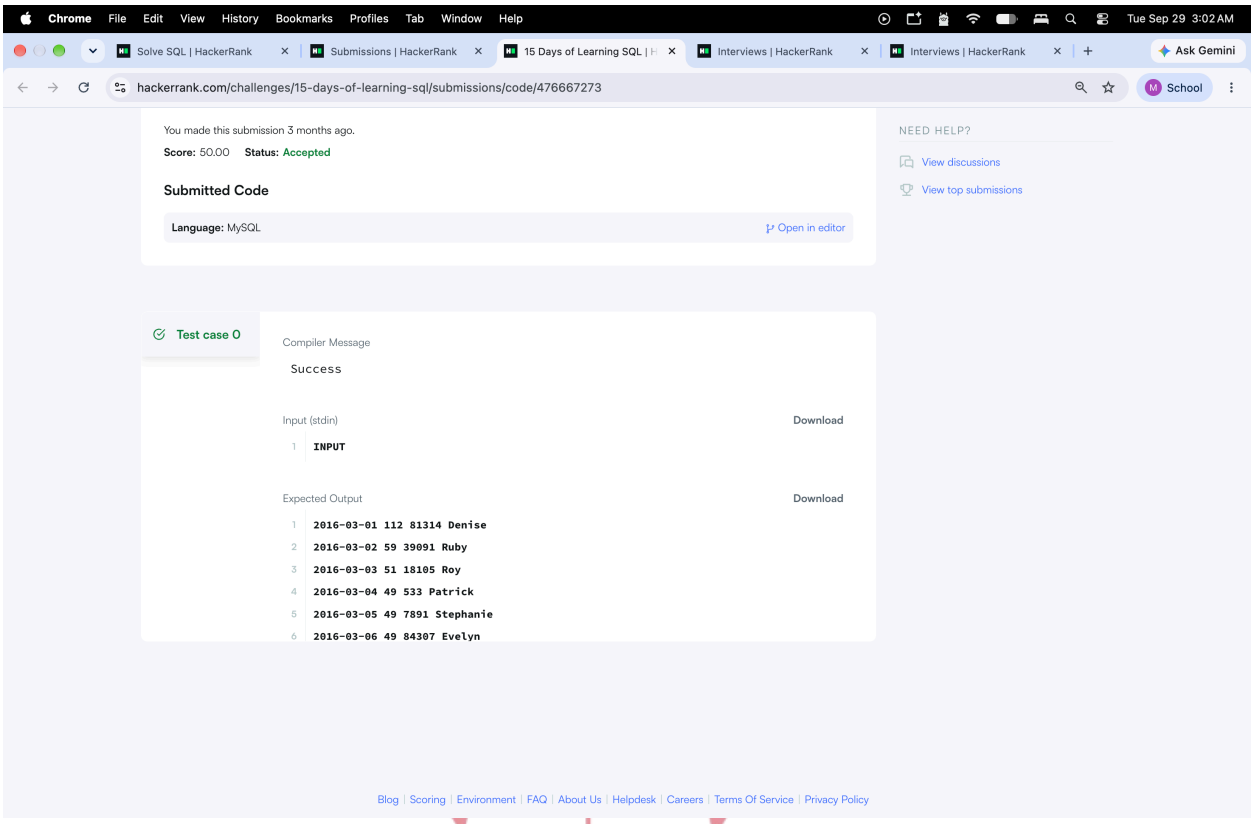
INPUT:

```
SELECT
    s.submission_date,
    (
        SELECT COUNT(*)
        FROM (
            SELECT hacker_id
            FROM Submissions s2
            WHERE s2.submission_date <= s.submission_date
            GROUP BY hacker_id
            HAVING COUNT(DISTINCT submission_date) =
                DATEDIFF(s.submission_date, '2016-03-01') + 1
        ) t
    ) AS total_hackers,
    h.hacker_id,
    h.name
FROM
(
    SELECT
        submission_date,
        hacker_id,
        COUNT(*) AS cnt,
        RANK() OVER (
            PARTITION BY submission_date
            ORDER BY COUNT(*) DESC, hacker_id
        ) AS rnk
    FROM Submissions
    GROUP BY submission_date, hacker_id
) s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.rnk = 1
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

```
ORDER BY s.submission_date;
```

OUTPUT:



AUTONOMOUS

33. Weather Observation Station 13

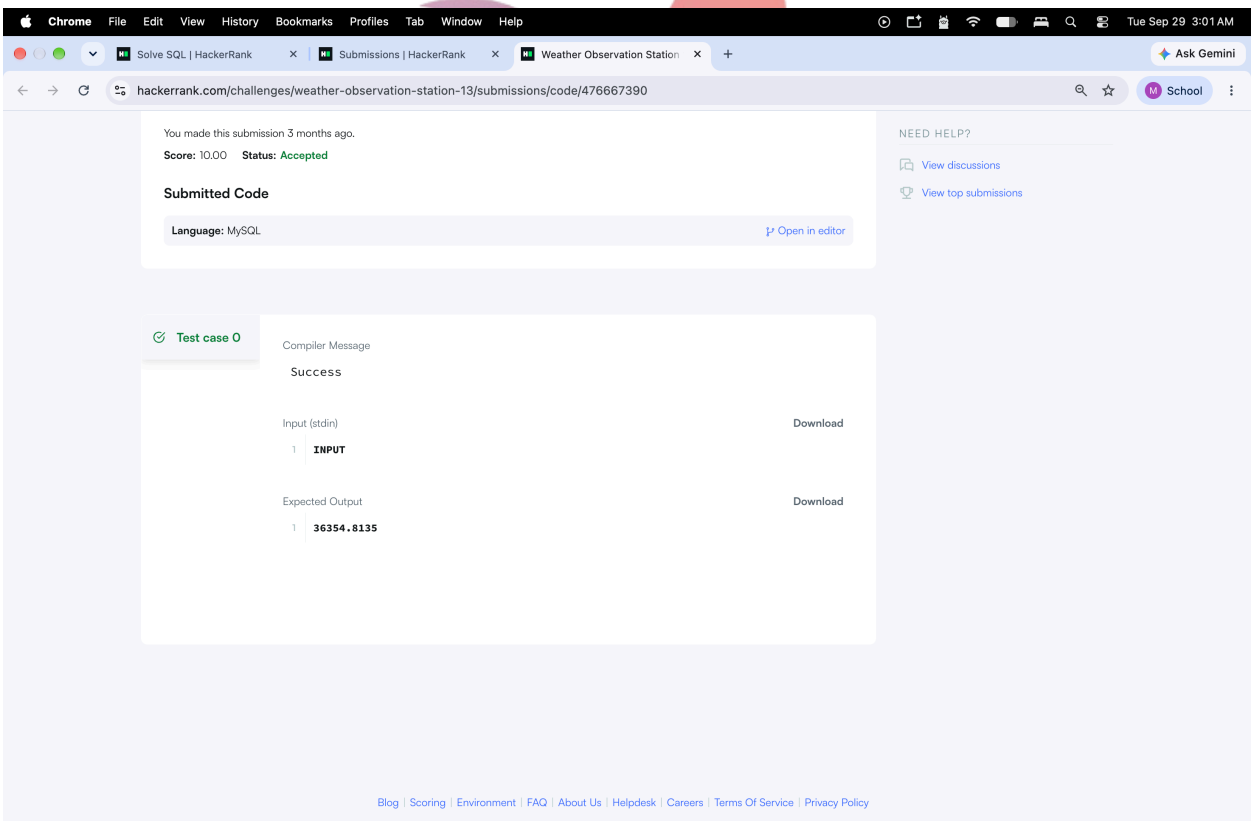
Challenge: Weather Observation Station 13

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT:



34. Weather Observation Station 14

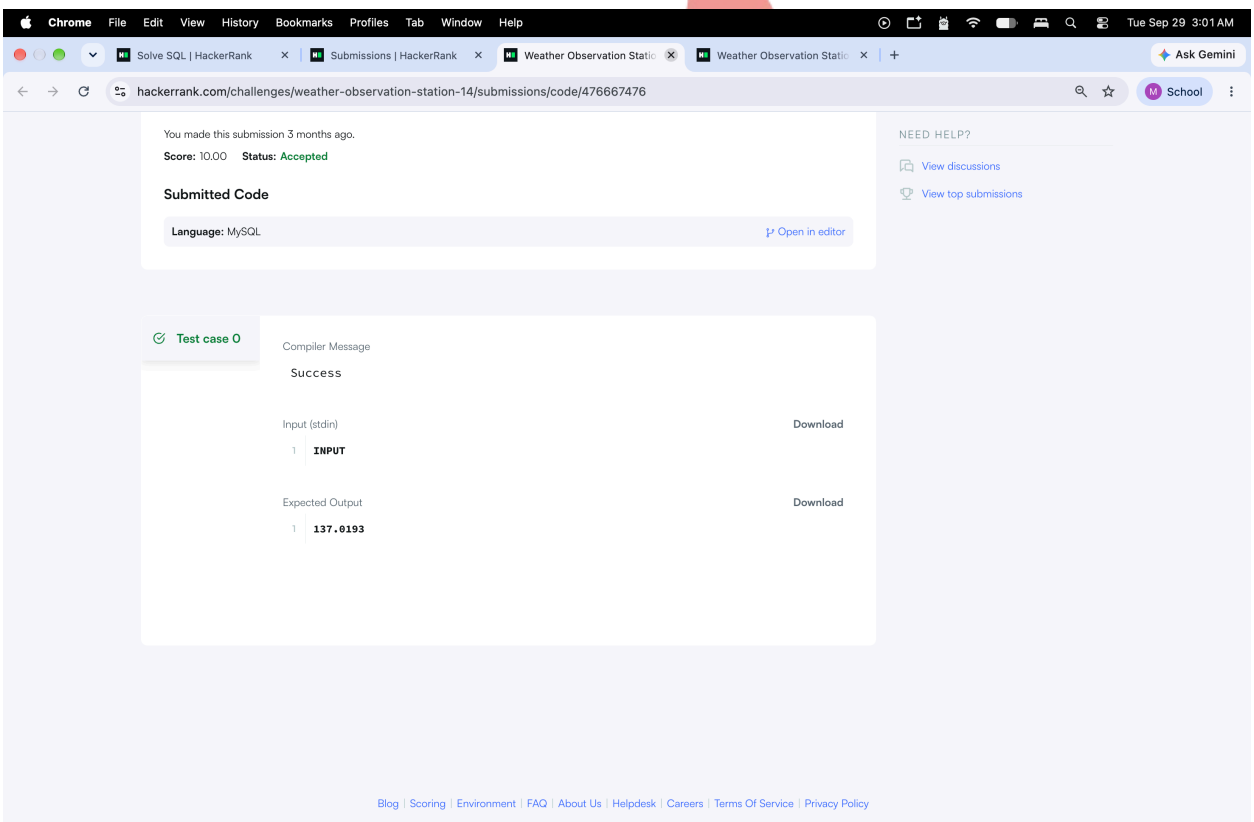
Challenge: Weather Observation Station 14

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT:



35. Weather Observation Station 15

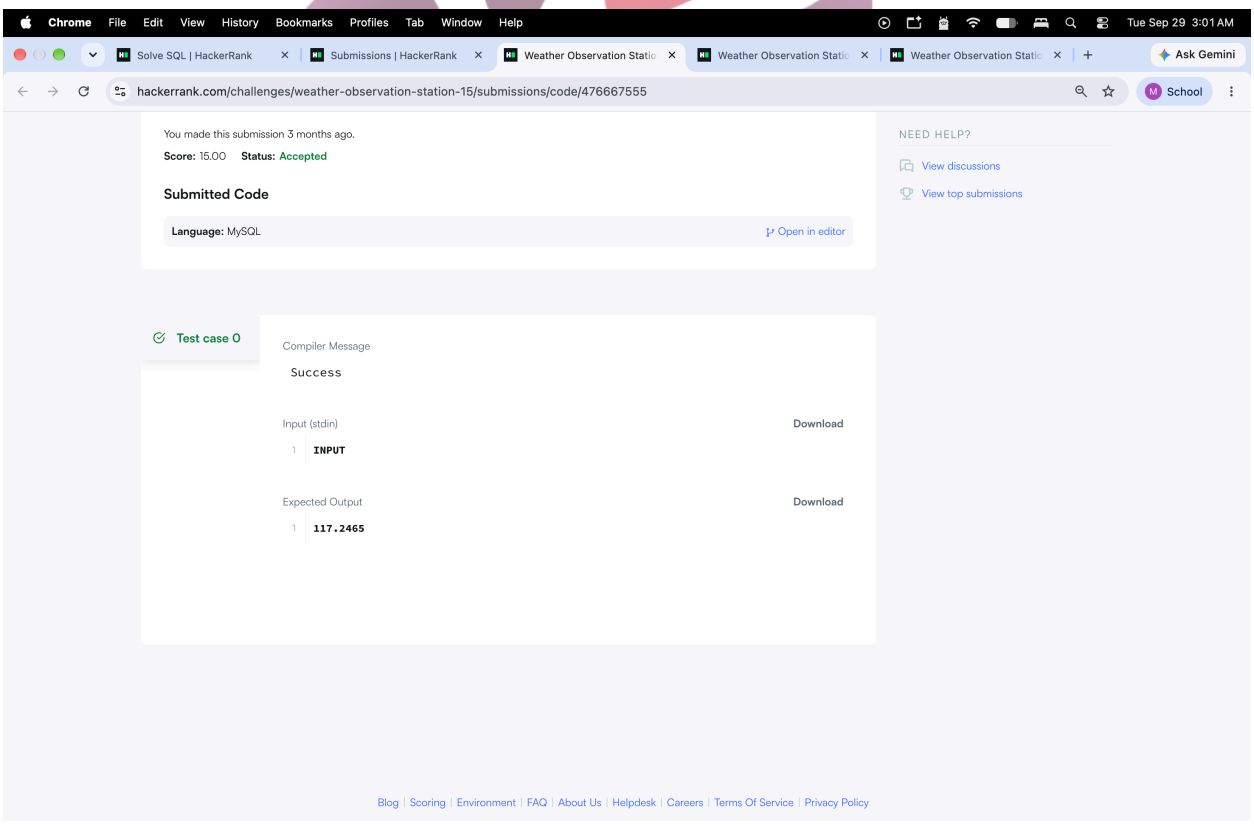
Challenge: Weather Observation Station 15

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
);
```

OUTPUT:



36. Weather Observation Station 16

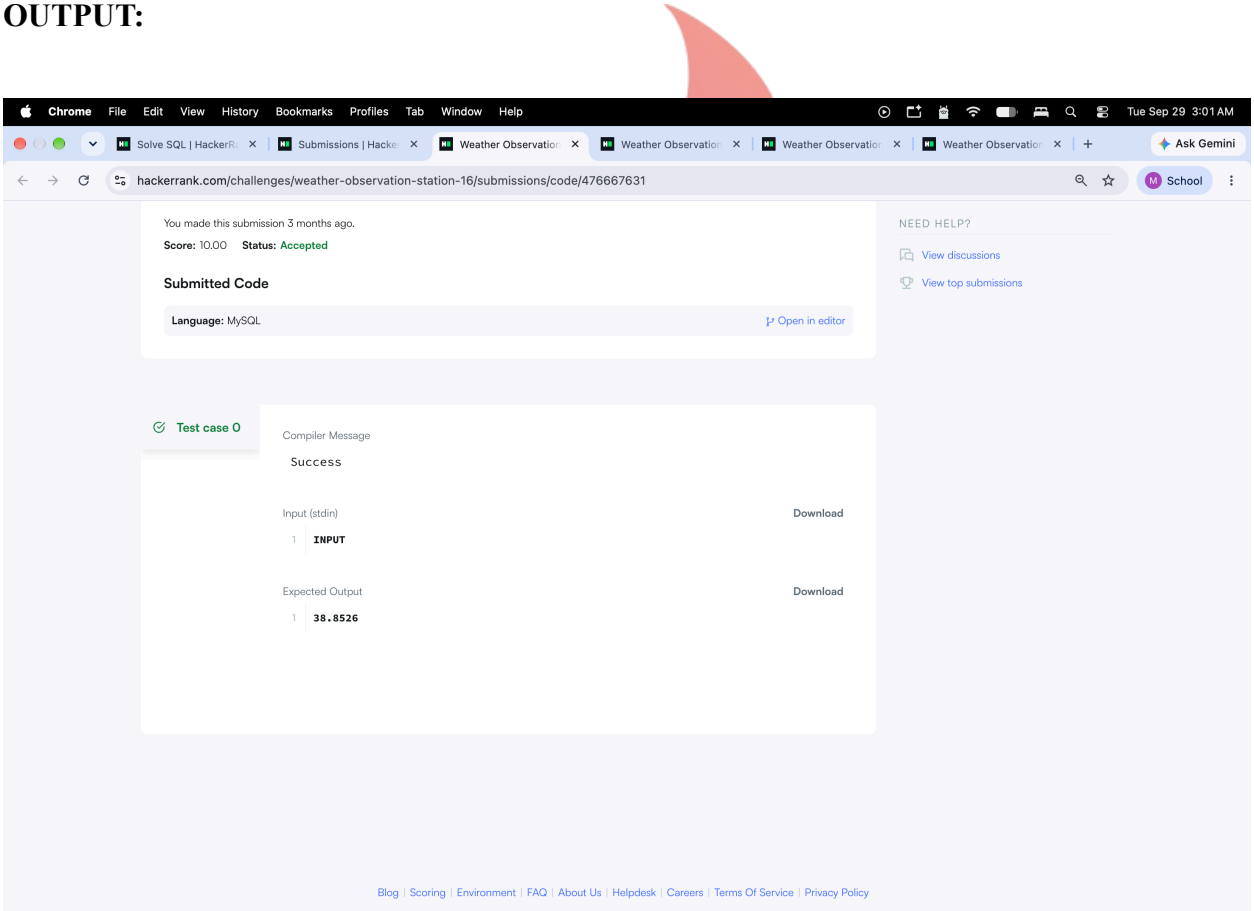
Challenge: Weather Observation Station 16

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7780;
```

OUTPUT:



37. Weather Observation Station 17

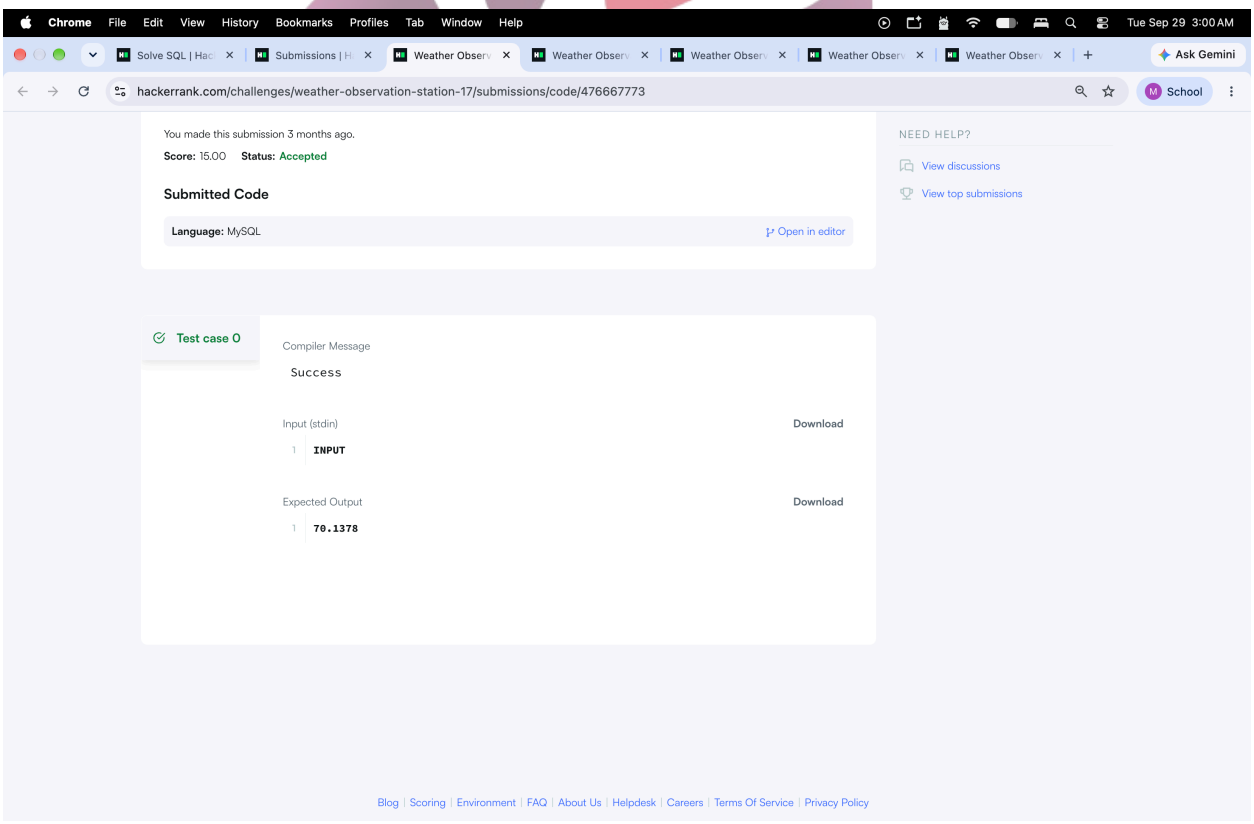
Challenge: Weather Observation Station 17

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT:



38. Weather Observation Station 18

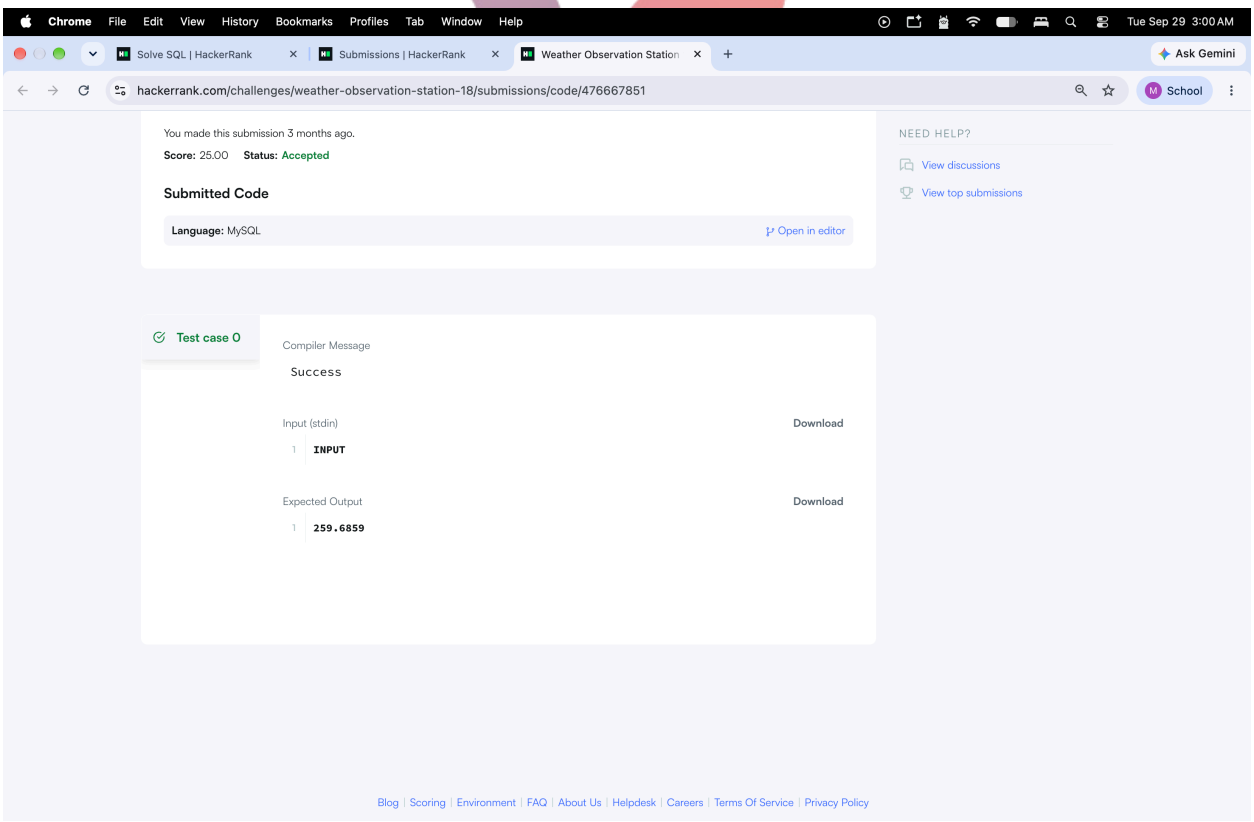
Challenge: Weather Observation Station 18

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(
   (MAX(LAT_N) - MIN(LAT_N)) + (MAX(LONG_W) - MIN(LONG_W)),
   4
)
FROM STATION;
```

OUTPUT:



39. Weather Observation Station 19

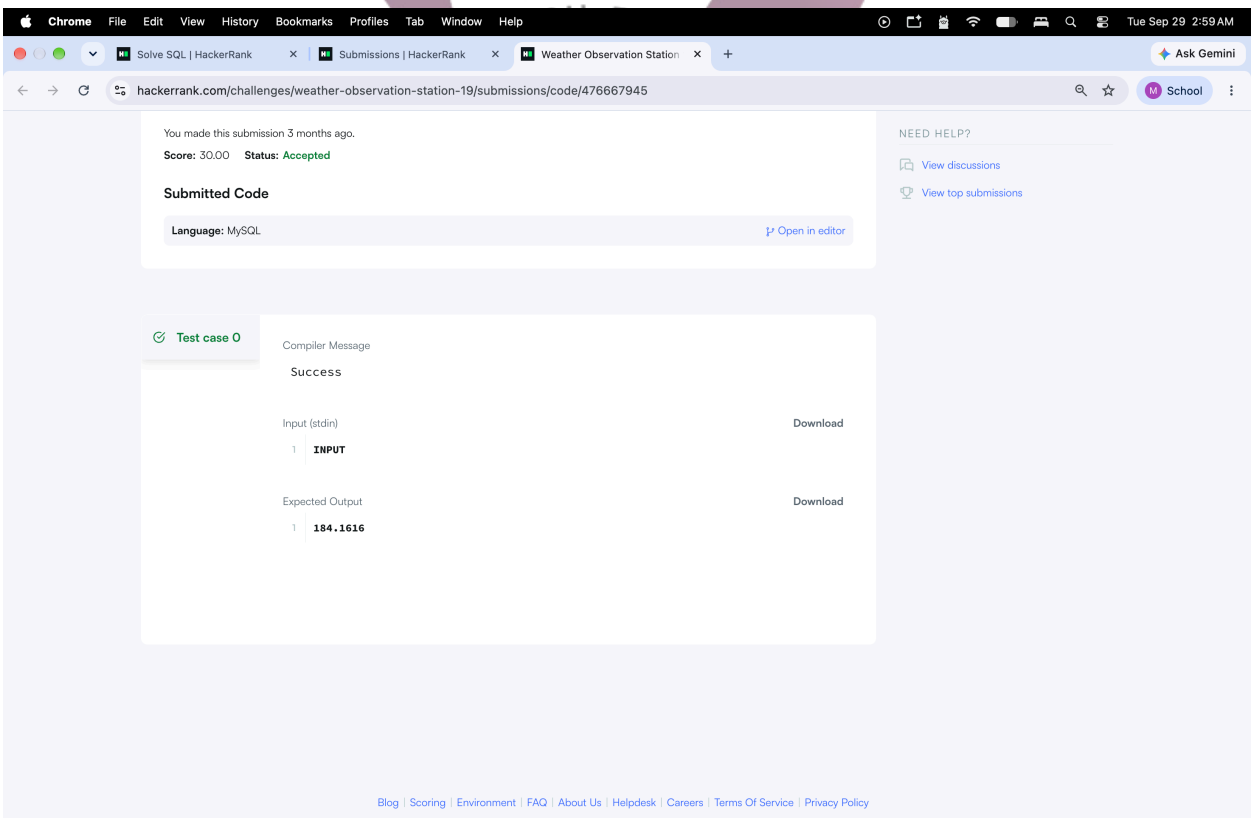
Challenge: Weather Observation Station 19

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
)
FROM STATION;
```

OUTPUT:



40. Weather Observation Station 20

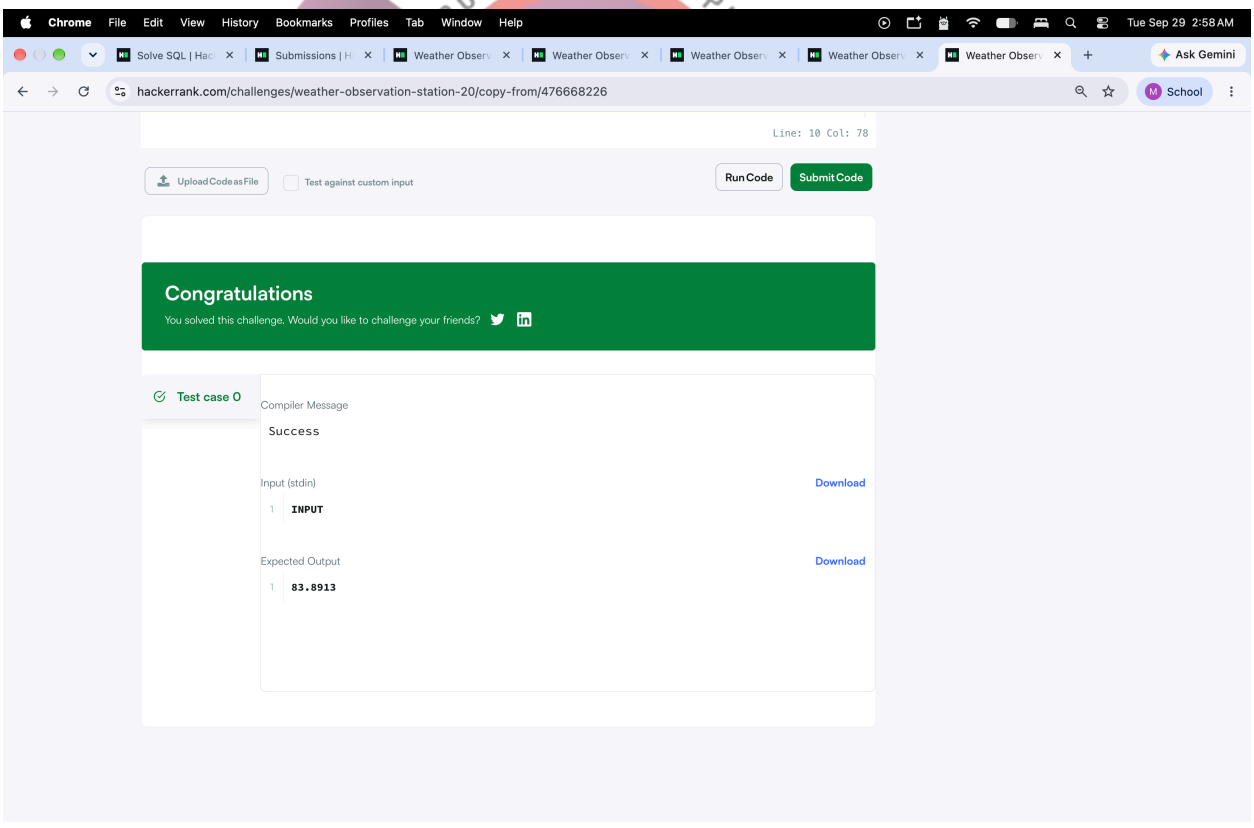
Challenge: Weather Observation Station 20

Language: MYSQL **Score:** 1.0

INPUT:

```
WITH RankedStation AS (
  SELECT
    LAT_N,
    ROW_NUMBER() OVER (ORDER BY LAT_N) AS row_num,
    COUNT(*) OVER () AS total_count
  FROM STATION
)
SELECT ROUND(AVG(LAT_N), 4)
FROM RankedStation
WHERE row_num IN (FLOOR((total_count + 1) / 2), CEIL((total_count + 1) / 2));
```

OUTPUT:



41. Revising Aggregations - The Count Function

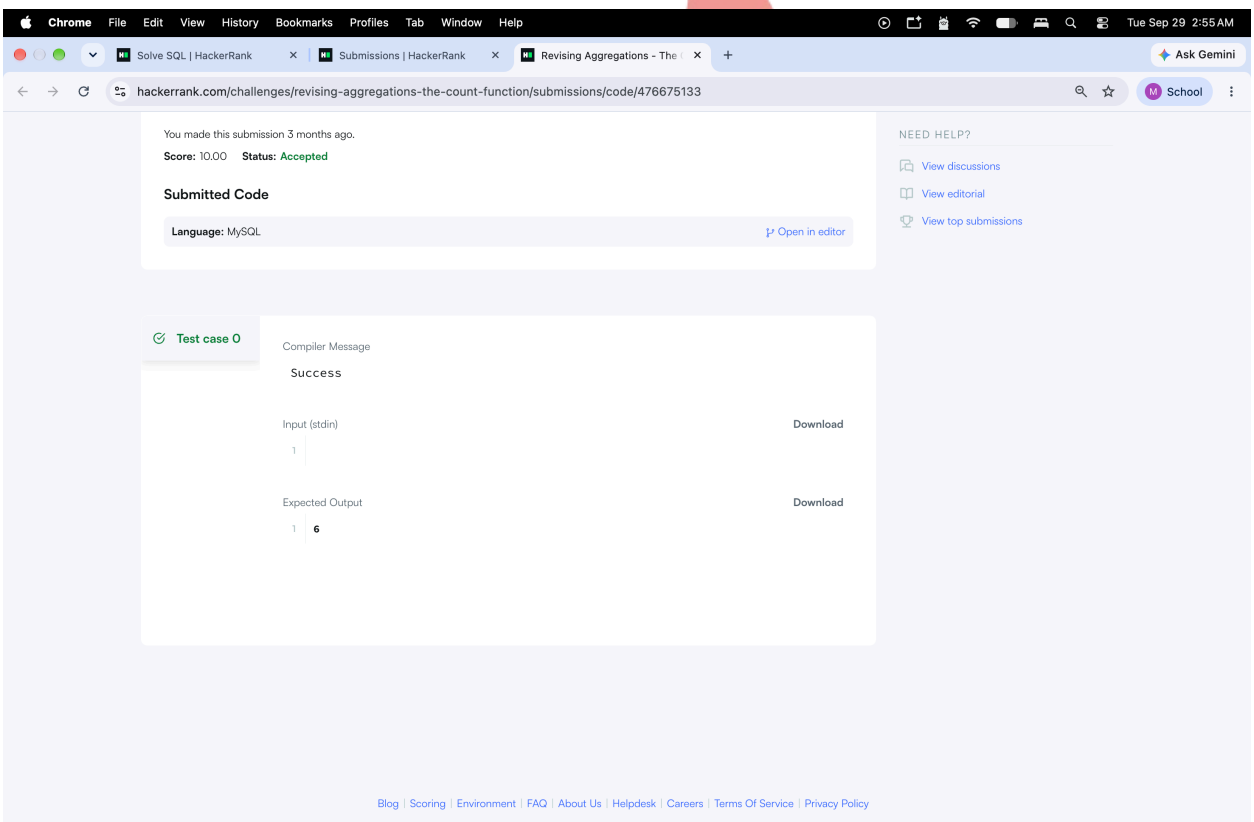
Challenge: Revising Aggregations - The Count Function

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT COUNT(*)
FROM CITY
WHERE Population > 100000;
```

OUTPUT:



42. Revising Aggregations - Averages

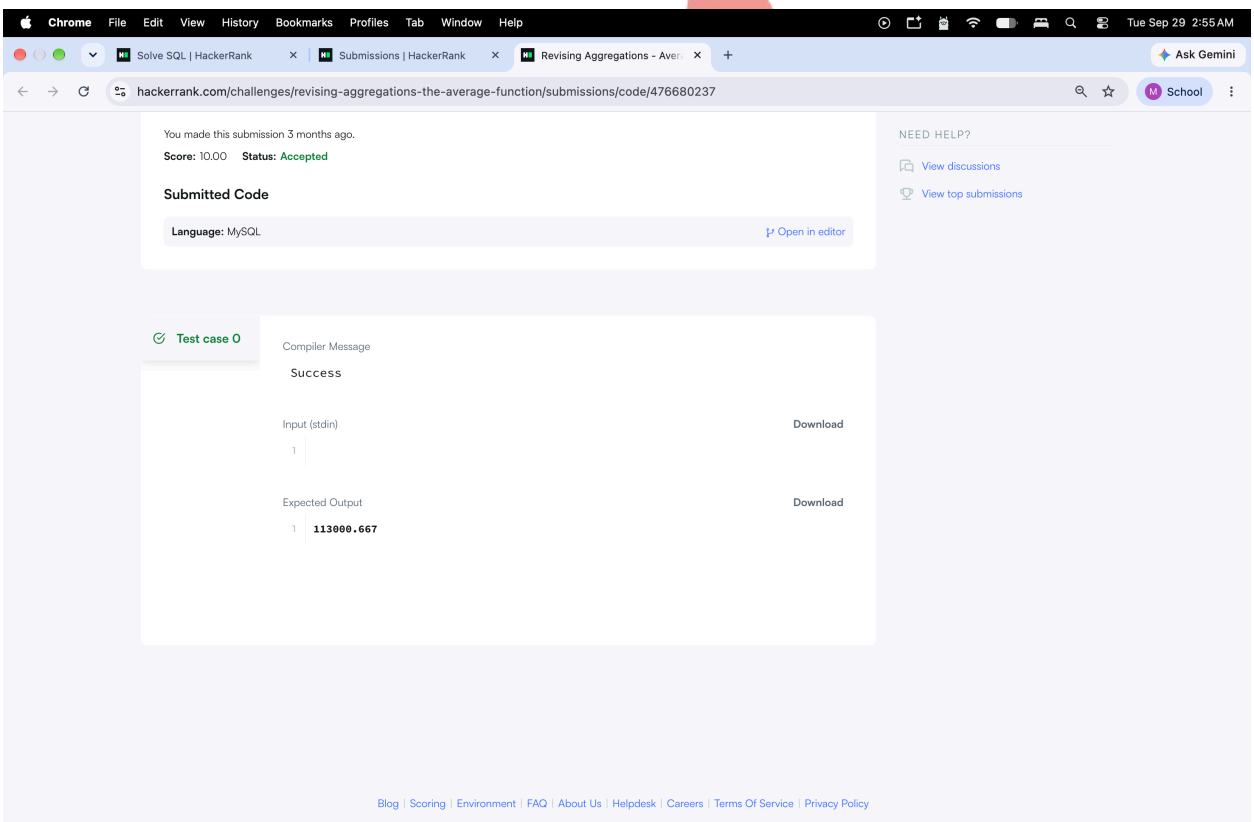
Challenge: Revising Aggregations - Averages

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT:



43. Average Population

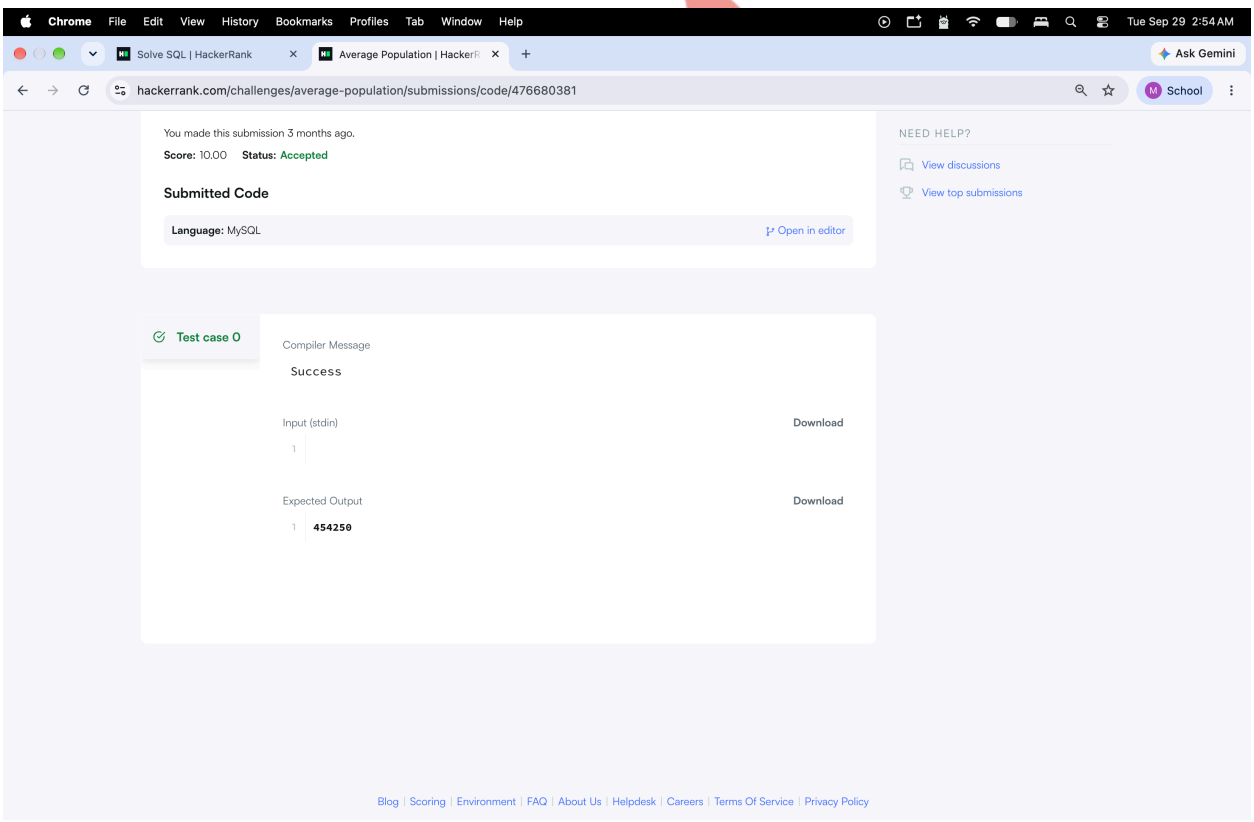
Challenge: Average Population

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT:



44. Japan Population

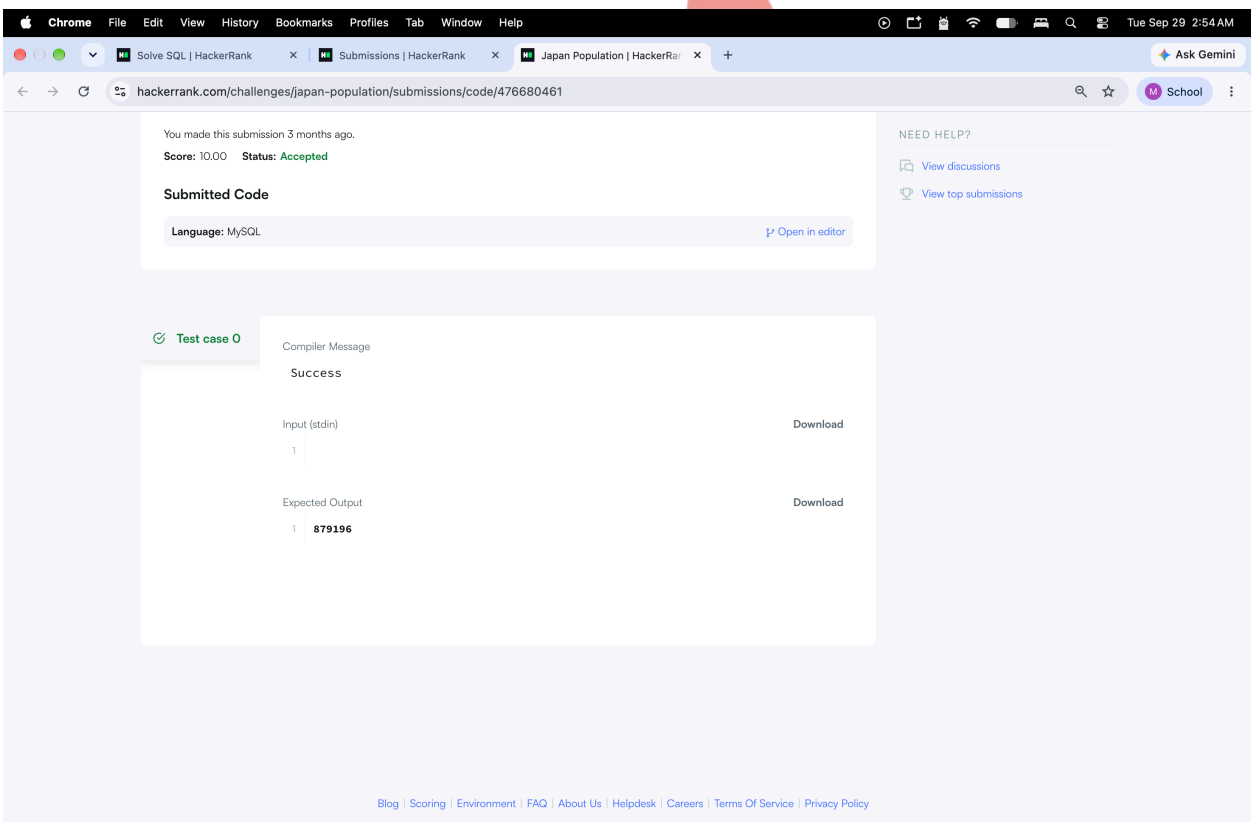
Challenge: Japan Population

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT:



45. Population Density Difference

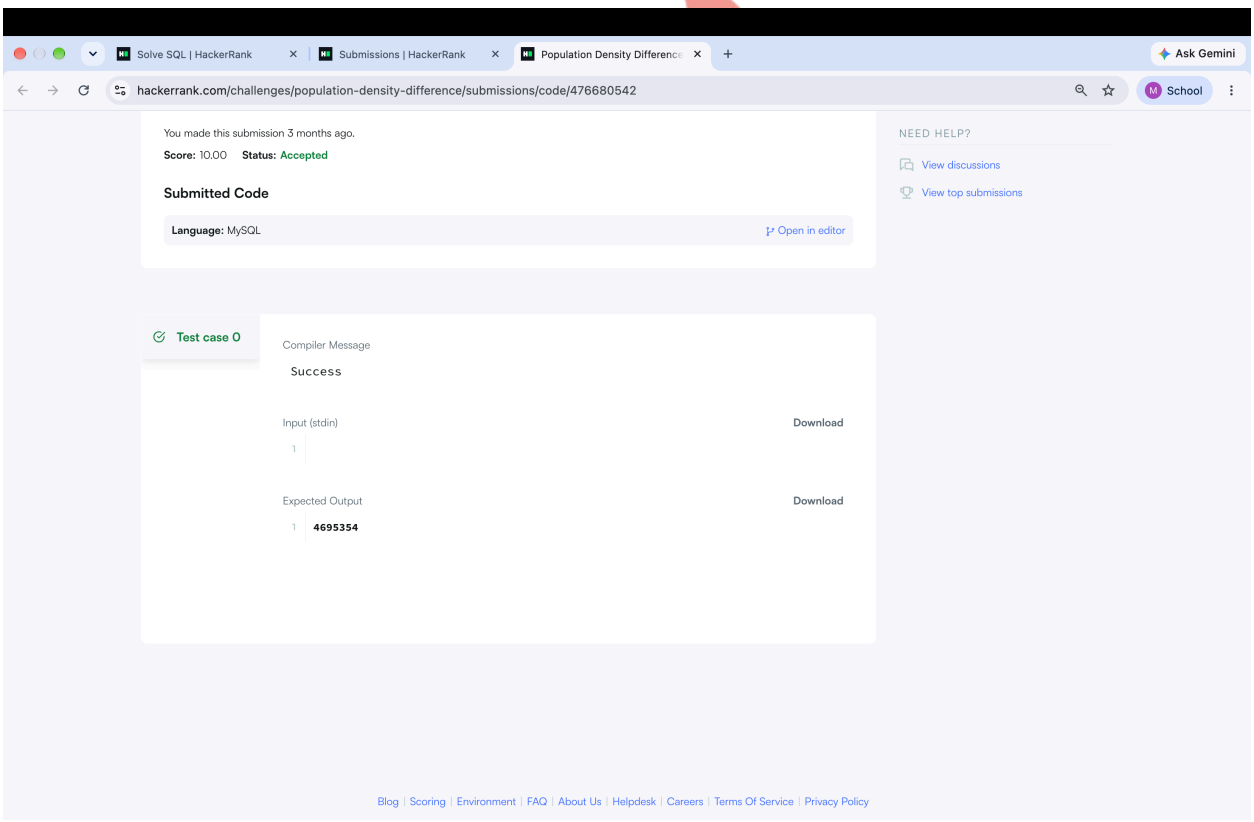
Challenge: Population Density Difference

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT MAX(POPULATION) - MIN(POPULATION)
FROM CITY;
```

OUTPUT:



46. The Blunder

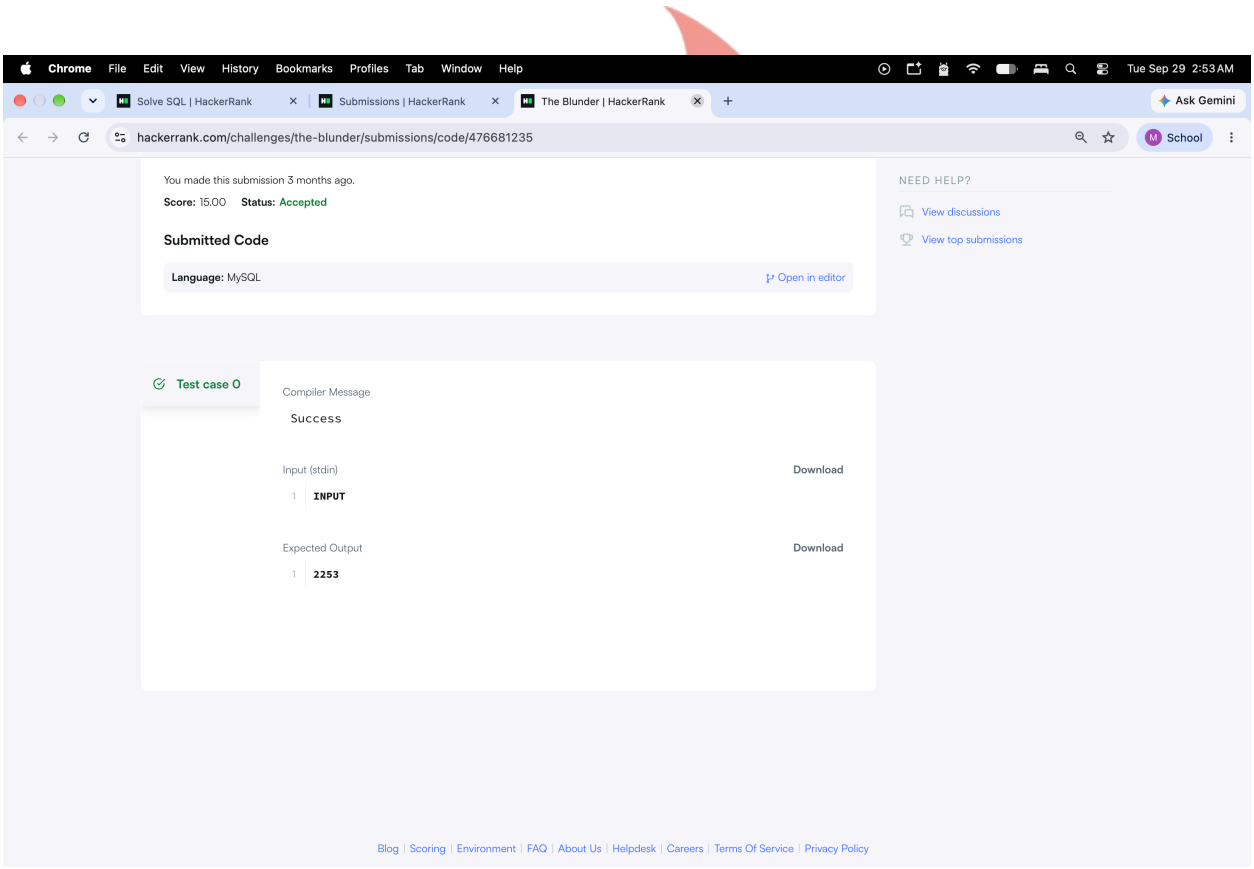
Challenge: The Blunder

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', '')))
FROM EMPLOYEES;
```

OUTPUT:



47. Top Earners

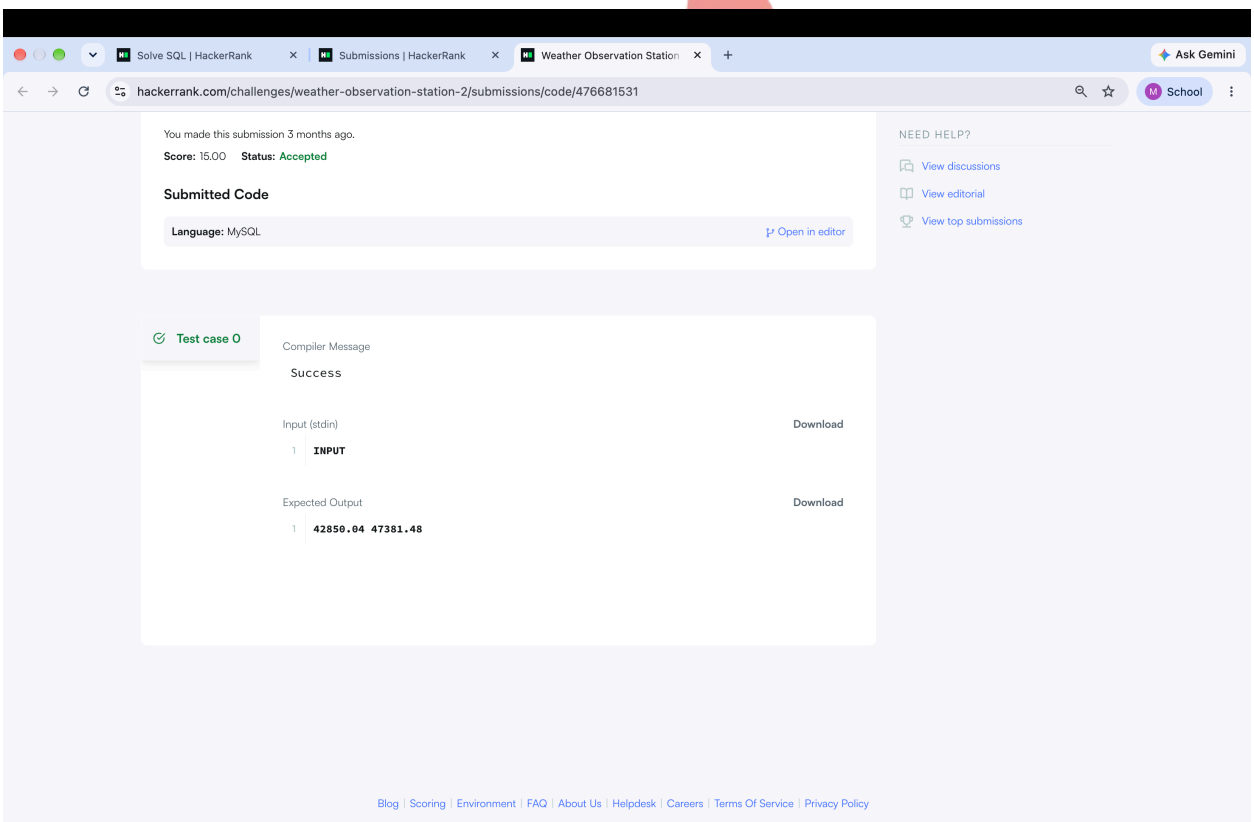
Challenge: Top Earners

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT MAX(months * salary), COUNT(*)
FROM Employee
WHERE months * salary = (SELECT MAX(months * salary) FROM Employee);
```

OUTPUT:



48. Weather Observation Station 2

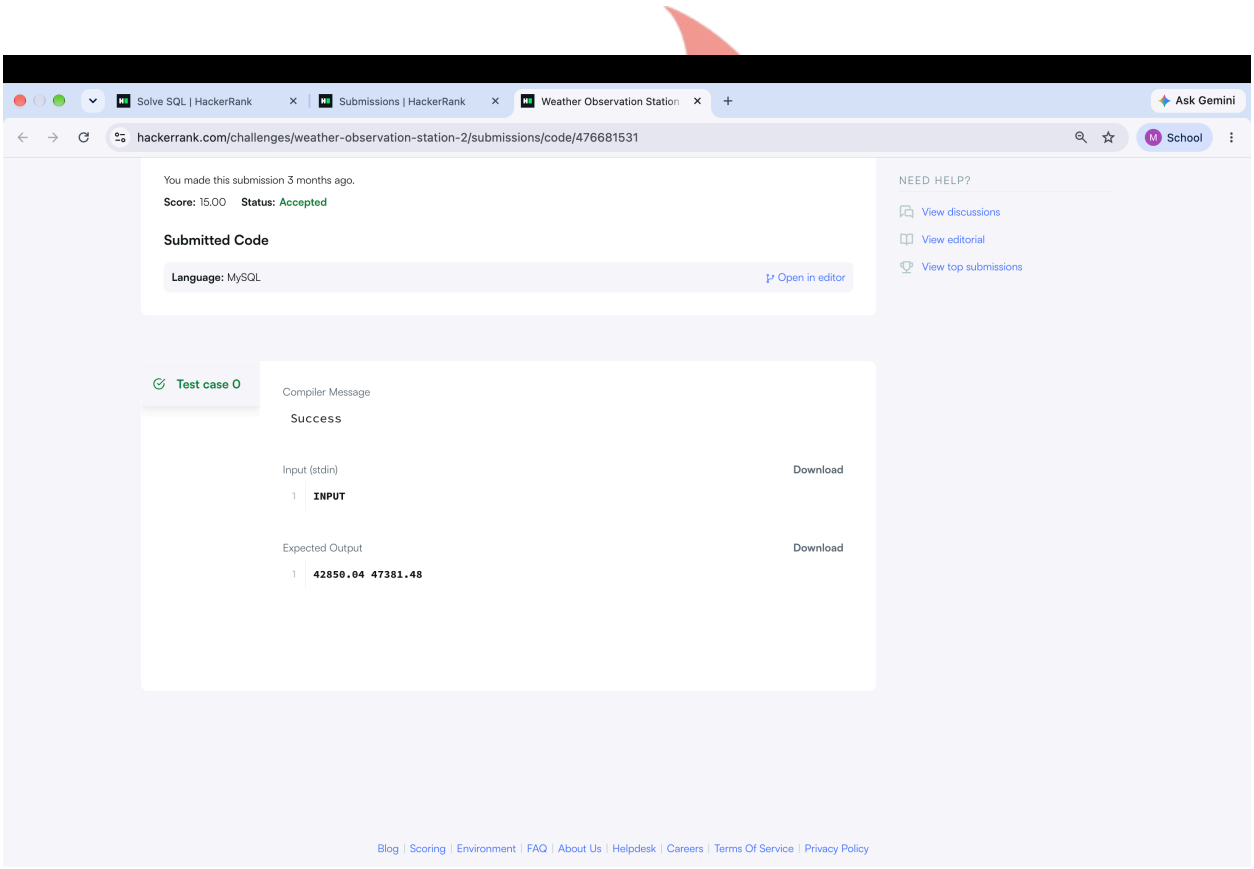
Challenge: Weather Observation Station 2

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT:



49. Average Population of Each Continent

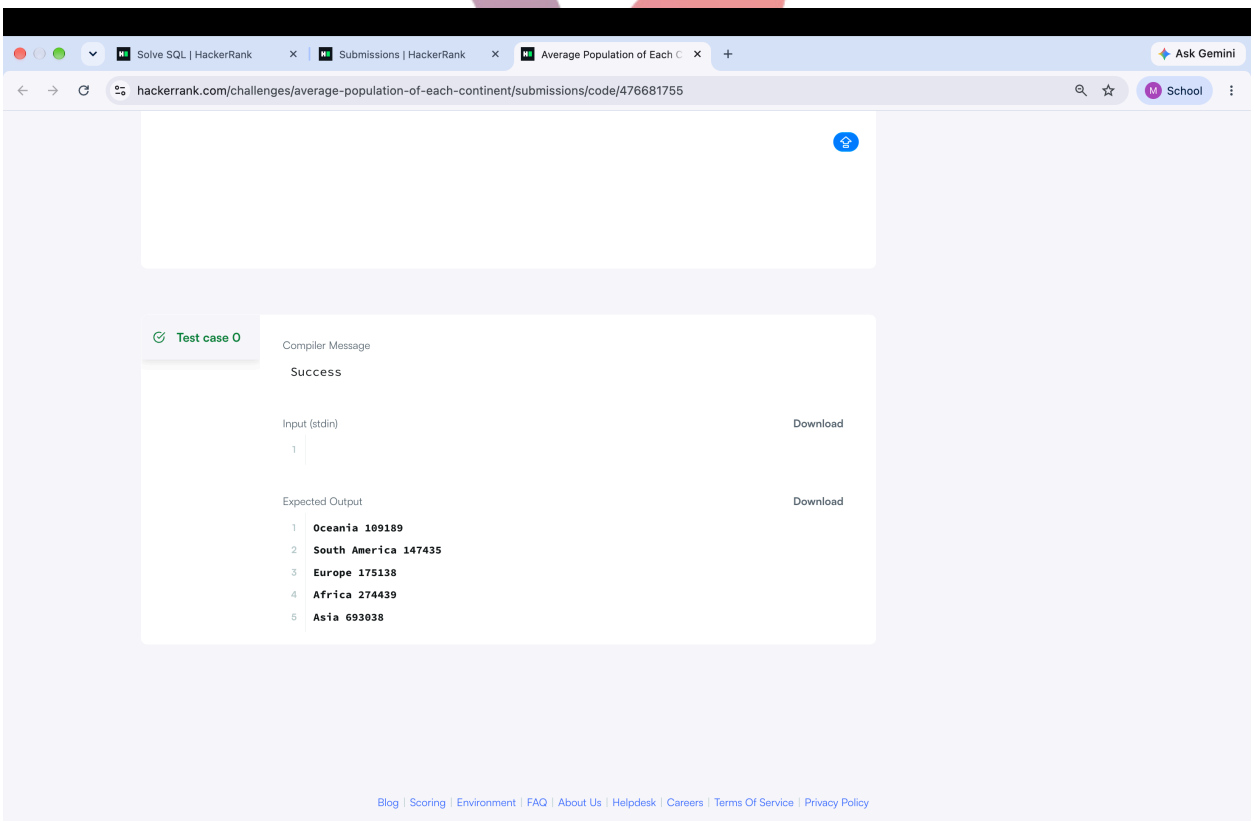
Challenge: Average Population of Each Continent

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT COUNTRY.CONTINENT, FLOOR(AVG(CITY.POPULATION))
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
GROUP BY COUNTRY.CONTINENT;
```

OUTPUT:



50. The Report

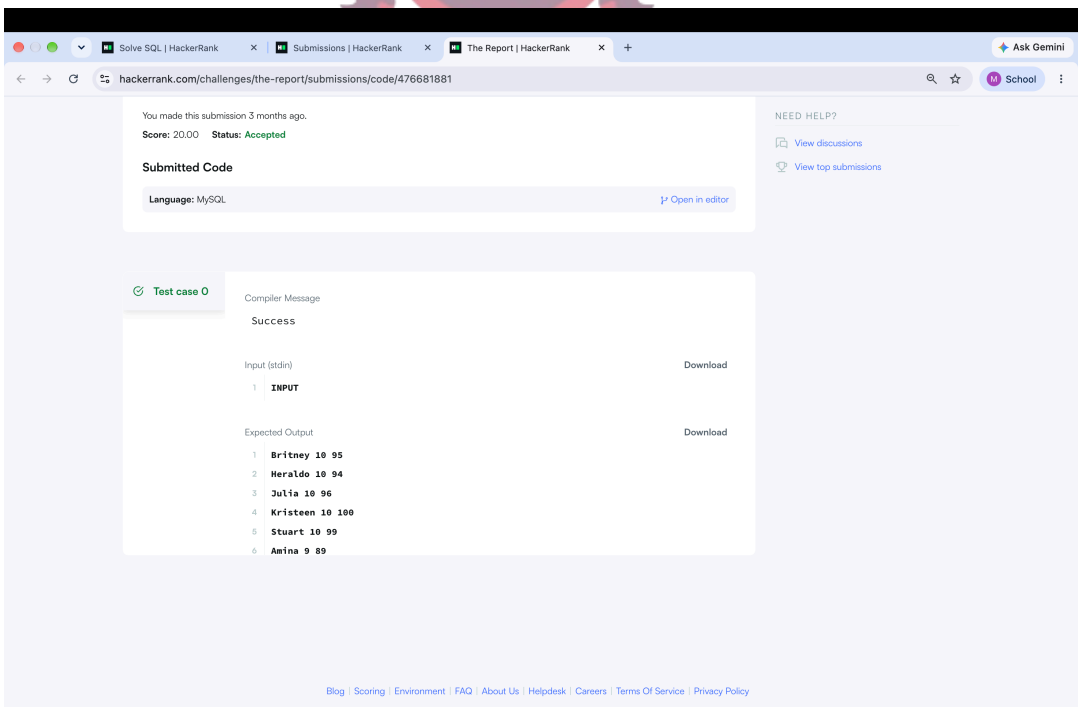
Challenge: The Report

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT
  CASE
    WHEN G.GRADE < 8 THEN 'NULL'
    ELSE S.NAME
  END AS NAME,
  G.GRADE,
  S.MARKS
FROM STUDENTS S
JOIN GRADES G
ON S.MARKS BETWEEN G.MIN_MARK AND G.MAX_MARK
ORDER BY
  G.GRADE DESC,
  CASE WHEN G.GRADE >= 8 THEN S.NAME END ASC,
  CASE WHEN G.GRADE < 8 THEN S.MARKS END ASC;
```

OUTPUT:



51. Top Competitors

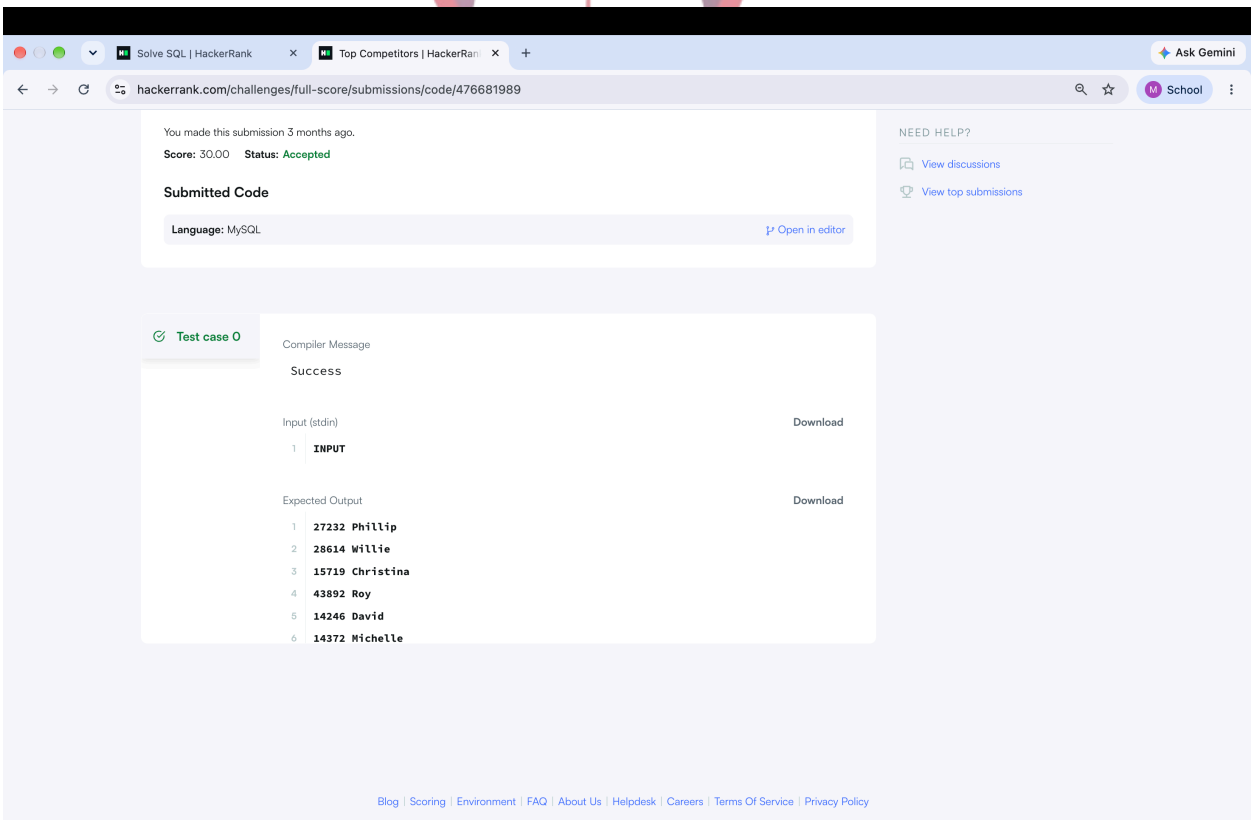
Challenge: Top Competitors

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT H.hacker_id, H.name
FROM Hackers H
JOIN Submissions S
ON H.hacker_id = S.hacker_id
JOIN Challenges C
ON S.challenge_id = C.challenge_id
JOIN Difficulty D
ON C.difficulty_level = D.difficulty_level
WHERE S.score = D.score
GROUP BY H.hacker_id, H.name
HAVING COUNT(*) > 1
ORDER BY COUNT(*) DESC, H.hacker_id ASC;
```

OUTPUT:



52. Ollivander's Inventory

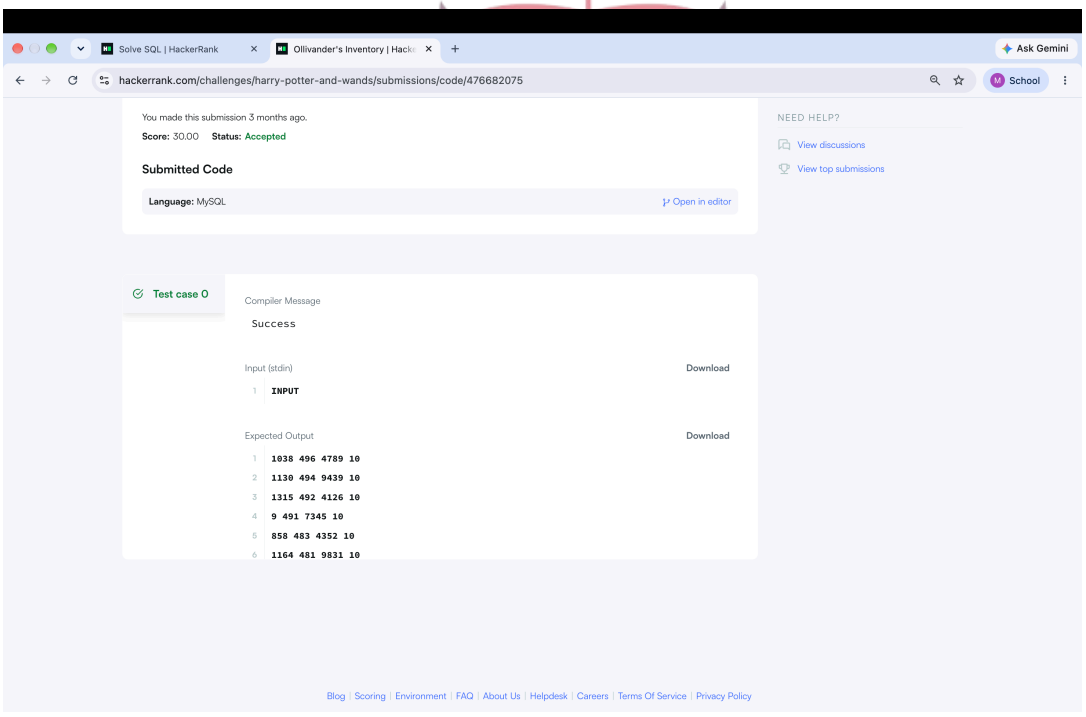
Challenge: Ollivander's Inventory

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT W.id, P.age, W.coins_needed, W.power
FROM Wands W
JOIN Wands_Property P
ON W.code = P.code
WHERE P.is_evil = 0
AND W.coins_needed = (
   SELECT MIN(W2.coins_needed)
   FROM Wands W2
   JOIN Wands_Property P2
   ON W2.code = P2.code
   WHERE P2.is_evil = 0
   AND W2.power = W.power
   AND P2.age = P.age
)
ORDER BY W.power DESC, P.age DESC;
```

OUTPUT:



53. Challenges

Challenge: Challenges

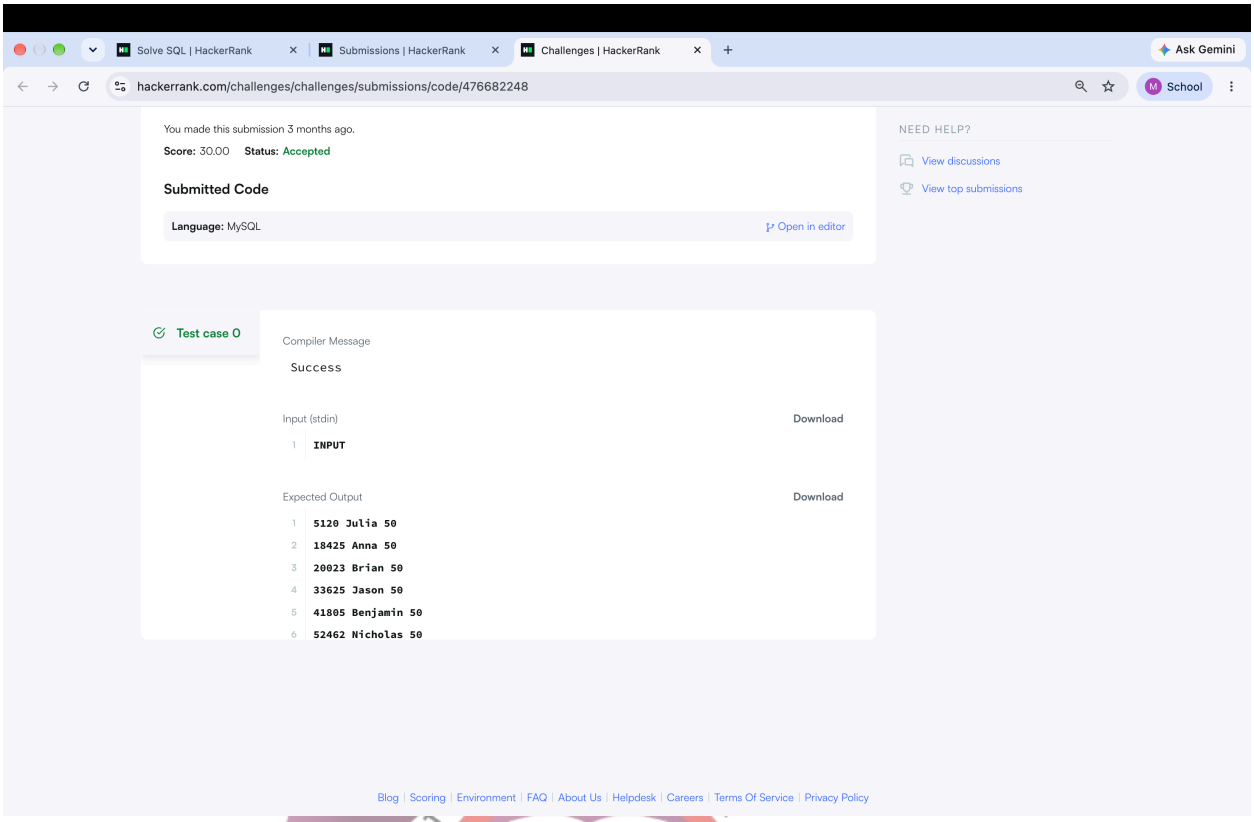
Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT H.hacker_id, H.name, COUNT(C.challenge_id) AS total_challenges
FROM Hackers H
JOIN Challenges C
ON H.hacker_id = C.hacker_id
GROUP BY H.hacker_id, H.name
HAVING
  total_challenges = (
    SELECT MAX(challenge_count)
    FROM (
      SELECT COUNT(challenge_id) AS challenge_count
      FROM Challenges
      GROUP BY hacker_id
    ) AS counts
  )
  OR total_challenges IN (
    SELECT challenge_count
    FROM (
      SELECT COUNT(challenge_id) AS challenge_count
      FROM Challenges
      GROUP BY hacker_id
    ) AS counts
    GROUP BY challenge_count
    HAVING COUNT(*) = 1
  )
ORDER BY total_challenges DESC, H.hacker_id ASC;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

OUTPUT:



AUTONOMOUS

54. Contest Leaderboard

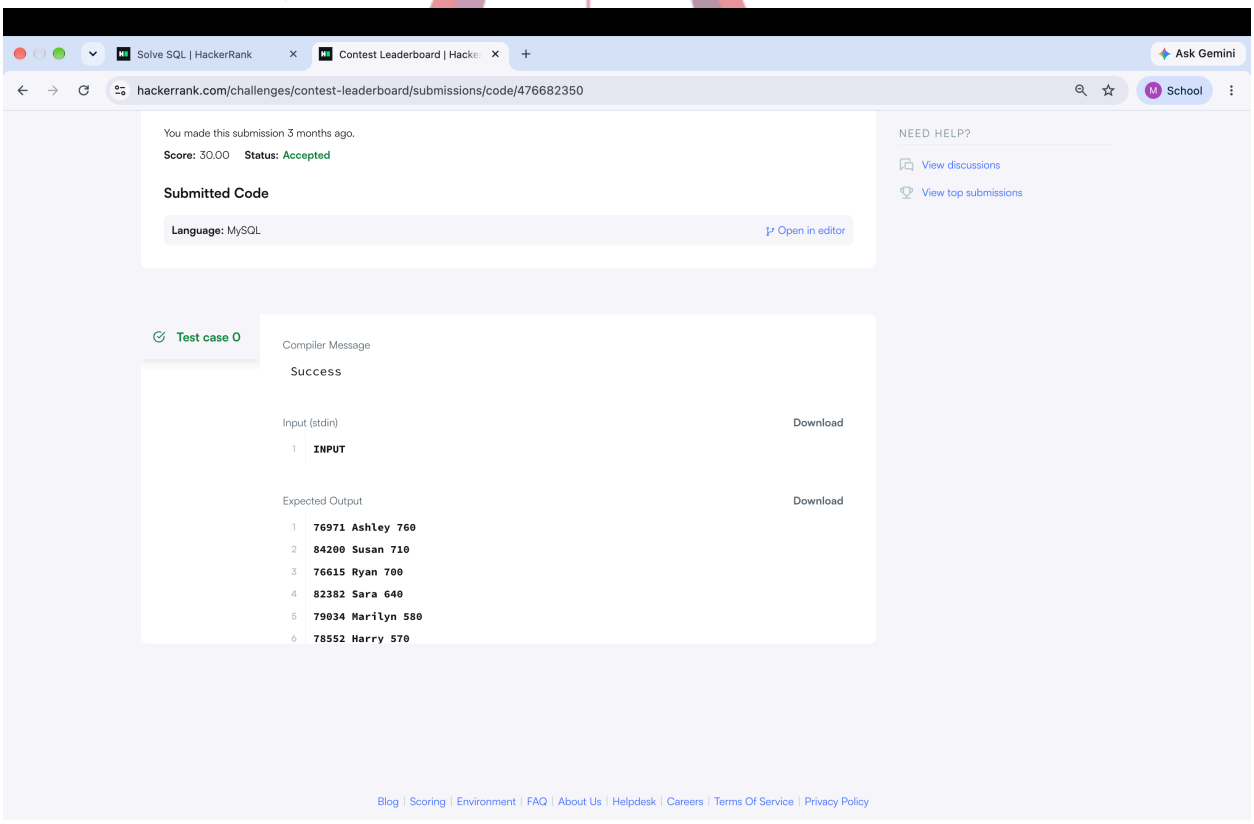
Challenge: Contest Leaderboard

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT H.hacker_id, H.name, SUM(S.max_score) AS total_score
FROM Hackers H
JOIN (
   SELECT hacker_id, challenge_id, MAX(score) AS max_score
   FROM Submissions
   GROUP BY hacker_id, challenge_id
) S
ON H.hacker_id = S.hacker_id
GROUP BY H.hacker_id, H.name
HAVING total_score > 0
ORDER BY total_score DESC, H.hacker_id ASC;
```

OUTPUT:



55. African Cities

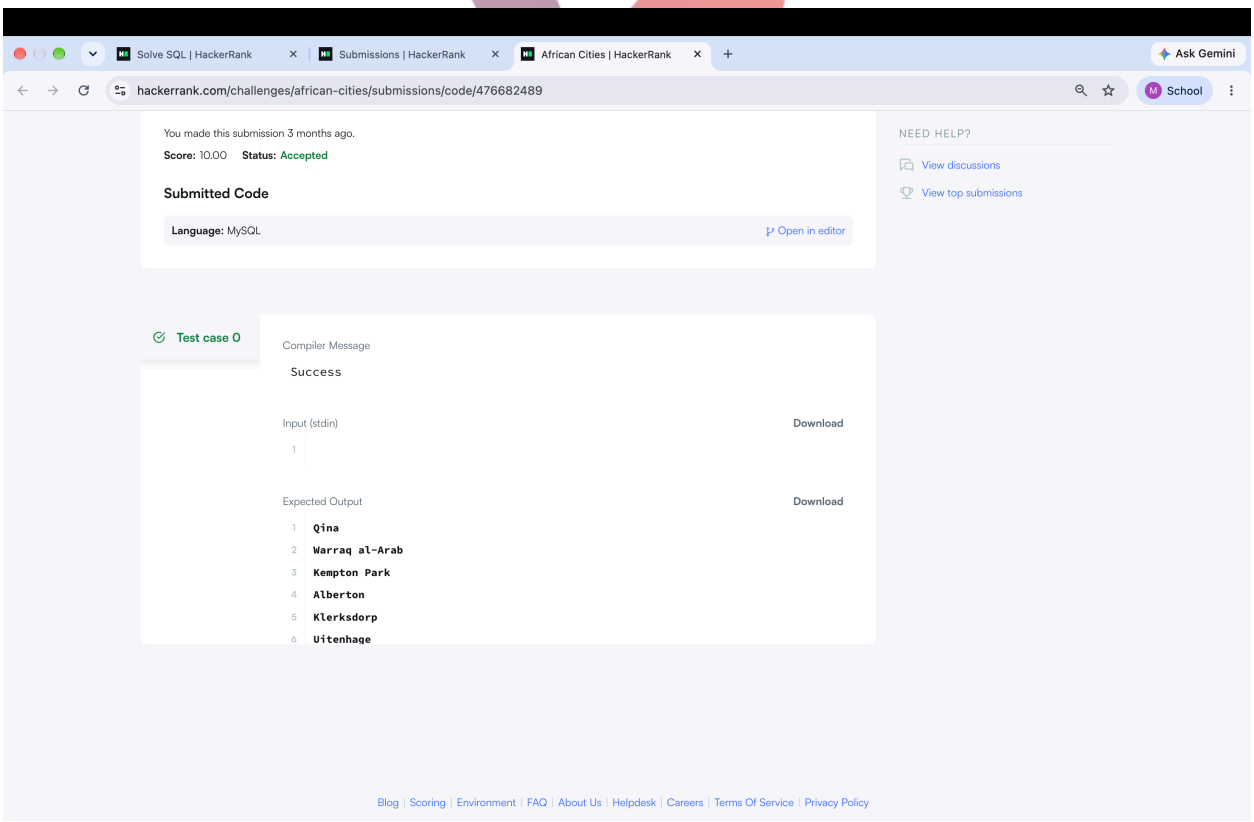
Challenge: African Cities

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Africa';
```

OUTPUT:



56. Population Census

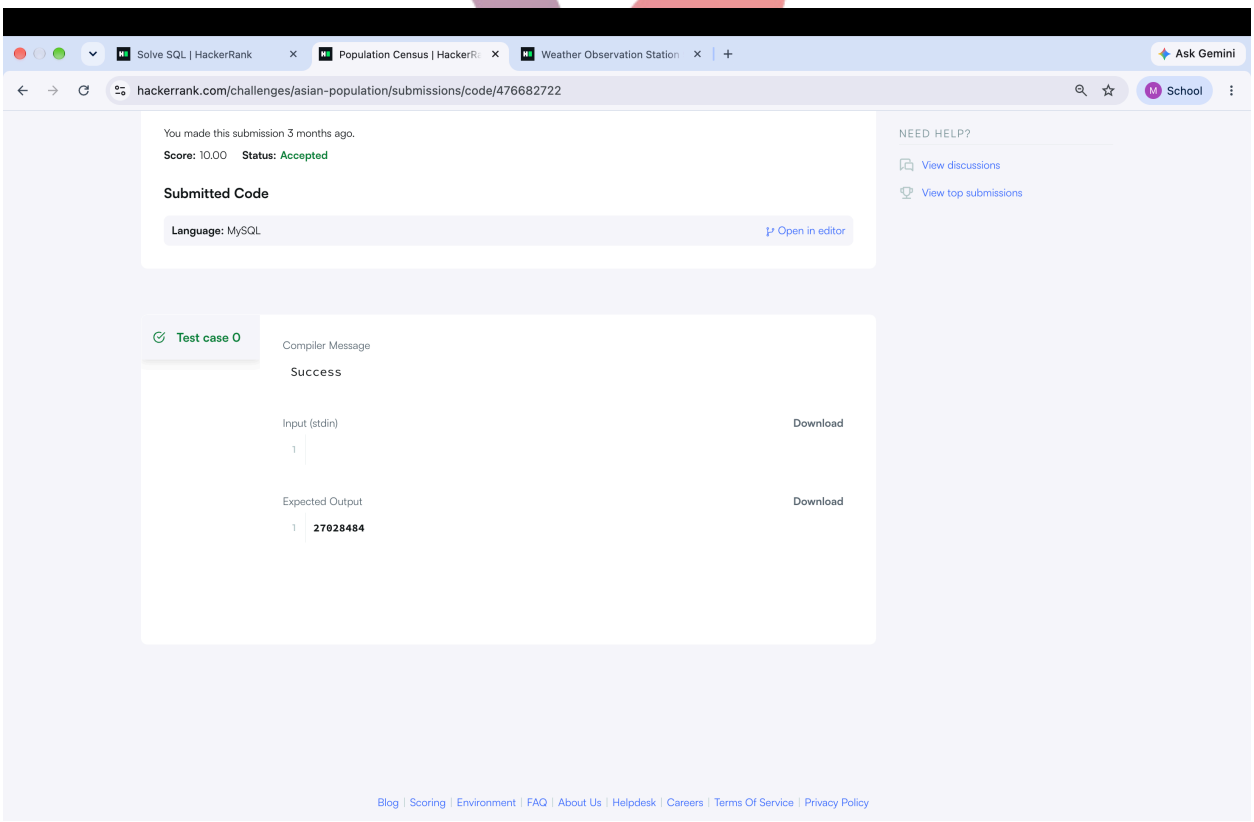
Challenge: Population Census

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT SUM(CITY.POPULATION)
FROM CITY
JOIN COUNTRY
ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

OUTPUT:



57. Draw The Triangle 1

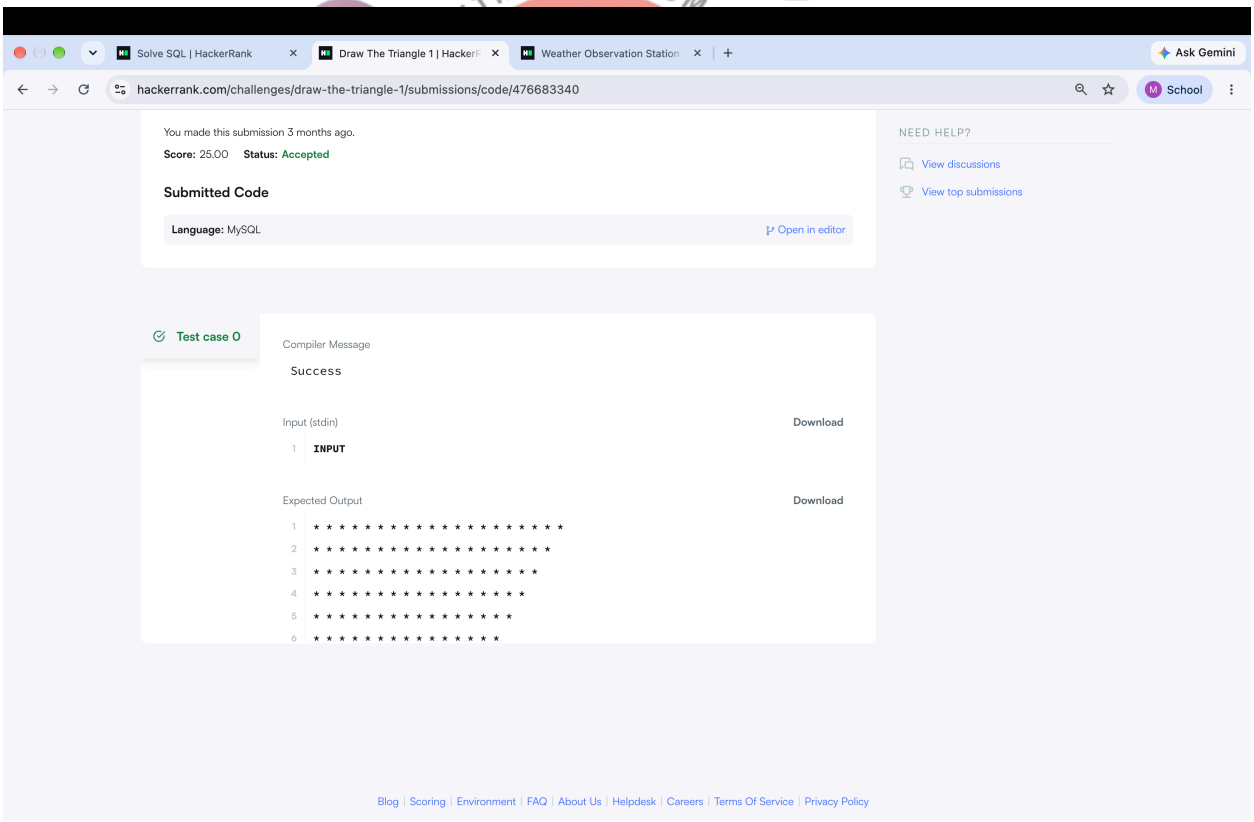
Challenge: Draw The Triangle 1

Language: MYSQL **Score:** 1.0

INPUT:

```
WITH RECURSIVE cte AS (
  SELECT 20 AS n
  UNION ALL
  SELECT n - 1
  FROM cte
  WHERE n > 1
)
SELECT REPEAT('* ', n)
FROM cte;
```

OUTPUT:



58. Revising Aggregations - The Sum Function

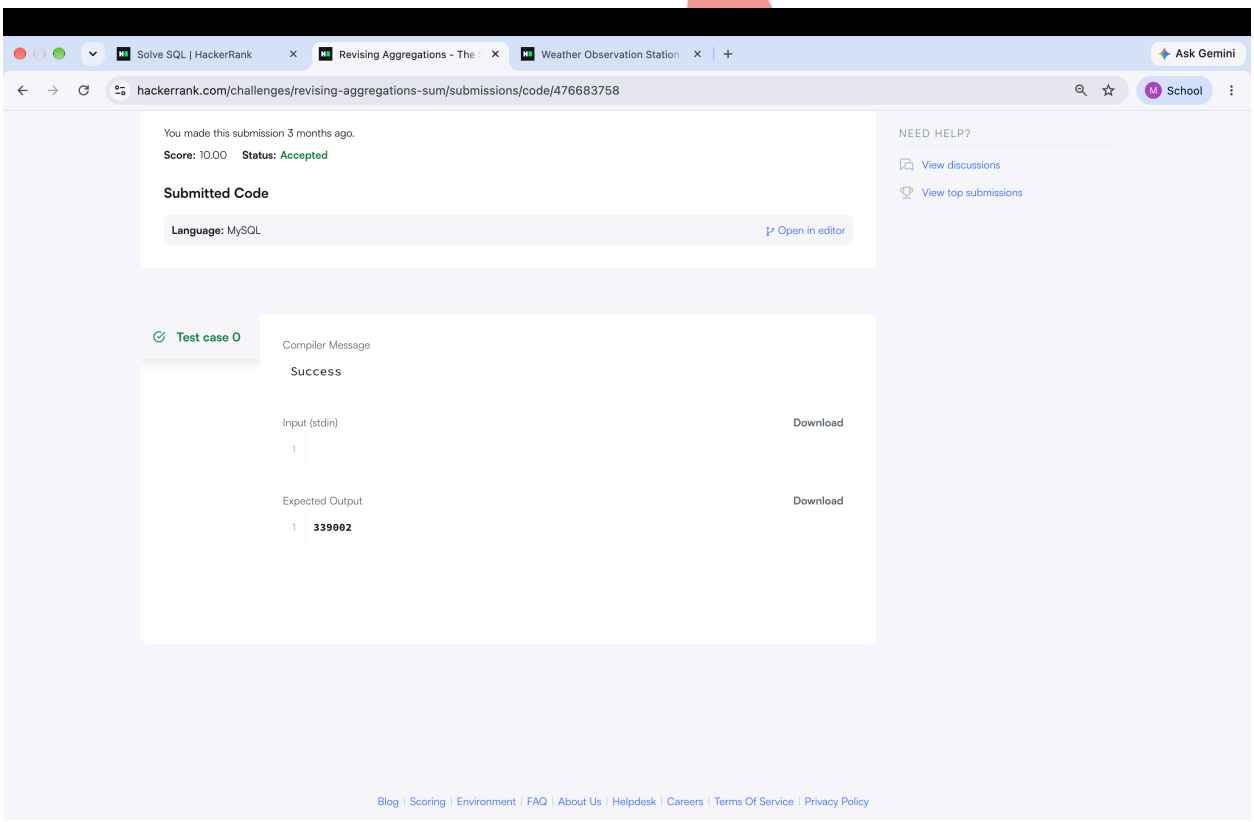
Challenge: Revising Aggregations - The Sum Function

Language: MYSQL **Score:** 1.0

INPUT:

```
SELECT SUM(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT:



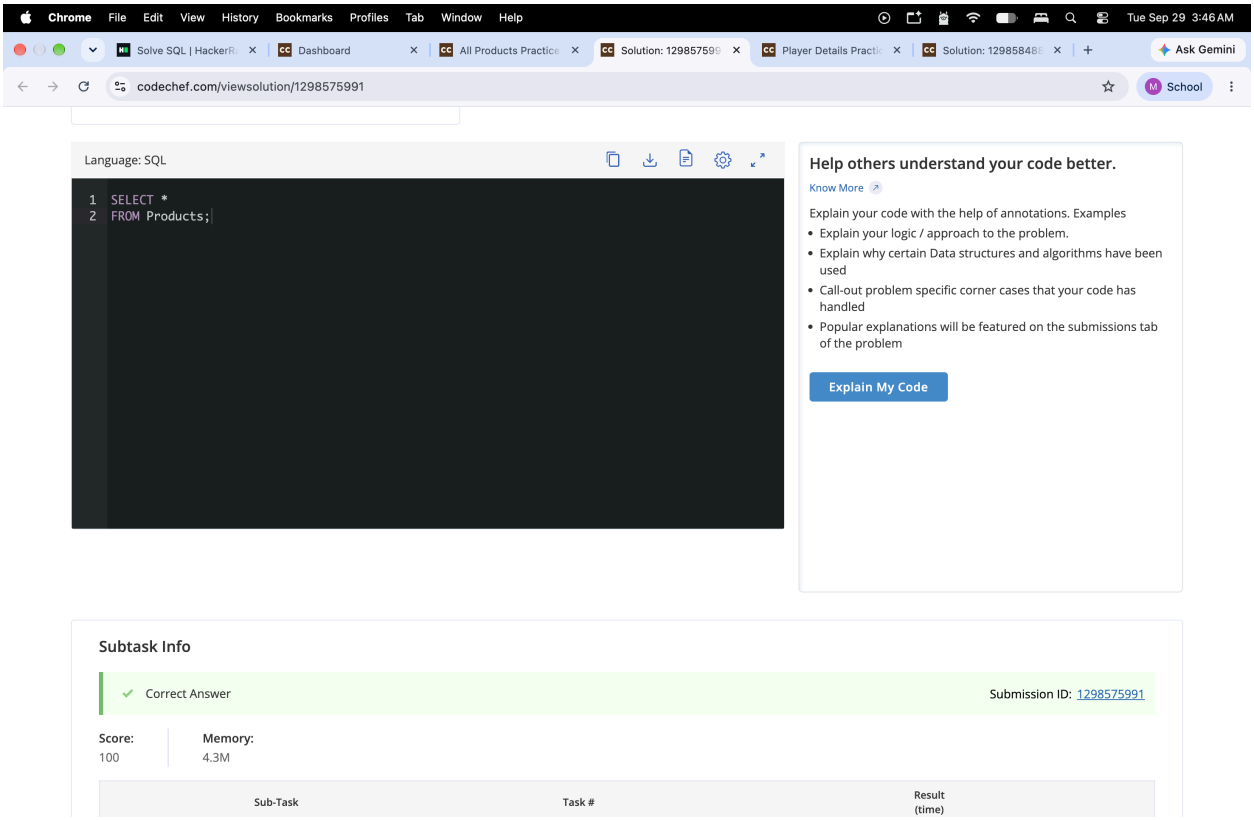
CODECHEF

1. All Products

Select all available products from the table

SELECT * FROM Products;

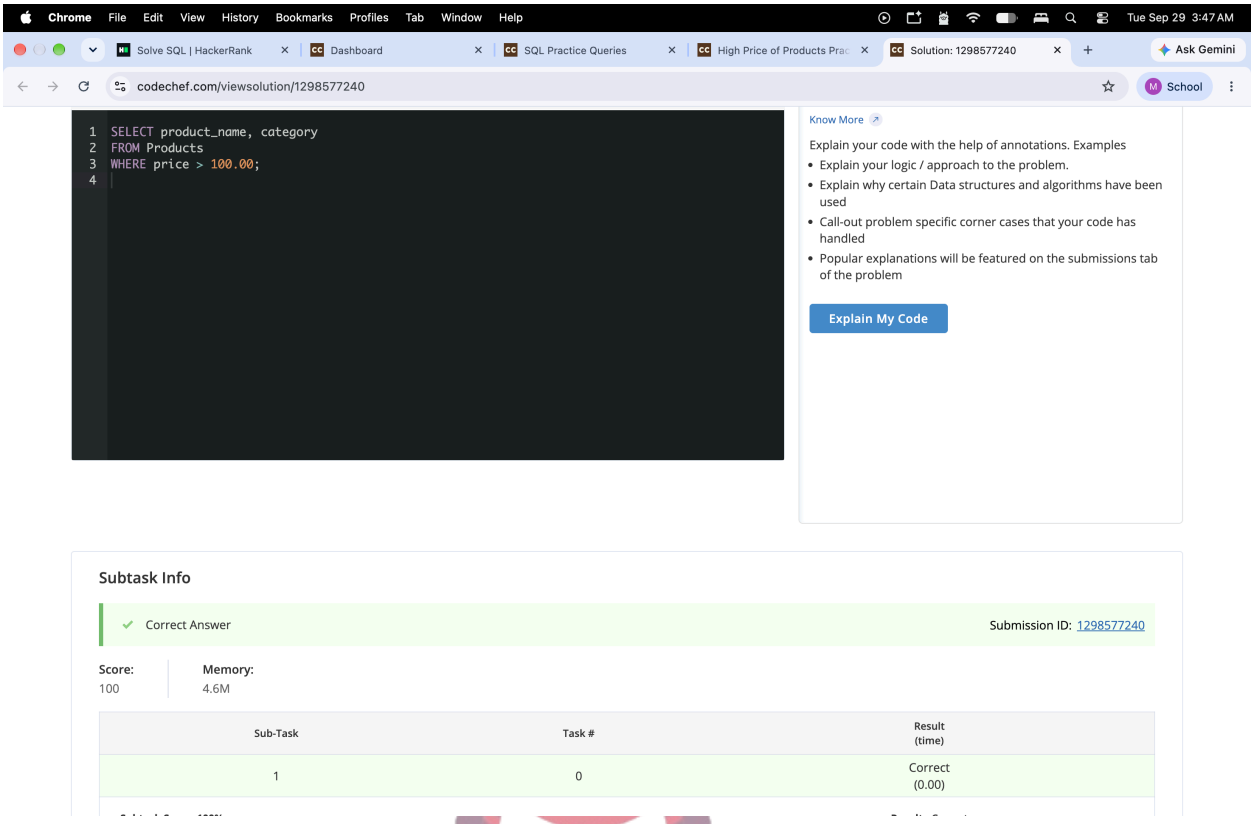
OUTPUT:



2. High Price of Products

```
SELECT product_name, category
FROM Products
WHERE price > 100.00;
```

OUTPUT:

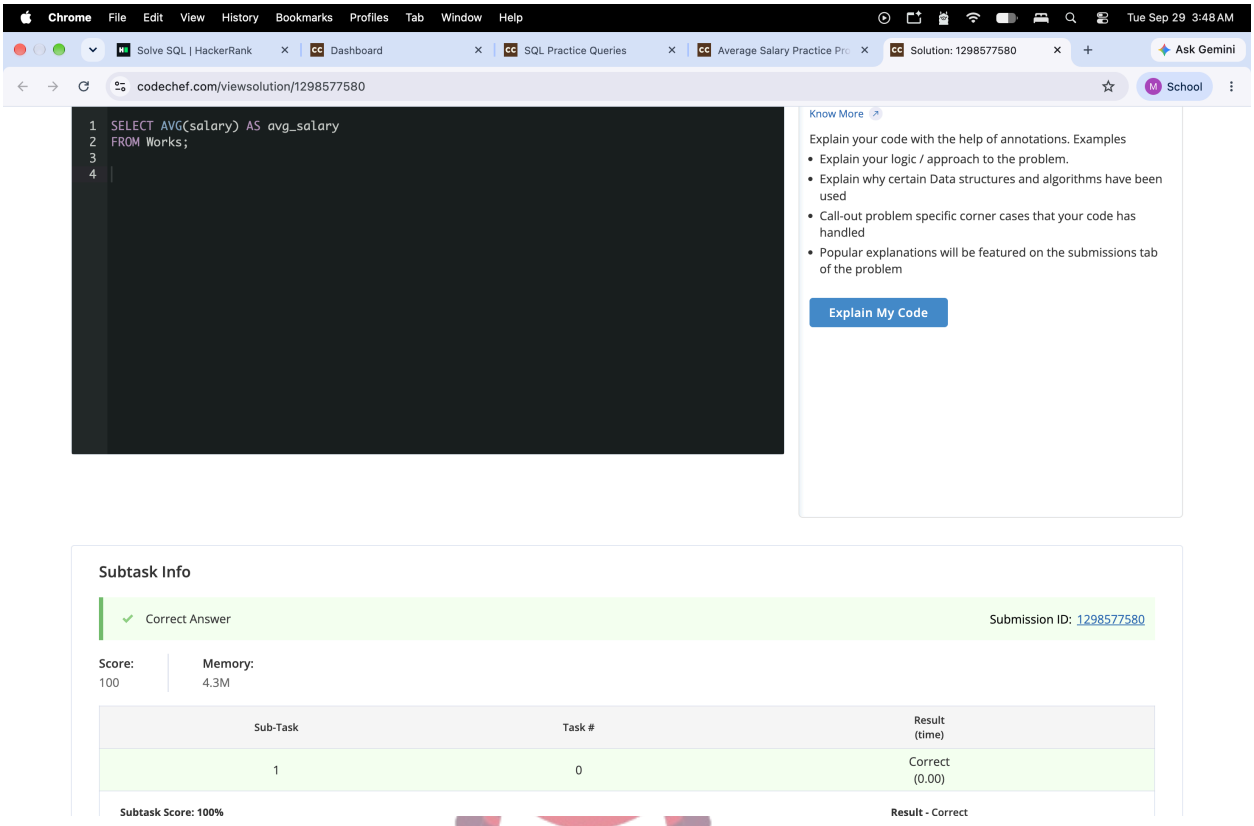


AUTONOMOUS

3. Average Salary

```
SELECT AVG(salary) AS avg_salary
FROM Works;
```

OUTPUT:

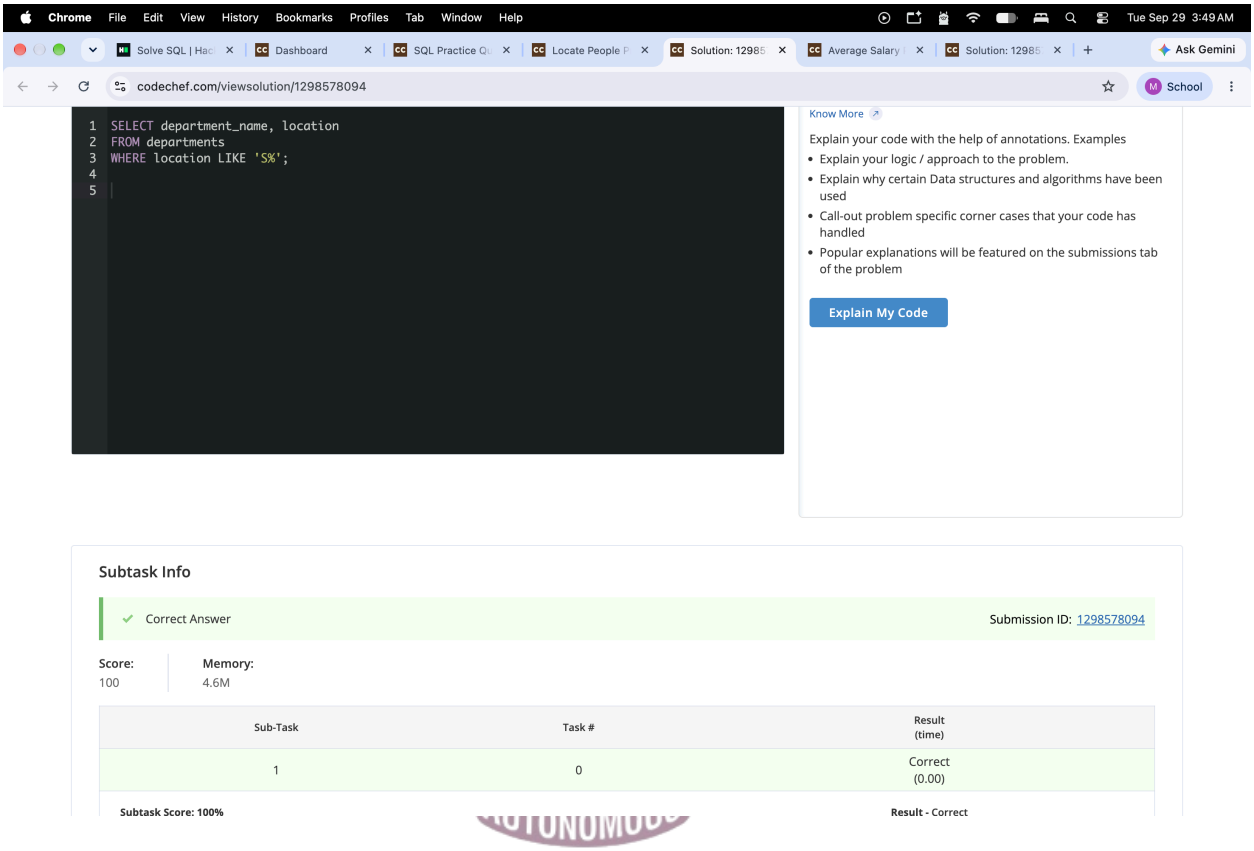


AUTONOMOUS

4. Locate People

```
SELECT department_name, location
FROM departments
WHERE location LIKE ’S%';
```

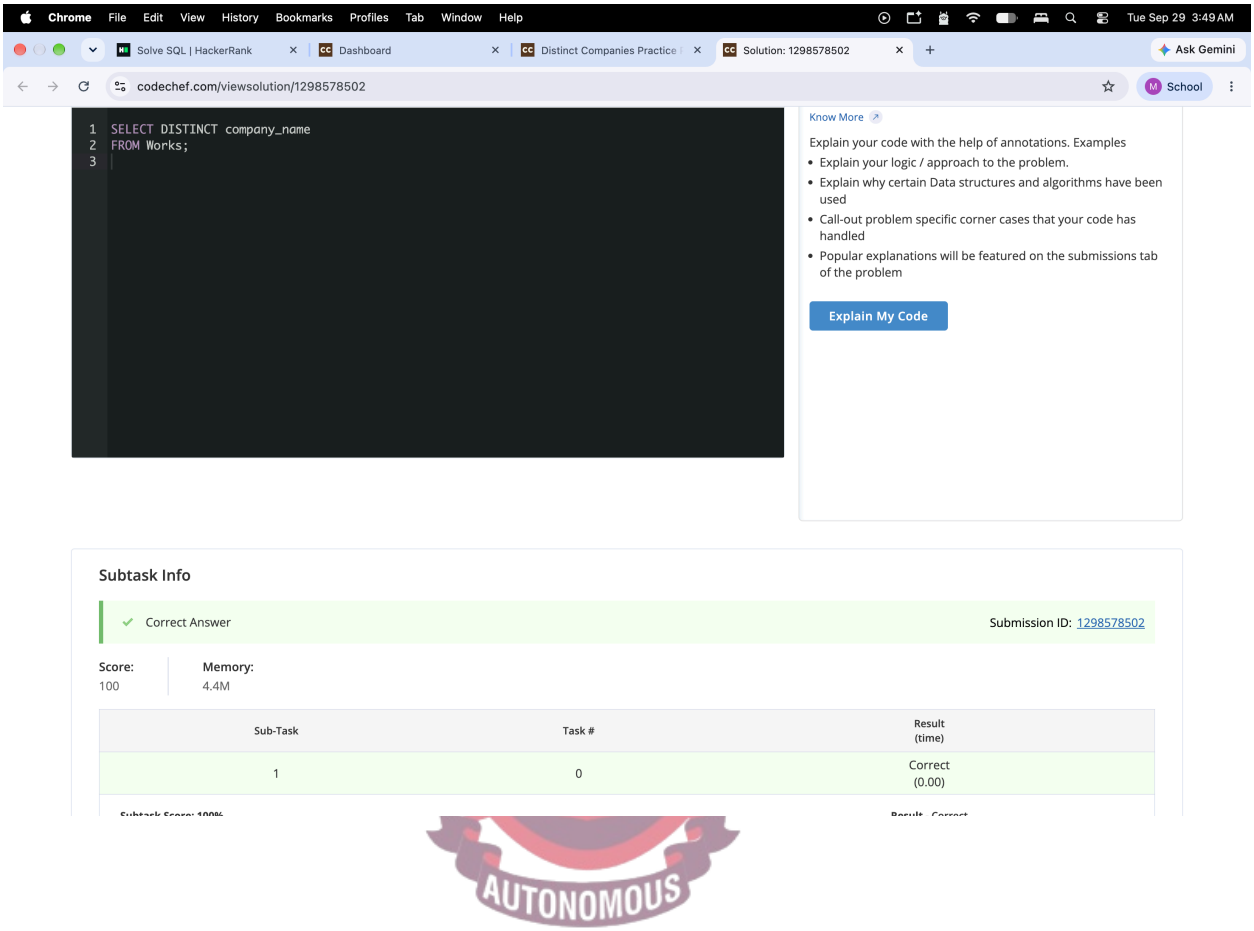
OUTPUT:



5. Distinct Companies

```
SELECT DISTINCT company_name
FROM Works;
```

OUTPUT:

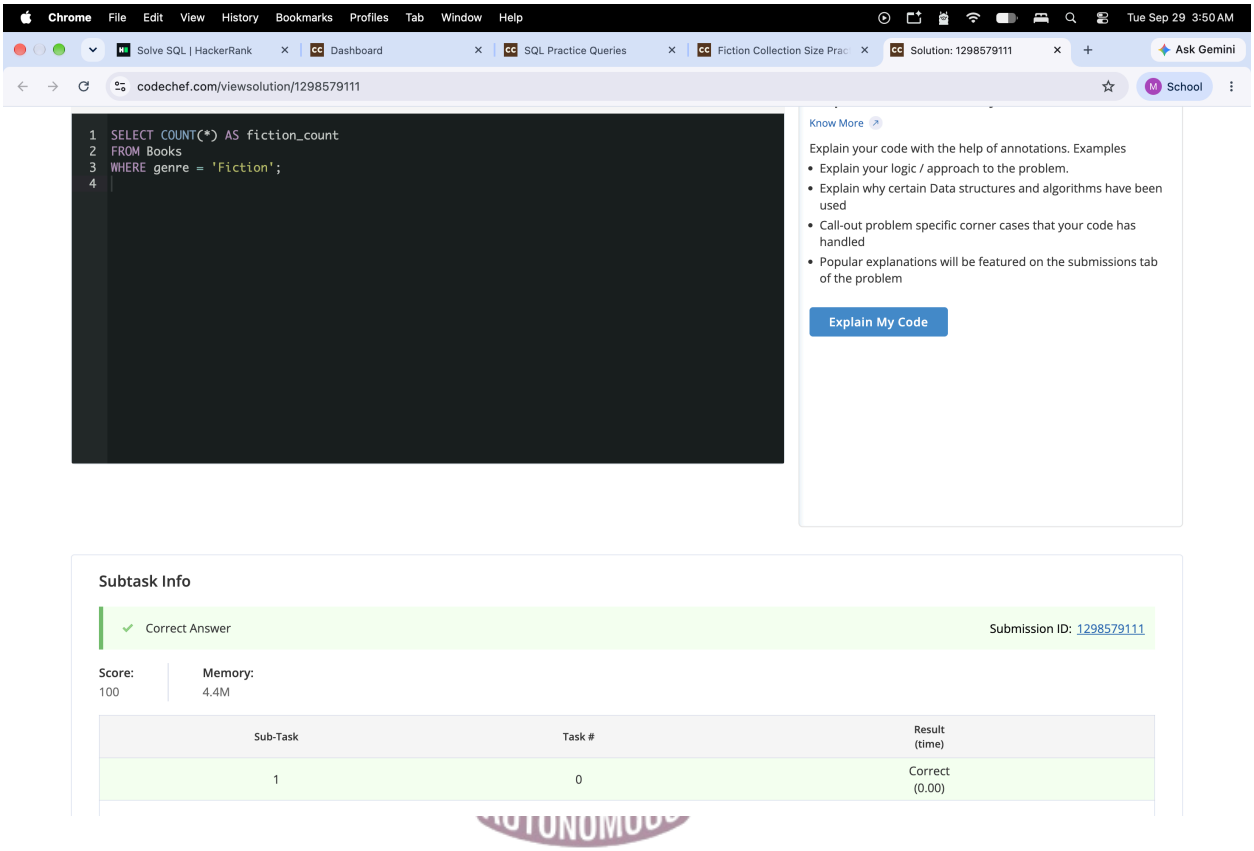


AUTONOMOUS

6. Fiction Collection Size

```
SELECT COUNT(*) AS fiction_count
FROM Books
WHERE genre = ‘Fiction';
```

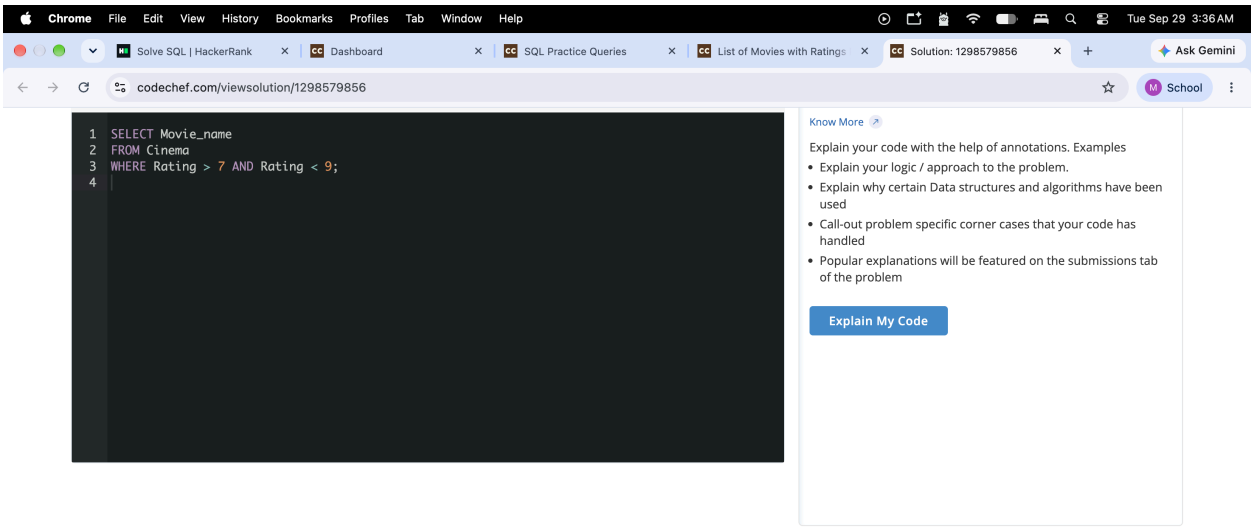
OUTPUT:

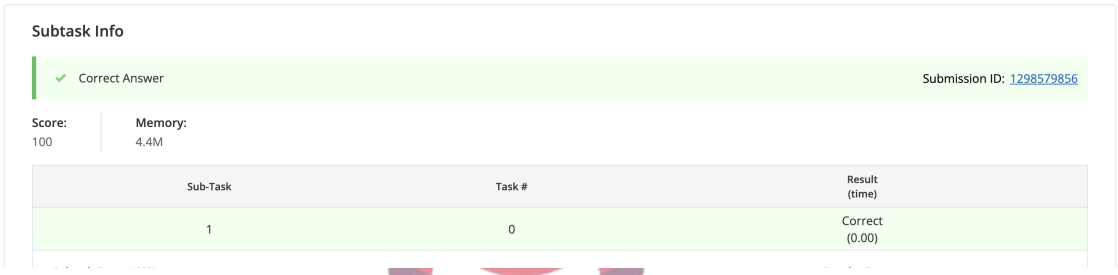


7. List of Movies with Ratings

```
SELECT Movie_name
FROM Cinema
WHERE Rating > 7 AND Rating < 9;
```

OUTPUT:



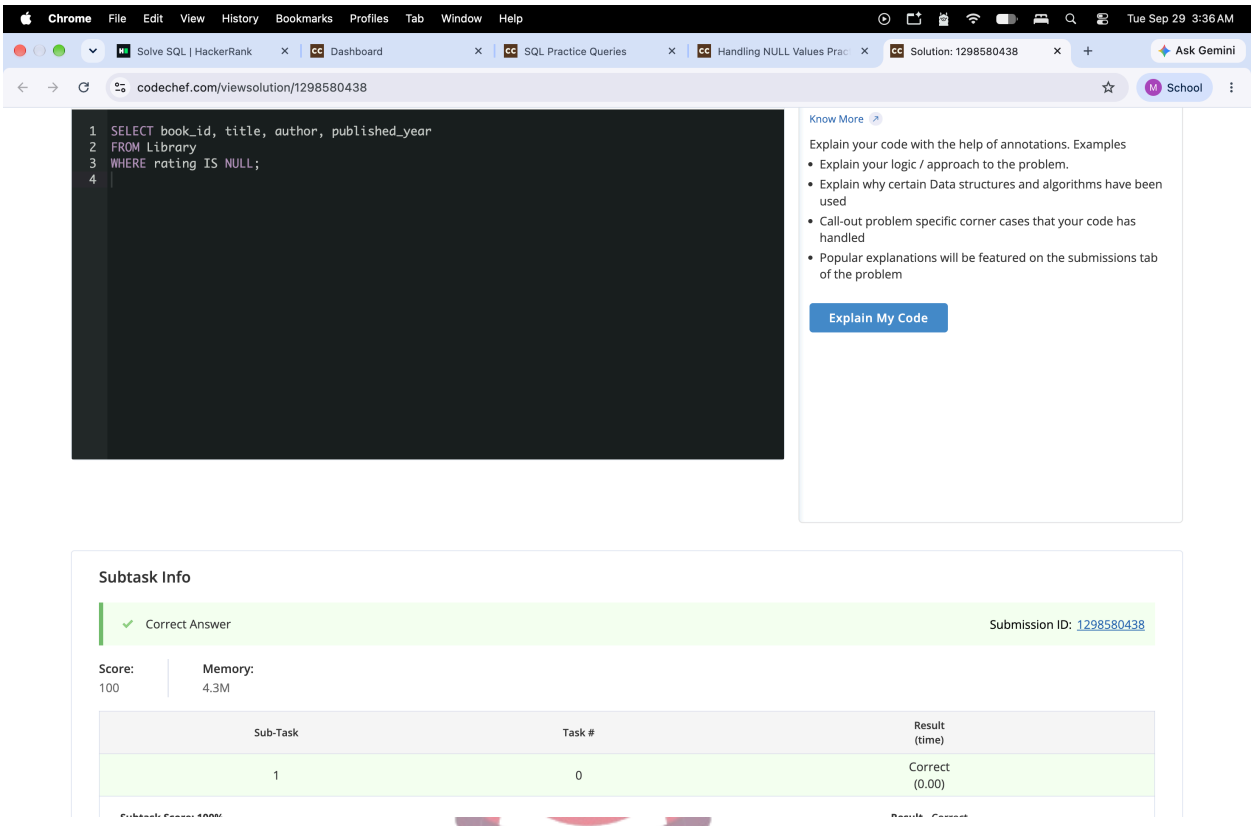


AUTONOMOUS

8. Handling NULL Values

```
SELECT book_id, title, author, published_year
FROM Library
WHERE rating IS NULL;
```

OUTPUT:

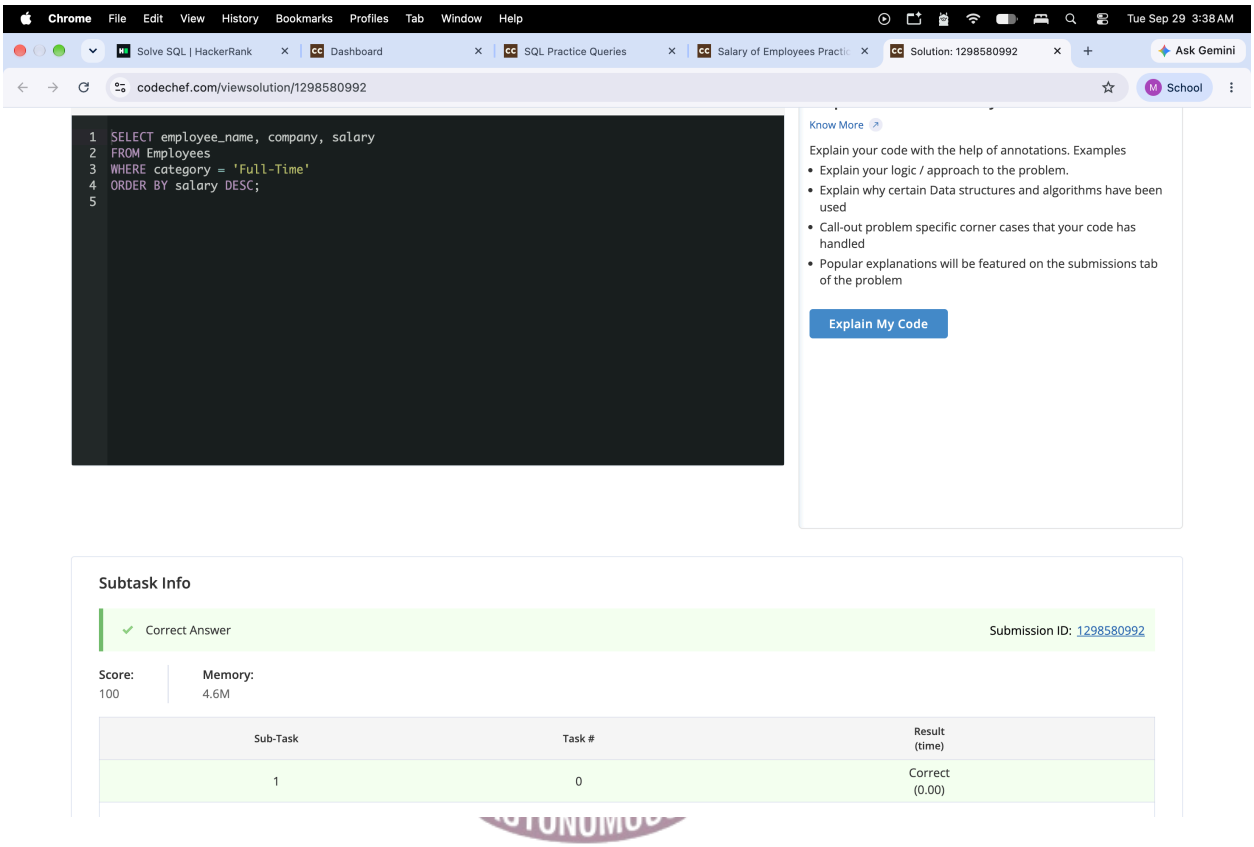


AUTONOMOUS

9. Salary of Employees

```
SELECT employee_name, company, salary
FROM Employees
WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

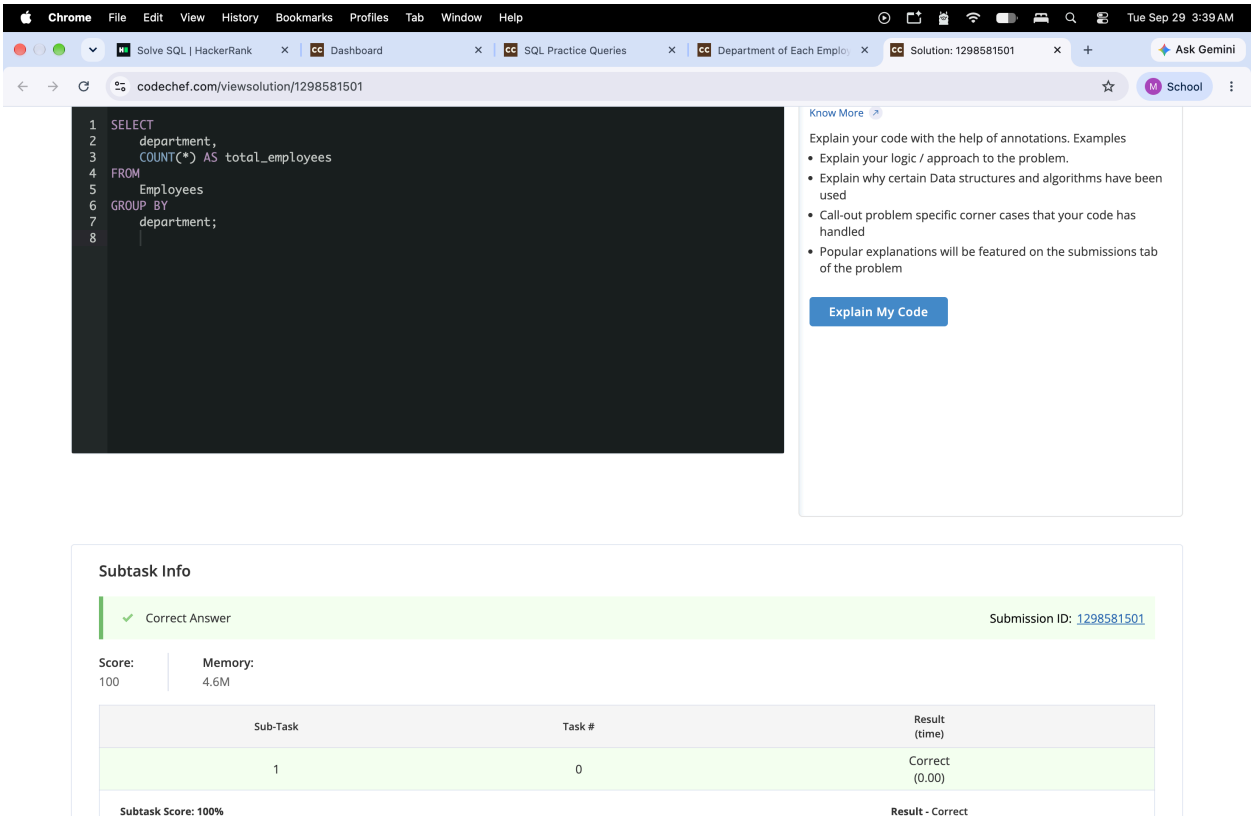
OUTPUT:



10. Department of Each Employee

```
SELECT
    department,
    COUNT(*) AS total_employees
FROM
    Employees
GROUP BY
    department;
```

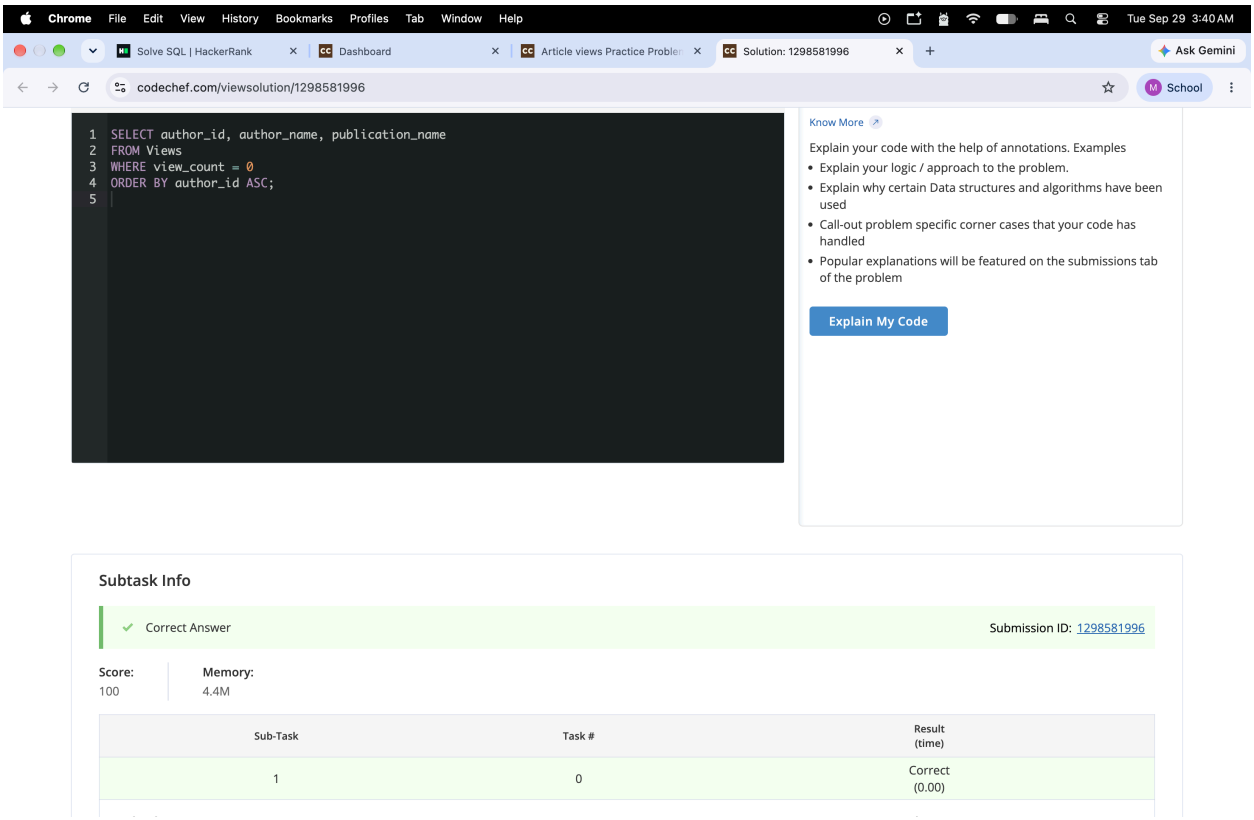
OUTPUT:



11. Article Views

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

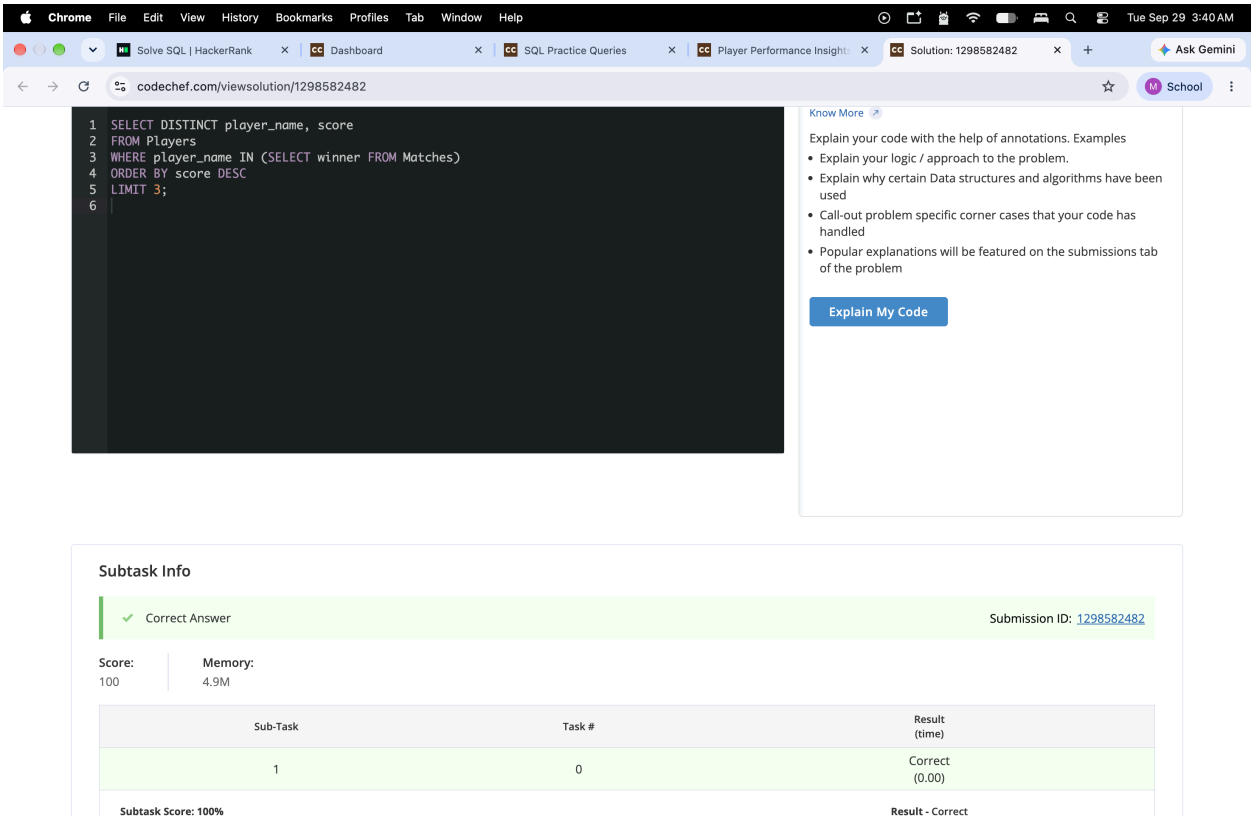
OUTPUT:



12. Player Performance Insights

```
SELECT DISTINCT player_name, score
FROM Players
WHERE player_name IN (SELECT winner FROM Matches)
ORDER BY score DESC
LIMIT 3;
```

OUTPUT:



13. Player Details

```
SELECT
    m.match_id,
    m.player_1,
    m.player_2,
    m.winner,
    m.match_date,
    p.score
FROM Matches m
JOIN Players p
    ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT: